408 详解 · 2015 年选择题 · 升级版

本册每题先给出原卷题干与选项全文（转录自公开渠道真题，来源与核对见《2015-升级日志》），再按 6 节讲解；讲解重心放在方法、推理、易混点上。

>

读法：先看标题话题与答案，再按 6 节往下读；每题自成体系，前置概念在题内讲清。

>

特别标注：第 16 题存在公开题库分歧，题内讲清推理并标注；第 34 题波形细节以原卷图为准，题末标注部分核对；第 38 题原卷路由表后两行接口印作 E3、E4，选项写作 S3、S4，属原卷印制不一致，题内如实标注，不影响结论。

升级版说明：每题标题下新增"**题干与选项：**"节（原卷题干与选项全文）；6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）。

第 1 题 · 递归函数调用时栈中函数信息的进出顺序（正确答案：A）

题干与选项：

已知程序如下：

```
int S(int n)
{ return (n<=0)?0:s(n-1)+n;}
void main()
{ cout<< S(1);}
```

程序运行时使用栈来保存调用过程的信息，自栈底到栈顶保存的信息依次对应的是 。

- A. main()→S(1)→S(0)
- B. S(0)→S(1)→main()
- C. main()→S(0)→S(1)
- D. S(1)→S(0)→main()

入手点： 先把调用顺序写出来，再按“先进栈的在底、后进栈的在顶”排一遍：谁先被调用，谁先占栈底；谁后被调用，谁压在上面。

专有名词从零解释：

- 先想“栈”：一种只能在一端进出的数据结构，像一摞盘子——后放上去的先拿下来，先放上去的最后拿下来；这就叫后进先出。
- 再想“函数调用的栈帧”：每次调用一个函数，系统就在栈里给它开一格，存返回地址、参数、局部变量这些信息；函数返回时，这一格就撤掉。
- 再想“递归调用”：函数在自己身体里又调用自己一次；每调一次，栈里就多一格，格子一层一层往上摞。

考查知识点：

- 先想调用过程：主函数 main 先运行，它调用 S(1)；S(1) 里又调用 S(0)；S(0) 满足 n≤0 的条件，直接返回 0，不再往下调。
- 再想进栈顺序：main 最先入栈，在最底下；接着 S(1) 入栈，压在 main 上面；最后 S(0) 入栈，压在最顶上。
- 再想从底到顶的顺序：main、S(1)、S(0)，就是 A。
- 拿生活例子验算：三个人排队进一个只能从上面进出的窄间，先进的 main 站最底下，后进的 S(0) 站最上面；出来时 S(0) 先出，main 最后出，顺序正好反过来。
- 边界情况：如果 S(0) 里还要再调 S(-1)，栈顶还会再多一格；本题 S(0) 直接返回，所以栈顶停在 S(0)。
- 易混对子：调用顺序 main→S(1)→S(0) 不是栈底到栈顶的倒序——调用顺序和从底到顶的顺序在这里是一致的，因为先调用的先入栈、位置更低；容易错的是把“返回顺序”当成“栈内顺序”。

正确选项为什么对： 从底到顶依次是 main、S(1)、S(0)，与 A 的顺序一致，选 A。

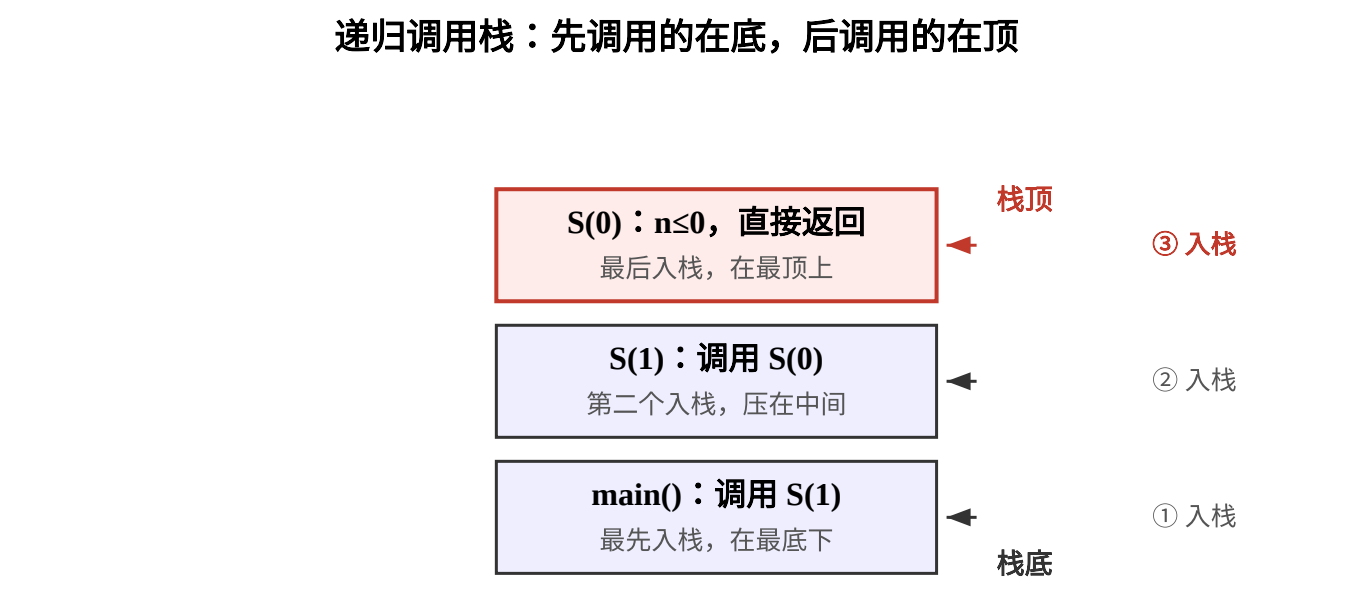
错误选项为什么错＋举例：

- B（S(0)→S(1)→main）：这是从顶到底的顺序，或者说是返回顺序；题目问的是从底到顶，所以方向反了。教训：先确认题目问的是“底→顶”还是“顶→底”。
- C（main→S(0)→S(1)）：把 S(1) 和 S(0) 的上下位置换了；S(0) 是后调用的，应该在 S(1) 上面，不是下面。教训：后调用的压在上面，先调用的在下面。

- D（S(1)→S(0)→main）：把 main 放到了顶上；main 是最先调用的，应该在最底下，不可能在顶上。教训：主函数是第一个入栈的，永远在底。

笔记本一句话： 第 1 题＋递归调用进栈：先调用的在底、后调用的在顶，main、S(1)、S(0) 从底到顶排。

【图像】



递归调用栈帧示意图：main 在底、S(1) 在中、S(0) 在顶，后调用的压在上面

【定位】 数据结构 · 第3章 栈和队列 · 递归调用时栈帧的进出顺序

【易混对比】

- "调用顺序"和"返回顺序"是两回事：调用顺序是 main→S(1)→S(0)，返回顺序正好反过来 S(0)→S(1)→main；问栈内"从底到顶"时跟调用顺序走，问"谁先返回"时跟返回顺序走。
- "栈底到栈顶"和"栈顶到栈底"是两回事：同一个栈，前者是 main→S(1)→S(0)，后者是 S(0)→S(1)→main；读题先看清方向再选字母。

谁先被调用谁先入栈、位置更低；谁后被调用谁后入栈、压在上面。从底到顶就是调用顺序 main→S(1)→S(0)，选 A。

【坑位】

别把"返回顺序"当成"栈内从底到顶的顺序"。

S(0) 最先返回，容易顺手写成 S(0)→S(1)→main。为什么错：返回时确实 S(0) 先出，但题目问的是栈里"从底到顶怎么排"，排的是入栈位置，不是出栈先后。正确做法：先写调用链，再按"先调的在底、后调的在顶"摆一遍，方向对了再对选项。

【套路】

- 第一步，写出调用链：main 调 S(1)，S(1) 调 S(0)，S(0) 直接返回。
- 第二步，按调用先后从底往上摆：main 最底，S(1) 压上，S(0) 最顶。

- 第三步，对照题目要的方向：问"底→顶"就照摆的顺序读，问"顶→底"就倒着读。

先写调用链，再按"先底后顶"摆，方向对齐再选。

【类比】

像摞盘子：main 的盘子先放桌上，S(1) 的盘子摞上去，S(0) 的盘子摞最上。收盘子时从最上面拿，S(0) 先收、main 最后收——但盘子在摞里的上下位置，从下往上数还是 main、S(1)、S(0)。术语贴回去：盘子就是"栈帧"，摞的顺序就是"从栈底到栈顶的保存顺序"。

【口诀】 递归进栈先来后到，先调居底后调居顶。

【考频】 「函数调用栈」在 2009–2022 年解析中出现 16 次 · 高频

第 2 题 · 给定先序序列时不同二叉树的计数（正确答案：B）

题干与选项：

先序序列为 a,b,c,d 的不同二叉树的个数是 。

- A. 13
- B. 14
- C. 15
- D. 16

入手点： 先定根，再分左右：先序的第一个一定是根，剩下的结点要分给左子树和右子树；左子树分到几个，右子树就分到剩下的几个；每种分法里左右各自的种数相乘，再把所有分法加起来。

专有名词从零解释：

- 先想“二叉树”：每个结点最多有两个孩子，分别叫左孩子和右孩子；左、右不能互换，换了就是另一棵树。
- 再想“先序遍历”：先访问根，再先序遍历左子树，最后先序遍历右子树；所以先序序列的第一个元素一定是整棵树的根。
- 再想“卡特兰数”：一串数，记作 C(n)，表示 n 个结点能组成多少种不同形状的二叉树；递推办法就是“根定下来，左右分开数，再乘起来求和”。

考查知识点：

- 先想根的确定：先序 a、b、c、d 里，a 是根；剩下的 b、c、d 要分给左子树和右子树。
- 再想分法枚举：左子树分 0 个、右子树分 3 个；左 1 个、右 2 个；左 2 个、右 1 个；左 3 个、右 0 个；一共 4 种分法。
- 再想每种分法的种数：空树只有 1 种；1 个结点的树只有 1 种；2 个结点的树有 2 种（根的孩子在左，或者在右）；3 个结点的树有 5 种。于是 4 种分法分别贡献 1×5＝5、1×2＝2、2×1＝2、5×1＝5。
- 再想求和：5＋2＋2＋5＝14，就是 B。
- 拿小例子验算：2 个结点先序 a、b，根是 a，b 可以在左也可以在右，共 2 种；3 个结点先序 a、b、c，分法是左 0 右 2、左 1 右 1、左 2 右 0，贡献 2＋1＋2＝5 种；规律就是这样一层一层推出来的。
- 边界情况：只给先序、不给中序时，树的形状不能唯一确定，种数大于 1；如果同时给了中序，形状就唯一确定了，种数是 1；本题只给先序，所以要计数。
- 易混对子：不要把“先序序列的排列数”当成答案——结点顺序已经固定为 a、b、c、d，变的只是形状，不是排列；也不要把左、右孩子互换当成同一种树，互换了就是不同形状。

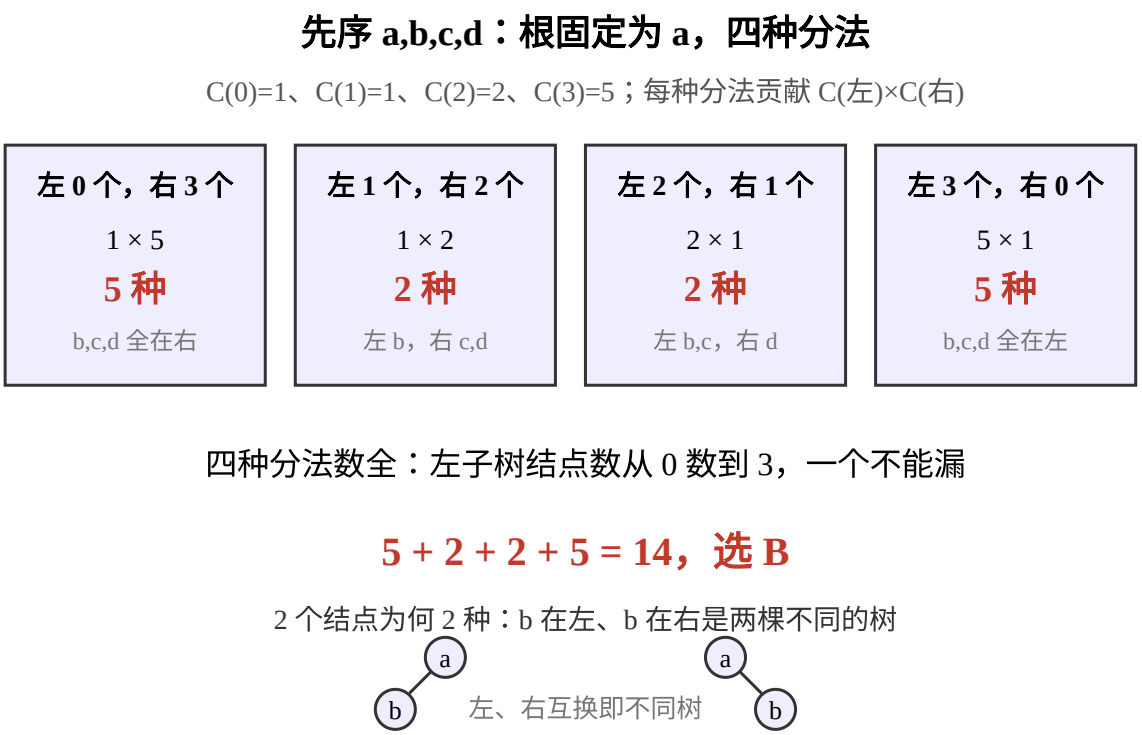
正确选项为什么对： 按根分左右的办法算出 14 种，就是 B，选 B。

错误选项为什么错＋举例：

- A（数值小于 14 的选项）：少算了分法；比如只算了“左子树为空”和“右子树为空”两种极端分法，漏了中间左 1 右 2、左 2 右 1 的分法。教训：分法要从左 0 个一直数到左 3 个，一个不能漏。
- C（数值大于 14 的选项）：多算了；比如把左、右孩子互换当成新树又重复算了一次，或者把结点排列也算了进去。教训：结点顺序固定，只数形状。
- D（数值大于 14 的另一选项）：同样是多算；比如误以为每个结点都有“在左、在右、没有”三种自由选择，把乘法用错了地方。教训：形状数要用“分左右再相乘求和”的递推，不是每个结点独立选。

笔记本一句话： 第 2 题＋只给先序数树形：根定下来，左右分开数、相乘再求和，4 个结点是 14 种。

【图像】



【定位】 数据结构 · 第5章 树与二叉树 · 给定先序序列的二叉树计数（卡特兰数）

【易混对比】

- "先序序列的个数"和"二叉树形状的个数"是两回事：序列 a、b、c、d 只有 1 种，变的是形状；答案是 14 种形状，不是序列的排列数。
- "左、右孩子互换"和"同一棵树"是两回事：左、右互换就是另一棵树，要分开数——2 个结点的树正因此有 2 种而不是 1 种。

先序第一个是根 a；b、c、d 分给左右：左 0 右 3、左 1 右 2、左 2 右 1、左 3 右 0；每种分法左右种数相乘，5+2+2+5=14，选 B。

【坑位】

四种分法要数全，漏掉"左 1 右 2"或"左 2 右 1"就少算。

最容易只数"左全空"和"右全空"两种极端，得到 5+5=10 就停手。为什么错：分法是从左 0 个连续数到左 3 个，中间两种同样合法。正确做法：写出左子树结点数 0、1、2、3 四种分法，每种算 C(左)×C(右) 再求和，小值直接背下来用。

【套路】

- 第一步，定根：先序第一个 a 是根。
- 第二步，枚举分法：左子树分 k 个（k 从 0 到 n-1），右子树分 n-1-k 个。
- 第三步，乘后求和：每种分法贡献 C(k)×C(n-1-k)，全部加起来；C(0)=1、C(1)=1、C(2)=2、C(3)=5 随手用。

定根、枚举分法、乘积求和；C(0)=1、C(1)=1、C(2)=2、C(3)=5 随手用。

【类比】

像分糖果：a 是家长固定不动，剩下 3 颗糖分给左口袋和右口袋。左口袋拿 0 颗、1 颗、2 颗、3 颗是四种分法；每个口袋里的糖还能摆出不同形状（2 颗糖有 2 种摆法，3 颗糖有 5 种）。术语贴回去：分法就是"左右子树结点分配"，摆法数就是"卡特兰数 C(n)"。

【口诀】 先序定根分左右，乘完再加卡特兰；四结点一共十四种。

【考频】 「卡特兰数」在 2009–2022 年解析中出现 1 次 · 低频

第 3 题 · 两条根到叶的权值序列能否属于同一棵哈夫曼树（正确答案：D）

题干与选项：

下列选项给出的是从根分别到达两个叶结点路径上的权值序列，能属于同一棵哈夫曼树的是 。

- A. 24,10,5 和 24,10,7
- B. 24,10,5 和 24,12,7
- C. 24,10,10 和 24,14,11
- D. 24,10,5 和 24,14,6

入手点： 先记住两条硬规矩，再一条一条验：第一，根的两个孩子的权值加起来必须等于根的权值；第二，叶子结点的兄弟权值等于父亲权值减去叶子自己的权值，而且这个兄弟权值必须大于 0。

专有名词从零解释：

- 先想“哈夫曼树”：一种带权路径长度最短的二叉树；构造办法是每次把权值最小的两个结点合并成一个父亲，父亲权值等于两个孩子权值之和，反复合并直到只剩一个根。
- 再想“权值”：每个叶子结点身上带的数字，可以理解成出现的频率或者重要程度；内部结点的权值是它下面所有叶子权值之和。
- 再想“根到叶的权值序列”：从根一路往下走到一片叶子，把路上每个结点的权值按顺序写出来；比如 24、10、5 表示根是 24，它的孩子是 10，再往下叶子是 5。

考查知识点：

- 先想第一条规矩：根是 24，它的两个孩子权值之和必须是 24；如果两条序列给出的根孩子加起来不是 24，这两条序列就不可能同属一棵树。
- 再想第二条规矩：叶子 5 的父亲是 10，那么叶子 5 的兄弟权值就是 10－5＝5，必须大于 0；如果算出来是 0 或者负数，说明这个父亲下面根本容不下这个叶子，序列就不可能。
- 再想 A 为什么不可能：两条路径共用内部结点 10，10 的孩子已经是 5＋5＝10，满了，容不下 7；第二条序列里的 7 没地方放，所以 A 不可能。
- 再想 B 为什么不可能：根孩子 10＋12＝22，不等于根 24；根的两个孩子之和对不上，第一条规矩就不过关，所以 B 不可能。
- 再想 C 为什么不可能：叶子 10 的父亲也是 10，兄弟权值是 10－10＝0；兄弟权值必须是正数，0 说明这个叶子没有兄弟，结构上不成立，所以 C 不可能。
- 再想 D 为什么可能：根孩子 10＋14＝24，对得上；叶子 5 的兄弟是 10－5＝5，大于 0；叶子 6 的兄弟是 14－6＝8，大于 0；两条规矩都通过，所以 D 可能。
- 拿小例子验算：根 10，孩子 4 和 6，两片叶子分别是 4 和 6 的孩子 1 和 3；叶子 1 的兄弟是 4－1＝3，大于 0，成立；如果叶子权值等于父亲权值，兄弟就是 0，那就不成立。
- 边界情况：哈夫曼树的每个内部结点都恰好有两个孩子，不存在只有一个孩子的情况；所以兄弟权值必须大于 0，不能等于 0。
- 易混对子：问句是“可能是”，不是“一定是”——只要找出一组满足规矩的树就算可能，不用证明所有树都行；反过来，排除一个选项只要找到一条违反的规矩就够了。

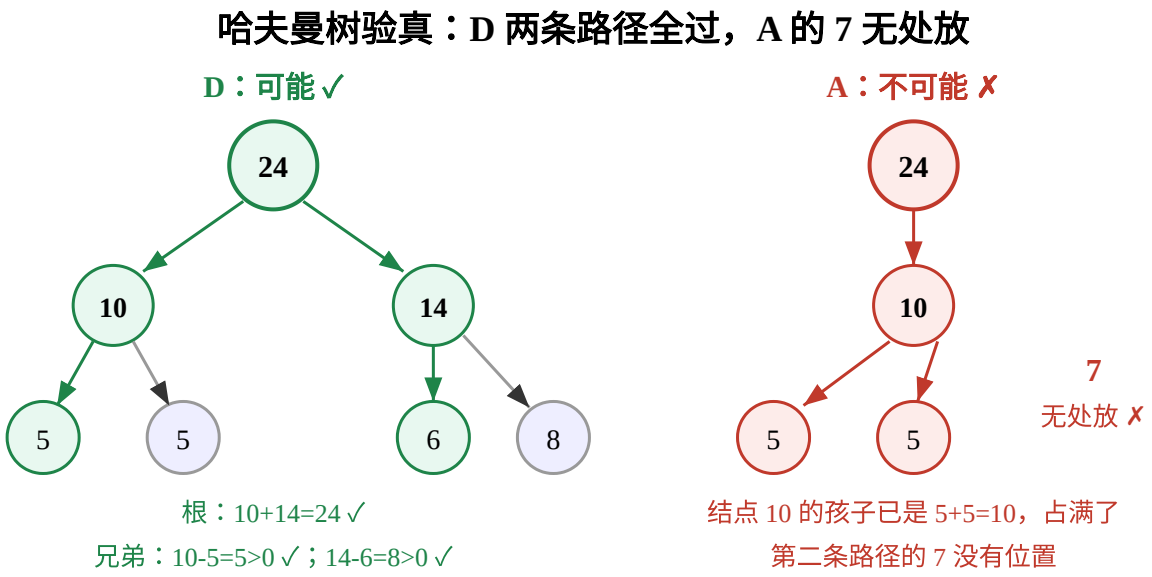
正确选项为什么对： D 的两条序列同时满足根孩子之和等于 24、兄弟权值都大于 0，所以可能同属一棵哈夫曼树，选 D。

错误选项为什么错＋举例：

- A：两条路径共用内部结点 10，10 的孩子已经是 5＋5，7 没有位置；好比一个盒子已经装满了 5＋5＝10，再塞一个 7 就超了。教训：共用结点时要把两条路径合起来看容量。
- B：根孩子 10＋12＝22，不等于根 24；根的孩子之和必须等于根，差 2 就不行。教训：先验根孩子之和，这是最快的一票否决。
- C：叶子 10 的父亲也是 10，兄弟权值是 0；兄弟不存在，结构不成立。教训：兄弟权值必须大于 0，等于 0 也不行。

笔记本一句话： 第 3 题＋哈夫曼树序列验真：根孩子之和等于根，兄弟权值＝父亲－叶子且大于 0，两条都过才可能。

【图像】



哈夫曼树验真示意图：D 的两条权值路径通过两条硬规矩检验，A 的结点 10 已被 5+5 占满、7 无处放

【定位】 数据结构 · 第5章 树与二叉树 · 哈夫曼树权值序列的合法性判断

【易混对比】

- "可能是"和"一定是"是两回事：可能是=存在一棵树容得下这两条路径；排除一个选项只需抓到一条违反的规矩，不用证死所有树。
- "兄弟权值"和"孩子权值"是两回事：兄弟权值=父亲权值减自己权值；别拿孩子权值去减，也别忘了它必须大于 0。

两条硬规矩：根的两个孩子权值之和等于根权值；任一叶子的兄弟权值=父亲权值减叶子权值，必须大于 0。D 的两条路径两条全过，选 D。

【坑位】

先验根孩子之和，这是最快的一票否决。

有人从叶子一路验上去，验到一半才发现根对不上。为什么错：顺序反了，根不过关后面全白验。正确做法：先把两条路径的根孩子加起来看等不等于 24（如 B 的 10+12=22，直接出局）；过了再验每片叶子的兄弟权值（C 的叶子 10 兄弟权值 10-10=0，出局；A 的 7 在结点 10 下无处放，出局）。

【套路】

- 第一步，验根：两条路径的根孩子权值相加，看等不等于根权值。
- 第二步，验兄弟：每个叶子算"父亲减自己"，看是否大于 0；共用内部结点时把两条路径合起来看容量。
- 第三步，看问句极性：问"可能是"，两条规矩全过即选；排除选项时，一条违反就够。

先验根和，再验兄弟差，共用结点看容量。

【类比】

像拼积木称重：根是 24 斤的大箱，两个孩子是分装的小箱，两小箱加起来必须正好 24 斤；每个小箱里，叶子占掉几斤，剩下的必须够分给它的兄弟（不能是 0 或负数，空箱子不存在）。术语贴回去：大箱=内部结点权值，叶子占的斤数=叶子权值，兄弟分到的=兄弟权值。

【口诀】 根下两孩和为根，兄弟权值父减自；大于零才算数，两条全过才能算。

【考频】 「哈夫曼树」在 2009–2022 年解析中出现 6 次 · 中频

第 4 题 · 平衡二叉树中序遍历结果及其节点性质的判断（正确答案：D）

题干与选项：

现有一棵无重复关键字的平衡二叉树（AVL 树），对其进行中序遍历可得到一个降序序列。下

列关于该平衡二叉树的叙述中，正确的是 。

- A. 根结点的度一定为 2
- B. 树中最小元素一定是叶结点
- C. 最后插入的元素一定是叶结点
- D. 树中最大元素一定是无左子树

入手点： 先把“中序从大到小”翻译成结构：正常的中序是左、根、右，得到从小到大；现在得到从大到小，说明这棵树的左、右是反过来的——左子树里的关键字都比根大，右子树里的都比根小。

专有名词从零解释：

- 先想“二叉排序树”：左子树所有关键字都比根小，右子树所有关键字都比根大；中序遍历正好得到从小到大的顺序。
- 再想“反向二叉排序树”：左子树所有关键字都比根大，右子树所有关键字都比根小；中序遍历得到从大到小的顺序；本题就是这种情况。
- 再想“平衡二叉树”：也叫 AVL 树，要求每个结点的左、右子树高度差不超过 1；插入、删除后如果失衡，要通过旋转调回平衡。
- 再想“结点的度”：一个结点有几个孩子，度就是几；叶子结点度是 0，只有一个孩子的结点度是 1，有两个孩子的结点度是 2。
- 再想“最大元素、最小元素”：最大元素就是关键字最大的结点，最小元素就是关键字最小的结点。

考查知识点：

- 先想最大元素的位置：在反向树里，越往左关键字越大，所以最大元素是最左边的结点，也就是一路往左走到头的那一个。
- 再想 D 为什么对：最大元素是最左结点；如果它还有左子树，左子树里的关键字都比它大，那就比“最大”还大，矛盾；所以最大元素一定没有左子树，D 对。
- 再想 A 为什么错：根结点的度不一定是 2；只有 1 个结点的树，根的度是 0；只有 2 个结点的树，根的度是 1；“一定为 2”太绝对了。
- 再想 B 为什么错：最小元素是最右结点；它可以带左子树——左子树里的关键字比它大，这不违反“它是最小的”，因为左子树关键字再大也只是比它大，不影响它是最小；所以最小元素不一定是叶子结点。
- 再想 C 为什么错：最后插入的元素一开始确实是叶子，但插入后可能要旋转调平衡；旋转后，这个结点可能被顶上去变成内部结点，带上孩子；所以“一定是叶子”不成立。
- 拿生活例子验算：排队时个子最高的站在最左边，如果最高的人左边还站着一个人，那个人就比他还高，矛盾；所以最高的人左边一定没人。
- 边界情况：1 个结点的树，最大元素同时也是最小元素，同时也是根；它既没有左子树也没有右子树，D 的说法照样成立。
- 易混对子：不要把正常二叉排序树的“最大元素在最右、一定没有右子树”直接搬过来；本题是反向的，最大在最左、一定没有左子树；方向反了，结论也跟着反。

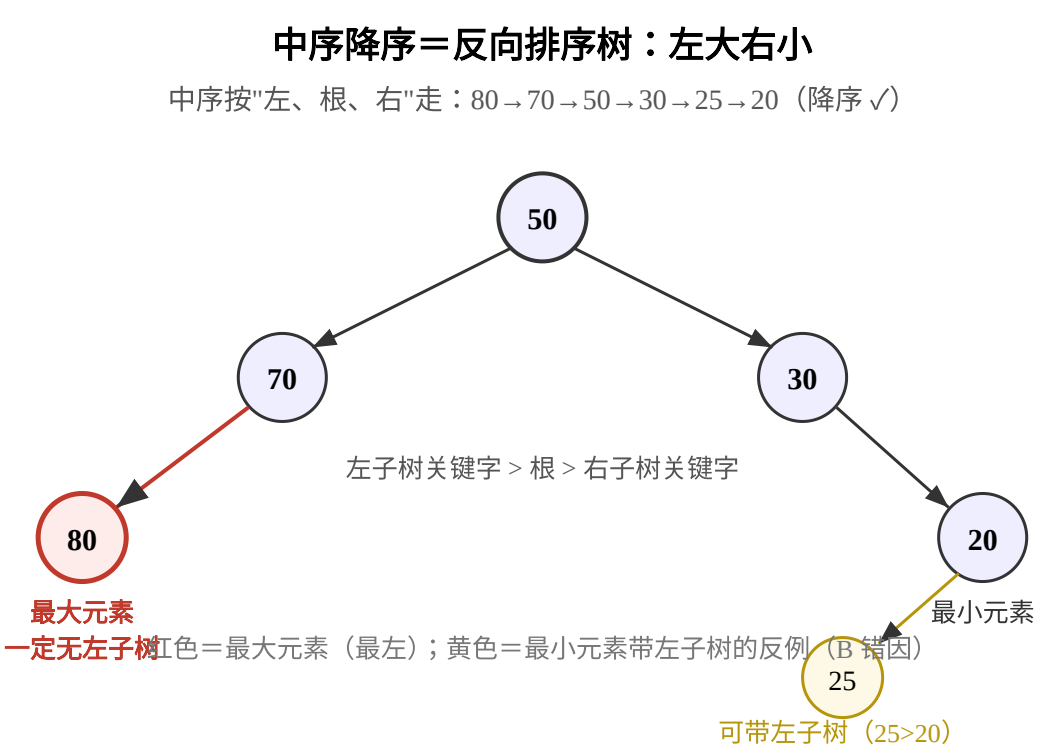
正确选项为什么对： 最大元素是最左结点，左子树若存在就会比它大，矛盾，所以最大元素一定没有左子树，选 D。

错误选项为什么错＋举例：

- A（根结点的度一定为 2）：1 个结点的树根度是 0，2 个结点的树根度是 1；“一定为 2”只在结点足够多且根有两个孩子时才成立。教训：带“一定”的说法，要先拿最小的树验一下。
- B（最小元素一定是叶结点）：最小元素是最右结点，它可以带左子树，左子树关键字比它大，不影响它最小；比如最小元素带一个左孩子，它就不是叶子。教训：最小元素只保证右子树为空，不保证左子树为空。
- C（最后插入的元素一定是叶结点）：插入后旋转可能把它顶成内部结点；比如插入后发生旋转，它下面挂上了原来的结点，就不再是叶子。教训：平衡树的旋转会改变新插入结点的位置身份。

笔记本一句话： 第 4 题＋反向排序树中序从大到小：最大在最左、一定无左子树；最小在最右、可带左子树；“一定”先拿 1～2 个结点的小树验。

【图像】



反向二叉排序树示意图：中序降序即左根右反着走，最大元素在最左且一定没有左子树

【定位】 数据结构 · 第7章 查找 · 平衡二叉树（AVL）中的反向排序结构

【易混对比】

- "正常排序树最大在右"和"反向树最大在左"是两回事：中序升序的树最大元素在最右、无右子树；中序降序的树方向全反，最大元素在最左、无左子树——结论要跟着方向反，别直接搬。
- "最小元素无右子树"和"最小元素是叶子"是两回事：反向树里最小在最右，它一定无右子树，但可以带左子树（左子树关键字更大，不影响它最小）——B 错就错在把"无右"说成了"一定是叶子"。

中序降序说明左、右反了：左子树关键字比根大。最大元素是最左结点，它若有左子树，左子树里就有比它更大的，矛盾，所以最大元素一定无左子树，选 D。

【坑位】

带"一定"的说法，先拿 1 到 2 个结点的小树验。

A 说"根的度一定为 2"：1 个结点的树根度为 0，直接反例。C 说"最后插入的元素一定是叶子"：插入后旋转可能把它顶成内部结点。为什么错：把"通常"当成了"一定"。正确做法：看到"一定"，先想最小的树能不能推翻它。

【套路】

- 第一步，翻译中序方向：降序=左、右反转的排序树。
- 第二步，定极值位置：最大在最左，最小在最右。
- 第三步，反证极值子树：最大元素的左子树若存在必有更大者，矛盾→无左子树；最小元素的右子树若存在必有更小者，矛盾→无右子树（左子树可带）。

降序即反向，最大最左无左枝，最小最右可带左。

【类比】

像按身高排队但反着站：平时"左小右大"，现在"左大右小"。最高的站在最左边；他左边要是还站着一个人，那人就比他还高，"最高"就不成立了——所以最高的人左边一定没人。术语贴回去：最高的人=最大元素，左边没人=无左子树。

【口诀】 降序中序树反向，最大最左无左枝；"一定"先拿小树验。

【考频】 「平衡二叉树」在 2009–2022 年解析中出现 8 次 · 中频

第 5 题 · 有向图从指定起点出发的不同深度优先遍历序列个数（正确答案：D）

题干与选项：

设有向图 G=(V,E)，顶点集 V={V0,V1,V2,V3}，边集 E={<v0,v1>，<v0,v2>，<v0,v3>，<v1,v3>}。若

从顶点 V0 开始对图进行深度优先遍历，则可能得到的不同遍历序列个数是 。

- A. 2
- B. 3
- C. 4
- D. 5

入手点： 按“先钻到底再回头”的规则画分支树：从 V0 出发，第一步有三个选择；每选一个，就钻到底、回头、再钻下一个没去过的邻居；把每条完整的访问顺序都数出来。

专有名词从零解释：

- 先想“有向图”：顶点之间用带箭头的边连起来，边只能顺着箭头方向走，不能逆行。
- 再想“深度优先遍历”：也叫 DFS，规则是“一条路走到黑”——从起点出发，顺着一条边走到下一个没访问过的顶点，继续往下走，直到走不动了，再退回上一个顶点，换一条没走过的边继续。
- 再想“访问序列”：按访问的先后顺序把顶点排成一串；不同的走法可能得到不同的序列。

考查知识点：

- 先想从 V0 出发的第一步：V0 的出边指向 V1、V2、V3，三个选择，分三支数。
- 再想 V0→V1 支：走到 V1，V1 指向 V3，走到 V3；V3 没有出边，回头到 V1、再回头到 V0，剩下 V2 没去，走 V2；序列是 V0、V1、V3、V2；这一支只有 1 条。
- 再想 V0→V2 支：走到 V2，V2 没有出边，回头到 V0；剩下 V1、V3 没去，先走 V1，再从 V1 走 V3，序列是 V0、V2、V1、V3；或者先走 V3，再走 V1，序列是 V0、V2、V3、V1；这一支有 2 条。
- 再想 V0→V3 支：走到 V3，V3 没有出边，回头到 V0；剩下 V1、V2 没去，先走 V1，再从 V1 走 V3（已经去过，不能再走），再走 V2，序列是 V0、V3、V1、V2；或者先走 V2，再走 V1，序列是 V0、V3、V2、V1；这一支有 2 条。
- 再想求和：1＋2＋2＝5，就是 D。
- 拿小例子验算：起点指向 A、B，A 又指向 B；从起点先走 A 的支只有 1 条（起点、A、B），先走 B 的支也只有 1 条（起点、B、A），共 2 条；规则就是“钻到底再回头”。
- 边界情况：访问过的顶点不能再访问第二次；回头时只能回头到还有没走过的出边的顶点；所有顶点都访问过，序列才算完整。
- 易混对子：不要把“邻接顶点的排列数”当成答案——DFS 的顺序受“钻到底”约束，不是所有排列都能出现；也不要漏掉“回头后换邻居”的分支。

正确选项为什么对： 三支分别是 1 条、2 条、2 条，共 5 条，就是 D，选 D。

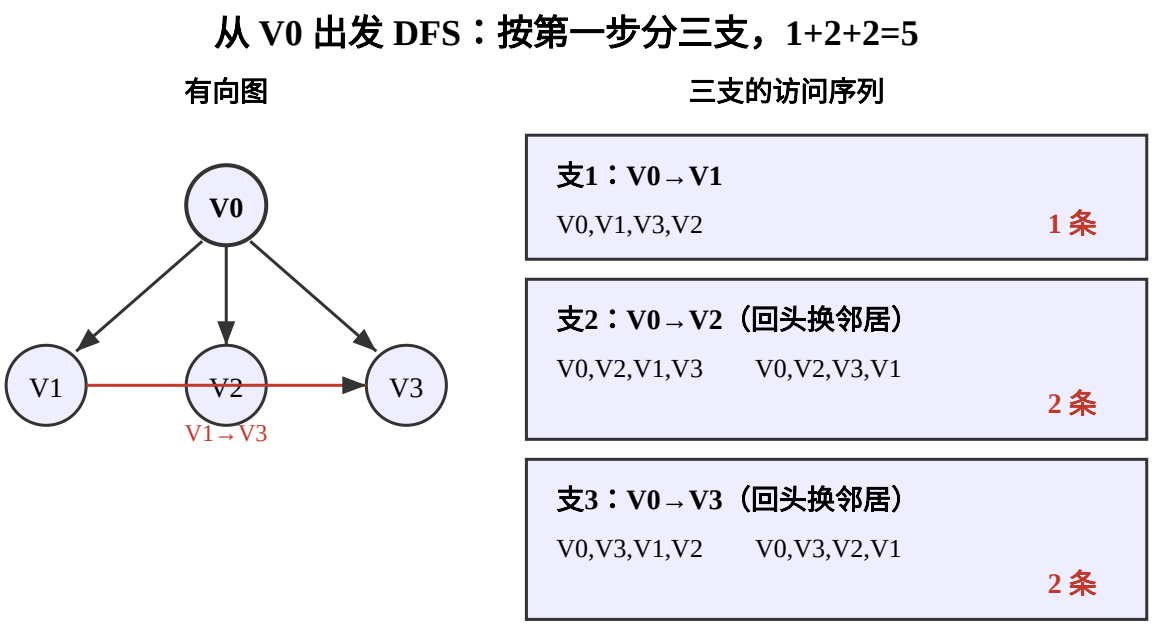
错误选项为什么错＋举例：

- A（2 条）：只数了一支，或者以为每支只有 1 条；比如只数了 V0→V1 支和 V0→V2 的一条，漏了回头换邻居的分支。教训：每支里回头后的不同选择都要数。
- B（3 条）：只按第一步的三个选择各算 1 条，漏了 V0→V2 支和 V0→V3 支里回头后的第二种走法。教训：第一步有 3 个选择，不等于总共只有 3 条序列。

- C（4 条）：漏了 1 条；比如 V0→V3 支里只数了先走 V1 的那条，漏了先走 V2 的那条。教训：每支都要把“剩下顶点的不同先后”数全。

笔记本一句话： 第 5 题＋DFS 计数：按第一步分三支，每支钻到底、回头换邻居，1＋2＋2＝5。

【图像】



有向图 DFS 分支示意图：V0 出发分三支，V0→V1 支 1 条、V0→V2 支 2 条、V0→V3 支 2 条，共 5 条

【定位】 数据结构 · 第6章 图 · 有向图深度优先遍历序列的计数

【易混对比】

- "邻接点排列数"和"DFS 序列数"是两回事：V0 的三个邻接点有 6 种排列，但 DFS 受"钻到底"约束，只有 5 种走法能真正走出来。
- "回头"和"走回头路"是两回事：回头是退回上一个顶点换条没走过的边继续，不是沿原边倒着走回去；有向边不能逆行。

按第一步分三支，每支"钻到底再回头换邻居"：V0→V1 支 1 条，V0→V2 支 2 条，V0→V3 支 2 条，1+2+2=5，选 D。

【坑位】

每支里"回头后换邻居"的分支最容易漏。

比如 V0→V2 支：到 V2 走不动，回头到 V0，剩下 V1、V3 没去——先 V1 后 V3 是一条，先 V3 后 V1 是另一条，只数一条就漏了。为什么错：把"第一步 3 个选择"当成了"总共 3 条"。正确做法：每支内部把"剩下顶点的不同先后"数全，再加总。

【套路】

- 第一步，按第一步分叉：起点 V0 的出边有几条就分几支。
- 第二步，每支钻到底：顺着边走到无路可走，记下访问顺序。

- 第三步，回头换邻居：退回上一个还有未访问邻居的顶点，把剩下的不同先后数全。
- 第四步，各支求和：1+2+2=5。

按第一步分叉，钻到底、回头换邻居，各支数全再求和。

【类比】

像逛迷宫：从入口 V0 出发有三条岔路；选一条走，走到死胡同就退回上一个路口，换条没走过的岔路继续。每条岔路走到底、退回、再换，算下来三条岔路一共能走出 5 条不同的完整路线。术语贴回去：死胡同=没有未访问出边的顶点，退回=DFS 的回溯。

【口诀】 DFS 一条路走黑，回头换道再出发；三支一二二相加，五条莫漏别落下。

【考频】 「深度优先遍历」在 2009–2022 年解析中出现 4 次 · 中频

第 6 题 · 最小生成树两种算法在某一步选中边的差别（正确答案：C）

题干与选项：

求下面带权图的最小（代价）生成树时，可能是克鲁斯卡尔（Kruskal）算法第 2 次选中但不是普里姆（Prim）算法（从 V4 开始）第 2 次选中的边是 。

- A. (V1,V3)
- B. (V1,V4)
- C. (V2,V3)
- D. (V3,V4)

> 注：原卷此处为带权图（求最小生成树），插图以原卷为准。

入手点： 两种方法各走两步写出来，再对比：第 1 步各选了什么，第 2 步的候选池分别是什么；第一种方法第 2 步能选、第二种方法第 2 步连候选资格都没有的边，就是答案。

专有名词从零解释：

- 先想“最小生成树”：把图里所有顶点连起来、用的边权值之和最小的一棵树；不能有环，边数等于顶点数减 1。
- 再想“Kruskal 算法”：也叫克鲁斯卡尔算法，做法是把所有边按权值从小到大排队，从小到大一根一根试；试的边如果和已选的边构成环就跳过，不构成环就选上；特点是“全局看边”，不管边在哪。
- 再想“Prim 算法”：也叫普里姆算法，做法是从一个起点出发，维护一个“已连通集合”；每一步只在“一端在集合内、一端在集合外”的边里选权值最小的一根，把外面的顶点拉进来；特点是“从集合往外长”。
- 再想“第几次选中”：按算法执行的先后顺序数，第一次选中的边是第 1 次，第二次选中的是第 2 次。

考查知识点：

- 先想边权表：V1-V2＝10、V1-V3＝8、V1-V4＝5、V2-V3＝8、V2-V4＝11、V3-V4＝8。
- 再想 Prim 从 V4 开始的第 1 步：集合是 {V4}，候选边是 V1-V4＝5、V2-V4＝11、V3-V4＝8；最小的是 5，选 (V1,V4)，集合变成 {V1,V4}。
- 再想 Prim 的第 2 步：候选边必须一端在 {V1,V4} 内、一端在外：(V1,V3)＝8、(V3,V4)＝8、(V1,V2)＝10、(V2,V4)＝11；(V2,V3) 两端 V2、V3 都在集合外，根本没有候选资格；所以 Prim 第 2 步绝不可能选 (V2,V3)。
- 再想 Kruskal 的第 2 步：边按权值排序，第 1 步选最小的 (V1,V4)＝5；第 2 步看权值 8 的边：(V1,V3)、(V2,V3)、(V3,V4) 都不构成环，都可能被选中；所以 (V2,V3) 是 Kruskal 可能第 2 次选中的边。
- 再想结论：(V2,V3) 是 Kruskal 可能第 2 次选中、Prim 第 2 步不可能选中的边，就是 C。
- 拿小例子验算：四个顶点排成一条线，Prim 从一端长，Kruskal 从最短边开始挑；两种方法第 2 步的候选池不一样，答案往往出在“全局最短但两端都不在 Prim 集合里”的边上。
- 边界情况：如果权值 8 的边里有一条和已选边构成环，Kruskal 会跳过它；本题权值 8 的三条边都不构成环，所以三条都有可能。
- 易混对子：不要以为两种方法每一步选的边都一样——它们只是最终总权值一样，中间步骤可以不同；也不要以为“权值最小”就是“两种方法都会选”，Prim 还要求一端在集合内。

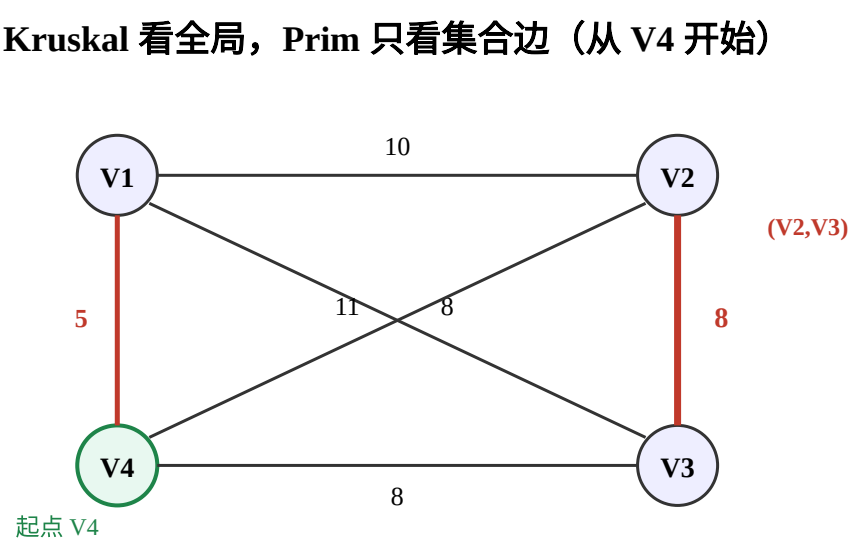
正确选项为什么对： (V2,V3) 满足“Kruskal 第 2 步可能选、Prim 第 2 步没资格选”，就是 C，选 C。

错误选项为什么错＋举例：

- A（(V1,V3)）：这条边 Kruskal 第 2 步可能选，Prim 第 2 步也可能选（它一端在 {V1,V4} 内）；题目要求“不是 Prim 第 2 次选中的”，A 不符合“不是”这个条件。教训：两个条件都要满足，缺一个就不行。
- B（(V1,V4)）：这是两种方法第 1 步都选的边，不是第 2 步的边。教训：先对齐“第几次”，第 1 步的边不能拿来答第 2 步的问。
- D（(V3,V4)）：这条边和 A 一样，Kruskal 第 2 步可能选，Prim 第 2 步也可能选（它一端在集合内）；不符合“不是 Prim 第 2 次选中”的条件。教训：Prim 第 2 步的候选池里有它，就不能选它。

笔记本一句话： 第 6 题＋Kruskal 对 Prim：Kruskal 全局按权挑，Prim 只挑一端在集合内的边；找“全局能挑、Prim 没资格”的边。

【图像】



第 1 步：双方都选 **(V1,V4)=5**，Prim 集合变为 {V1,V4}

Kruskal 第 2 步候选（全局权 8）：(V1,V3)、**(V2,V3)**、(V3,V4)

Prim 第 2 步须一端在 {V1,V4} 内：**(V2,V3) 两端都在外，没资格**

答案 C；A、D 两条边 Prim 第 2 步也有资格，不符合"不是"

最小生成树算法对比示意图：带权图标出 Kruskal 第 2 次候选边与 Prim（从 V4）第 2 步候选池，(V2,V3) 两端在集合外无资格

【定位】 数据结构 · 第6章 图 · 最小生成树两种算法的选边差异（Kruskal 与 Prim）

【易混对比】

- "全局权值最小的边"和"Prim 能选的边"是两回事：Prim 只在"一端在已连通集合内、一端在外"的边里挑；(V2,V3) 权 8 是全局最小档，但两端都不在 {V1,V4} 里，Prim 第 2 步连候选资格都没有。
- "第几次选中"和"最终选中"是两回事：(V1,V4) 是两种算法第 1 次都选的边，问第 2 次就不能拿它答；对齐"第几次"再比较。

Kruskal 第 2 步看全局：权 8 的 (V1,V3)、(V2,V3)、(V3,V4) 都可能；Prim 从 V4 第 2 步只看一端在 {V1,V4} 内的边，(V2,V3) 两端都在外，没资格。符合"可能是 Kruskal 第 2 次、不是 Prim 第 2 次"的是 (V2,V3)，选 C。

【坑位】

权值并列时 Kruskal"可能"选中，别去纠结它到底选三条里的哪一条。

有人算到 Kruskal 第 2 步三条权 8 的边都有可能，就卡住不敢选。为什么错：题目问的是"可能是……但不是……"，只要这条边在 Kruskal 的候选池里、不在 Prim 的候选池里，条件就满足了。正确做法：两边各写候选

池，找"在 Kruskal 池里、不在 Prim 池里"的边。

【套路】

- 第一步，双方各走一步：Kruskal 选全局最小 (V1,V4)=5；Prim 从 V4 选 (V1,V4)=5，集合变为 {V1,V4}。
- 第二步，写 Kruskal 第 2 步候选池：全局次小的边，不成环即可：(V1,V3)、(V2,V3)、(V3,V4)。
- 第三步，写 Prim 第 2 步候选池：一端在 {V1,V4} 内、一端在外：(V1,V3)、(V3,V4)、(V1,V2)、(V2,V4)。
- 第四步，找差集：(V2,V3) 在 Kruskal 池里、不在 Prim 池里，就是它。

两边各写候选池，找"全局能挑、Prim 没资格"的边。

【类比】

像两种招人方式：Kruskal 看全场简历，按分数从高到低挑；Prim 只从"已入职员工推荐的人"里挑。(V2,V3) 分数够高（权 8），Kruskal 那边排得上号，但它谁也不认识（两端都不在集合里），Prim 这边连面试资格都没有。术语贴回去：已入职=已连通集合，面试资格=一端在集合内的候选条件。

【口诀】 克鲁斯卡尔看全局，普里姆只认集合边；两端在外没资格，差集里面找答案。

【考频】 「最小生成树」在 2009–2022 年解析中出现 3 次 · 中频

第 7 题 · 给定的关键字序列能否成为折半查找的比较序列（正确答案：A）

题干与选项：

下列选项中，不能构成折半查找中关键字比较序列的是 。

- A. 500,200,450,180
- B. 500,450,200,180
- C. 180,500,200,450
- D. 180,200,500,450

入手点： 用“区间”验自洽：每比较一个数，就能把目标的范围缩小一截；下一个比较的数必须落在前面比较划定的范围里；只要有一个数落到范围外面，整串就不可能。

专有名词从零解释：

- 先想“折半查找”：也叫二分查找，只对排好序的表用；每次比较中间位置的关键字，比目标大就往左半边找，比目标小就往右半边找，范围每次大约砍掉一半。
- 再想“比较序列”：一次查找过程中依次比较过的关键字排成的串；比如先比 500，再比 200，再比 450，就是一串比较序列。
- 再想“自洽”：一串比较序列要能对应某个真实存在的目标；也就是说，存在一个目标值，使得按折半查找的规则走，依次比到的正是这串数。

考查知识点：

- 先想验算规则：设目标是 x；比到 500 时，如果 x＜500，范围变成“小于 500”；如果 x＞500，范围变成“大于 500”；下一个比较的数必须在这个范围里；依此类推，每一步都要落在当前范围里。
- 再想 A 为什么不可能：500→200，说明目标＜500，范围是“小于 500”；200→450，说明目标在 200 和 500 之间，范围是 (200,500)；下一步是 180，180 既不在 (200,450) 里，也不在 (450,500) 里——不管目标在 (200,450) 还是 (450,500)，180 都落不到范围里；所以 A 没有自洽的目标，不可能。
- 再想 B 为什么可能：500→450→200→180，一路都是“目标比当前数小”，范围一路缩小到“小于 200”，最后 180 落在范围里；目标比如取 100，就能走出这串比较，B 可能。
- 再想 C 为什么可能：180→500，说明目标在 180 和 500 之间；500→200，说明目标在 200 和 500 之间；200→450，说明目标在 200 和 500 之间、且在 450 的一侧；目标比如取 300，就能走出这串比较，C 可能。
- 再想 D 为什么可能：180→200，说明目标大于 200；200→500，说明目标在 200 和 500 之间；500→450，说明目标在 200 和 500 之间、且在 450 的一侧；目标比如取 300，就能走出这串比较，D 可能。
- 拿小例子验算：表是 10、20、30、40、50，比较序列 30、40、35；30→40 说明目标在 30 和 50 之间，40→35 说明目标在 30 和 40 之间，35 落在范围里，自洽；如果第三个数是 25，25 不在 (30,40) 里，就不自洽。
- 边界情况：比较序列里相邻两个数不能相等（相等意味着已经找到了，不会继续比下去）；本题四串都没有相邻相等，不用操心这一点。
- 易混对子：不要以为“序列本身是递增或递减的才可能”——折半查找的比较序列可以上下跳，只要每一步都落在前面划定的区间里就行；也不要以为“没有给表和目标就没法验”——验自洽不需要具体的表，只需要区间逻辑。

正确选项为什么对： A 的最后一个数 180 落到前面划定的区间外面，没有自洽的目标，所以 A 不能成为比较过程，题目问的就是“不能”的那一个，选 A。

错误选项为什么错＋举例：

- B（500,450,200,180）：一路往小走，范围一路缩小，180 落在“小于 200”的范围里，自洽；题目问“不能”，B 能，所以 B 不对。教训：单调递减的串不一定不可能，要验区间。
- C（180,500,200,450）：180→500 定出 (180,500)，500→200 定出 (200,500)，200→450 落在 (200,500) 里，自洽；所以 C 能，不是答案。教训：上下跳的串只要每一步落在区间里，就是可能的。
- D（180,200,500,450）：180→200 定出“大于 200”，200→500 定出 (200,500)，500→450 落在 (200,500) 里，自洽；所以 D 能，不是答案。教训：最后一个数不一定要比前一个大或小，落在区间里就行。

笔记本一句话： 第 7 题＋折半比较序列验真：每步落在前面划定的区间里才自洽，落到外面就是不可能。

【图像】



折半查找区间验真示意图：500→200→450 后目标被限定在 (200,500)，180 落在区间外，不自洽

【定位】 数据结构 · 第7章 查找 · 折半查找比较序列的合法性判断

【易混对比】

- "序列单调"和"序列自洽"是两回事：比较序列可以上下跳（如 180、500、200、450），只要每一步都落在前面划定的区间里就是合法的；单调递减也不保证合法。
- "验自洽"和"有具体表"是两回事：验自洽不需要知道表长和目标值，只需要"存在一个目标走得出这串比较"——区间逻辑够了。

设目标为 x，每比较一次就把 x 的范围缩一截，下一个比较的数必须落在这个范围里。500→200→450 把 x 限定在 (200,500)，180 落在外面，没有自洽的目标，所以 A 不能成为比较序列，选 A。

【坑位】

最后一个数最容易"看起来顺眼"就放过，区间要验到最后一步。

A 的前三个数 500、200、450 自洽，有人验到 450 就停手。为什么错：合法性要求"每一步"都落在区间里，最后一步 180 落在 (200,450) 和 (450,500) 之外，整串作废。正确做法：把区间更新到最后一个数，一个都不能跳。

【套路】

- 第一步，设目标区间：初始区间是整个范围，目标 x 待定。
- 第二步，逐个比较缩区间：比到 a，若下一个比 a 大，区间变为 (a, 右界)；若小，变为 (左界, a)。
- 第三步，验下一个数落界内外：落在区间内继续，落到外面整串判死。

设区间、步步缩、个数个验，落出界即判死。

【类比】

像猜数字游戏：心里想一个数，对方说"500？"，你说"小了"，范围变成"比 500 小"；对方说"200？"，你说"大了"，范围变成"200 到 500 之间"；对方说"450？"，你说"小了"，范围变成"450 到 500 之间"或"200 到 450 之间"；这时对方说"180？"——180 根本不在任何可能的范围里，这串猜测就不可能是真实玩出来的。术语贴回去：范围=目标所在的区间，猜测=比较序列。

【口诀】 折半比较验自洽，步步落在区间里；一数出界全盘输。

【考频】 「折半查找」在 2009–2022 年解析中出现 6 次 · 中频

第 8 题 · KMP 匹配首次失配后下一轮两个指针的位置（正确答案：C）

题干与选项：

已知字符串 S 为“abaabaabacacaabaabcc”，模式串 t 为“abaabc”。采用 KMP 算法进行匹配，第一次出现“失配”(s[i]≠t[j]) 时，i=j=5，则下次开始匹配时，i 和 j 的值分别是 。

- A. i=1,j=0
- B. i=5,j=0
- C. i=5,j=2
- D. i=6,j=2

入手点： 先找失配点，再查模式串的“前后缀”表：主串指针不动，模式串指针回到“已经匹配的那段前缀里最长的、同时也是后缀的那段”的长度。

专有名词从零解释：

- 先想“主串、模式串”：主串是被搜索的长串，模式串是要找的短串；比对就是把模式串对准主串的一段，一个字符一个字符比。
- 再想“指针 i、j”：i 指主串当前比到的位置，j 指模式串当前比到的位置；两个指针一起往后走，比对上了就都加 1，对不上了就要处理。
- 再想“next 数组”：模式串自己的一张表；next[j] 表示“模式串前 j 个字符里，最长的一段既是前缀又是后缀的长度”；失配时，模式串指针就回到 next[j]，相当于把模式串往右滑到一个“已经匹配的部分还能对上”的位置。
- 再想“前缀、后缀”：前缀是从开头开始的一段，后缀是到结尾的一段；比如“abaab”的前缀有“a”、“ab”、“aba”、“abab”，后缀有“b”、“ab”、“aab”、“baab”，既是前缀又是后缀的有“ab”，长度是 2。

考查知识点：

- 先想失配现场：主串 S＝abaabaabacaabaabcc，模式串 t＝abaabc；首次失配时 i＝j＝5，意思是前 5 个字符已经对上了，第 6 个字符对不上。
- 再想前 5 个字符是什么：t 的前 5 个字符是“abaab”；主串对应位置也是“abaab”；第 6 个字符，t 是“c”，主串是“a”，对不上，失配。
- 再想 next[5] 怎么算：看“abaab”的最长相等前后缀——前缀“ab”和后缀“ab”相等，长度 2；再长的“aba”和“aab”不相等，“abab”和“baab”不相等；所以最长是 2，next[5]＝2。
- 再想下一轮指针：主串指针 i 不动，还是 5；模式串指针 j 回到 next[5]＝2；下一轮就是 i＝5、j＝2，就是 C。
- 再想为什么 i 不动：KMP 的精髓就是主串指针不回退——已经比对过的主串字符不用再比一遍，模式串自己往右滑，滑到“已匹配部分的后缀和模式串前缀对上”的位置，继续比。
- 拿小例子验算：模式串“aab”，前 2 个字符“aa”匹配，第 3 个失配；“aa”的最长相等前后缀是“a”，长度 1，next 就是 1，主串指针不动、模式串指针回到 1；不用把主串指针退回起点重比。
- 边界情况：如果 next 值是 0，模式串指针回到 0，相当于模式串整体往右滑过已匹配段，从头开始比；如果 next 值等于已匹配长度，说明整段都是前后缀相等，滑不动，这种情况不会在失配时出现。
- 易混对子：不要把“i、j 都回到起点”当成答案——那是朴素匹配的做法，KMP 恰恰省掉了主串回退；也不要把 next 值算成“最长前缀”本身的长度以外的数，比如把“abaab”的前后缀算成 3，那是把“aba”和“aab”当成相等了，实际不相等。

正确选项为什么对： 失配时 i 不动仍是 5，j 回到 next[5]＝2，组合 i＝5、j＝2，就是 C，选 C。

错误选项为什么错＋举例：

- A（i＝1、j＝0）：这是把两个指针都退回起点附近的做法，是朴素匹配的思路；KMP 里主串指针不回退，i 不会变回 1。教训：KMP 省的就是主串回退，i 不动。
- B（i＝5、j＝0）：i 不动是对的，但 j 回到 0 是把模式串滑得太远了；“abaab”的后缀“ab”和前缀“ab”还能对上，滑到 0 就浪费了已经匹配的“ab”。教训：j 回到 next 值，不是回到 0。
- D（i＝6、j＝2）：j 回到 2 是对的，但 i 变成 6 是错的；失配位置的主串字符还没有比对成功，不能跳过去。教训：i 不动，失配点的主串字符下一轮还要参与比对。

笔记本一句话： 第 8 题＋KMP 失配：i 不动，j 回到 next 值；“abaab”的最长相等前后缀是“ab”，next＝2。

【图像】

![KMP 失配示意图：i=j=5 处失配，i 不动，j 回到 next[5]=2（"abaab" 的最长相等前后缀 "ab"）](2015-diagrams/q08-kmp-mismatch.svg)

【定位】 数据结构 · 第4章 串 · KMP 算法失配后指针的移动

【易混对比】

- "KMP 的 i"和"朴素匹配的 i"是两回事：朴素匹配失配后 i 退回起点加 1 重比；KMP 的 i 从不回退，失配点的主串字符下一轮还要参与比对。
- "j 回到 next[j]"和"j 回到 0"是两回事：next[5]=2 说明已匹配的"abaab"里后缀"ab"还能和前缀对上，滑到 2 就够了；滑到 0 等于把"ab"这段已对上的白白扔掉。

失配时 i 不动仍是 5；"abaab" 的最长相等前后缀是 "ab"，长度 2，所以 j 回到 next[5]=2；下轮 i=5、j=2，选 C。

【坑位】

i 不能变成 6：失配位置的主串字符还没比对成功，不能跳过去。

D 选项 i=6、j=2，前半对了、后半把 i 前移了一位。为什么错：i=5 处的字符和 j=2 处的字符下一轮还要比，i 前移到 6 等于跳过了这个字符。正确做法：i 纹丝不动，只动 j。

【套路】

- 第一步，定失配点：i=j=5，前 5 个字符 "abaab" 已匹配，第 6 个对不上。
- 第二步，算最长相等前后缀：看 "abaab" 的前缀和后缀，"ab"="ab" 最长，长度 2。
- 第三步，移指针：i 不动=5，j=前后缀长度=2。

i 不动，j 回前后缀长；"abaab" 回 2。

【类比】

像拉链错位：拉到一半发现上下牙对不上了（失配），不用把拉链全拉开重拉（i 不回退）；看已经拉好的那段，花纹里有一截上下是对得上的（前后缀 "ab"），把拉头滑回到那截花纹的末尾接着拉就行。术语贴回去：拉头=模式串指针 j，花纹对上的长度=next 值。

【口诀】 KMP 失配主串不动，模式回跳看前后；"abaab" 回二别回零。

【考频】 「KMP」在 2009–2022 年解析中出现 2 次 · 低频

第 9 题 · 元素移动次数与初始排列无关的排序算法（正确答案：C）

题干与选项：

下列排序算法中，元素的移动次数与关键字的初始排列次序无关的是 。

- A. 直接插入排序
- B. 起泡排序
- C. 基数排序
- D. 快速排序

入手点： 一个一个看搬动次数由什么决定：只要搬动次数跟着“逆序数”或者“划分结果”走，就和初始排列有关；只要搬动次数只跟着“元素个数”和“位数”走，就和初始排列无关。

专有名词从零解释：

- 先想“直接插入排序”：像整理扑克牌，每摸一张新牌，就把它插进已经排好的牌里；插的时候要把比它大的牌往后挪，挪几次取决于新牌前面有多少张比它大的牌。
- 再想“起泡排序”：也叫冒泡排序，相邻两个元素比大小，顺序不对就交换；交换几次取决于原来有多少对“顺序颠倒”的元素。
- 再想“基数排序”：不直接比大小，而是按位分桶——先按个位把元素分到 10 个桶里，再按十位分，再按百位分，每轮分一次、收一次；分几轮取决于最大的数有几位，收几次取决于元素个数。
- 再想“快速排序”：挑一个基准，把比基准小的放左边、大的放右边，再对左右两边分别重复；搬动次数取决于每次挑的基准把序列分成几比几，而这跟初始排列有关。
- 再想“逆序数”：一串数里，前面比后面大的数对有多少对；比如 3、1、2 里，3 比 1 大、3 比 2 大，逆序数是 2；逆序数越大，序列越乱。

考查知识点：

- 先想基数排序为什么无关：每轮分配收集，每个元素都要被搬一次；轮数由最大数的位数决定，元素个数固定、位数固定，搬动总次数就固定；不管原来怎么排，每轮每个元素都搬一次，总次数不变，所以 C 对。
- 再想直接插入排序为什么有关：每插入一个元素，要挪动的次数等于它前面比它大的元素个数，也就是它带来的逆序数；原来越乱，逆序数越大，挪动越多；原来已经排好，挪动是 0；所以有关。
- 再想起泡排序为什么有关：每次交换都是一对逆序；总交换次数等于逆序数；原来已经排好，交换 0 次；原来全倒着排，交换次数最多；所以有关。
- 再想快速排序为什么有关：划分结果取决于基准的位置；基准如果正好在中间，两边均匀，搬动少；基准如果在边上，一边几乎是空的，搬动多；基准位置跟初始排列有关，所以搬动次数也有关。
- 拿小例子验算：三个数 1、2、3 和 3、2、1，基数排序都是按个位分一轮、收一轮，搬动次数一样；直接插入排序前者搬 0 次，后者搬 3 次，不一样。
- 边界情况：基数排序的“无关”指的是移动次数无关，不是比较次数无关——基数排序本来就不怎么比大小；也要注意，基数排序要求关键字能按位拆分，比如整数。
- 易混对子：不要把“时间复杂度稳定”和“移动次数无关”混为一谈——归并排序时间稳定，但移动次数也固定，不过本题选项里没有归并排序；也不要以为“起泡排序加了提前结束标志就无关了”——提前结束只是让“已经排好”的情况少搬，搬动次数照样跟着初始排列走。

正确选项为什么对： 基数排序按位分配收集，移动次数只由元素个数和位数决定，与初始排列无关，选 C。

错误选项为什么错＋举例：

- A（直接插入排序）：插入时要挪动的次数等于新元素前面比它大的个数；1、2、3 搬 0 次，3、2、1 搬 3 次，搬动次数跟着初始排列变。教训：插入类排序的移动次数跟着逆序数走。
- B（起泡排序）：交换次数等于逆序数；1、2、3 交换 0 次，3、2、1 交换 3 次，交换次数跟着初始排列变。教训：交换类排序的交换次数跟着逆序数走。
- D（快速排序）：划分结果取决于基准位置；基准在中间和在边上，搬动次数差很多；基准位置跟初始排列有关，所以搬动次数也有关。教训：划分结果不固定，搬动次数就不固定。

笔记本一句话： 第 9 题＋移动次数与初始排列无关：基数排序按位分桶收桶，次数只看个数和位数；插入、起泡看逆序数，快排看划分。

【图像】

移动次数与初始排列：只有基数排序无关

排序算法	移动次数与初排	由什么决定
直接插入排序	有关	逆序数
起泡排序	有关	逆序数
基数排序	**无关**	元素个数和位数
快速排序	有关	划分结果

基数排序：每轮每个元素分配、收集各搬 1 次，轮数=最大数的位数

个数、位数固定 → 搬动总次数固定 → 选 C

红色行＝答案；"无关"指移动次数，不管原来怎么排

排序移动次数与初始排列关系表：直接插入、起泡、快速有关，基数排序无关（红标）

【定位】 数据结构 · 第8章 排序 · 各排序算法移动次数与初始排列的关系

【易混对比】

- "时间复杂度稳定"和"移动次数无关"是两回事：归并排序时间稳定、移动次数也固定，但本题选项里没有它；别拿"稳定"去套"移动无关"。
- "比较次数无关"和"移动次数无关"是两回事：基数排序本来就不怎么比大小，题目问的是"搬动元素的次数"，只看搬了多少次。

基数排序按位分配、收集：每轮每个元素搬一次，轮数由最大数的位数定；个数和位数固定，搬动总次数就固定，和原来怎么排没关系，选 C。

【坑位】

起泡排序加了"提前结束"标志，也还是有关。

有人觉得加了标志、排好就提前停，搬动次数就"固定"了。为什么错：提前结束只是让"本来就有序"的情况少搬，搬动次数照样跟着初始排列变（有序搬 0 次，逆序搬最多）。正确做法：看搬动次数由什么决定——跟着逆序数或划分走的，都有关。

【类比】

像按楼层分信：不管信原来在袋子里多乱，分拣员每轮把每封信按个位、十位、百位各分一次、收一次；每封信被经手次数=位数×2，和原来乱不乱没关系。而插牌（直接插入）、冒泡换位，牌越乱搬得越多。术语贴回去：分信=基数排序的分配收集，经手次数=移动次数。

【口诀】 基数分桶按位走，搬动次数看位头；插入起泡看逆序，快排划分看基准。

【考频】 「基数排序」在 2009–2022 年解析中出现 6 次 · 中频

第 10 题 · 小根堆删除堆顶关键字后重建过程中比较次数（正确答案：C）

题干与选项：

已知小根堆为 8,15,10,21,34,16,12，删除关键字 8 之后需重建堆，在此过程中，关键字之间的

比较次数是 。

- A. 1
- B. 2
- C. 3
- D. 4

入手点： 按“末尾补顶、往下沉”的步骤走：把最后一个元素提到堆顶，再让它一层一层往下沉；每沉一层，先比两个孩子挑小的，再比父子决定换不换；两次比较都要数。

专有名词从零解释：

- 先想“小根堆”：一种完全二叉树，父亲的关键字一定不比孩子大；所以堆顶就是最小的元素，删堆顶就是删最小的。
- 再想“完全二叉树”：除了最后一层，其他层都是满的；最后一层的结点都靠左排，没有空洞；用数组存时，结点 i 的孩子就是 2i＋1 和 2i＋2（下标从 0 开始时）。
- 再想“删除堆顶”：分两步——先把最后一个元素移到堆顶（补上空位），再让这个元素往下沉，直到父亲不比孩子大为止；这个过程叫向下调整。
- 再想“比较次数”：每次“两个孩子比大小”算 1 次，每次“父亲和较小的孩子比大小”也算 1 次；两种比较都要数，不能只数一种。

考查知识点：

- 先想初始堆：数组是 8、15、10、21、34、16、12；堆顶 8 删掉，末尾元素 12 移到堆顶，堆变成 12、15、10、21、34、16。
- 再想第一层：12 的孩子是 15 和 10；先比两个孩子，15 比 10 大，挑小的 10，这是第 1 次比较；再比父亲 12 和孩子 10，12 比 10 大，要交换，这是第 2 次比较；交换后 12 落到原来 10 的位置。
- 再想第二层：12 现在的孩子只有 16（另一个孩子位置已经超出堆的范围）；比父亲 12 和孩子 16，12 比 16 小，不用交换，这是第 3 次比较；调整结束。
- 再想总数：1＋1＋1＝3 次，就是 C。
- 拿小例子验算：堆 5、9、6，删 5，末尾 6 补顶；孩子只有 9，比 6 和 9，6 小，不交换，比较 1 次；如果孩子是两个，就要先比孩子再比父子，比较 2 次起步。
- 边界情况：如果移到堆顶的元素本来就是最小的，第一层比一次父子就停了，总次数就是 1 次；如果它一路沉到底，每层都要比两次（最后一层只有一个孩子时只比一次）。
- 易混对子：不要只数“父子比较”漏掉“孩子之间比较”——本题孩子之间比了 1 次，漏了它就会算成 2 次；也不要把“交换次数”当成“比较次数”——本题交换 1 次，比较 3 次，两者不一样。

正确选项为什么对： 孩子之间比 1 次、父子比 2 次，共 3 次，就是 C，选 C。

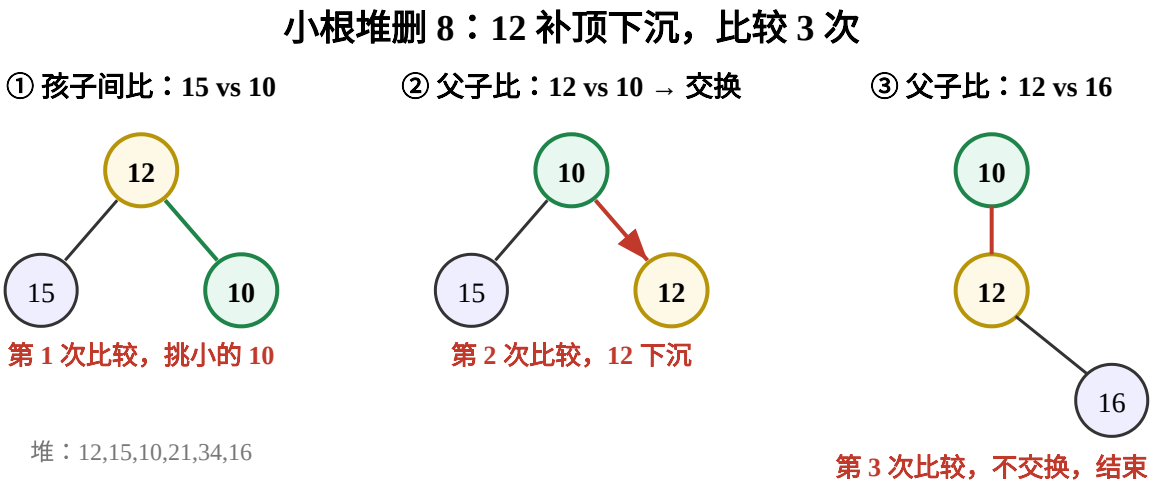
错误选项为什么错＋举例：

- A（1 次）：只数了最后一次父子比较，或者以为移到堆顶的元素一次就到位；漏了孩子之间比大小和第一次父子比较。教训：每层两种比较都要数。

- B（2 次）：数了孩子之间 1 次和第一次父子 1 次，漏了第二层 12 和 16 的父子比较。教训：沉到新位置后，还要和新位置的孩子再比一次。
- D（4 次）：多算了 1 次；比如把“12 和 16 比完不交换”之后又多算了一次不存在的比较，或者把交换本身也算成一次比较。教训：比较次数只数“比大小”，交换不算比较。

笔记本一句话： 第 10 题＋小根堆删顶：末尾补顶往下沉，每层先比孩子挑小的、再比父子，两种比较都数；本题 1＋2＝3 次。

【图像】



每层两种比较都要数：先比孩子挑小的，再比父子；一个孩子时只比父子

1 + 1 + 1 = 3 次，选 C

红色数字＝比较次数；比较≠交换（本题交换 1 次，比较 3 次）

小根堆删除堆顶示意图：12 补顶后下沉，孩子间比 1 次、父子比 2 次，共 3 次

【定位】 数据结构 · 第8章 排序 · 小根堆删除堆顶后的向下调整与比较计数

【易混对比】

- "比较次数"和"交换次数"是两回事：本题交换 1 次，比较 3 次；数的是"比大小"的次数，不是"换位置"的次数。
- "孩子之间比"和"父子比"是两回事：每沉一层，先比两个孩子挑小的（1 次），再比父亲和较小的孩子（1 次）；只数后者会漏掉前者。

12 补到堆顶往下沉：第一层孩子 15 和 10 比 1 次挑 10，父子 12 和 10 比 1 次交换；第二层 12 和 16 比 1 次不交换；共 3 次，选 C。

【坑位】

只有一个孩子的层，只比 1 次父子，没有"孩子之间比"。

第二层 12 的孩子只有 16，有人按"每层 2 次"套成 4 次。为什么错：孩子之间比大小的前提是有两个孩子，一个孩子时这一步不存在。正确做法：每层先看孩子个数：两个孩子=先比孩子再比父子，一个孩子=只比父子。

【套路】

- 第一步，末尾补顶：删 8，把末尾 12 移到堆顶。

- 第二步，逐层下沉：每层先比两个孩子挑小的（1 次），再比父亲和它（1 次）；父亲大就交换、继续沉，否则停。
- 第三步，两种比较都数：孩子间比较+父子比较，加总。

末尾补顶往下沉，孩子先比挑小者，再与父亲比高低，两种比较都要数。

【类比】

像擂台守擂：12 空降到擂主位（堆顶），两个挑战者 15 和 10 先打一场（孩子间比 1 次），赢的 10 再挑战擂主 12（父子比 1 次），12 输了让位，掉到下一层；新一层只有一个挑战者 16，直接和 12 打（父子比 1 次），12 赢了守住，比赛结束。术语贴回去：擂主=待调整的父结点，挑战者=孩子结点，让位=交换下沉。

【口诀】 删顶补尾往下沉，孩子先比挑小者；再与父亲比高低，两种比较都要数。

【考频】 「小根堆」在 2009–2022 年解析中出现 16 次 · 高频

第 11 题 · 希尔排序中组内采用的排序方法（正确答案：A）

题干与选项：

希尔排序的组内排序采用的是 。

- A. 直接插入排序
- B. 折半插入排序
- C. 快速排序
- D. 归并排序

入手点： 回到希尔排序的定义：它就是“分组的直接插入排序”——先按间隔把元素分成几组，每组内部用直接插入排序排好，再把间隔缩小，重复，直到间隔变成 1。

专有名词从零解释：

- 先想“希尔排序”：也叫缩小增量排序；先取一个比较大的间隔，把序列分成几组（间隔相同的元素一组），每组内部排序；再把间隔缩小，分组变少、每组元素变多，再排；最后间隔变成 1，就是一次普通的直接插入排序，这时序列已经基本有序，插入很快。
- 再想“直接插入排序”：每摸一张新牌，就把它插进已经排好的牌里；插的时候把比它大的牌往后挪，直到找到合适的位置。
- 再想“折半插入排序”：和直接插入排序一样是插入，只是找插入位置时用折半查找找得快一点；找得快不等于挪得少，挪动元素的次数和直接插入排序一样。

考查知识点：

- 先想希尔排序的定义：组内用的就是直接插入排序，这是定义本身，不是推导出来的；所以 A 对。
- 再想为什么不是折半插入：折半插入虽然也是插入类，但希尔排序的原始定义和教材说法都是“组内用直接插入排序”；把组内方法换成折半插入，就不是标准希尔排序了。
- 再想为什么不是快速排序：快速排序是划分、递归的做法，和“分组后组内插入”的思路完全不同；希尔排序的组内排序是插入，不是划分。
- 再想为什么不是归并排序：归并排序是把两段有序段合并成一段的做法，组内排序用不上合并；希尔排序的组内排序是插入，不是合并。
- 拿生活例子验算：把一副牌按花色分成四组，每组内部按点数插排；分组插排就是希尔排序的组内做法；如果组内用“挑基准分两边”的办法，那就不是插排了。
- 边界情况：间隔变成 1 时，希尔排序退化成一次普通的直接插入排序；这时整个序列已经基本有序，插入的挪动很少，这就是希尔排序比直接插入排序快的原因。
- 易混对子：不要把“折半插入也是插入”当成“希尔排序组内就是折半插入”——插入类里有很多种，希尔排序指定的是直接插入；也不要把“希尔排序快”理解成“组内用了快排”，快的原因是间隔由大到小、序列越来越有序。

正确选项为什么对： 希尔排序的定义就是分组后组内用直接插入排序，选 A。

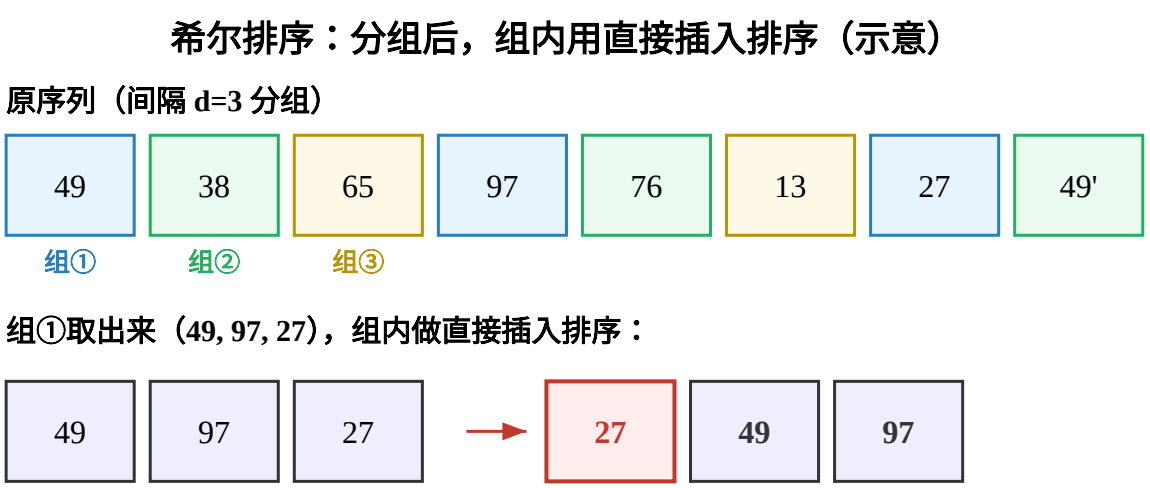
错误选项为什么错＋举例：

- B（折半插入排序）：折半插入只是找位置快，挪动次数和直接插入一样；但希尔排序的定义写的是直接插入，不是折半插入。教训：定义题按定义答，不要按“差不多”答。

- C（快速排序）：快速排序是划分的做法，和组内插入的思路不同；希尔排序没有“挑基准分两边”这一步。教训：组内排序的方法要和整体思路一致。
- D（归并排序）：归并排序是合并两段有序段的做法，组内排序用不上合并；希尔排序的组内是插入，不是合并。教训：合并和插入是两种不同的排序思路。

笔记本一句话： 第 11 题＋希尔排序组内方法：分组后组内就是直接插入排序，定义题按定义答。

【图像】



希尔排序分组组内直接插入排序示意：原序列按间隔3分成三组，组①的49、97、27取出做直接插入排成27、49、97

【定位】 数据结构 · 第8章 排序 · 希尔排序组内排序方法

【易混对比】

- "直接插入排序"和"折半插入排序"是两回事：一个顺序找插入位置、一个折半找插入位置，但希尔排序的定义里组内写的是直接插入——定义题按定义答，不按"差不多"答。
- "希尔排序快"和"组内用了快排"是两回事：快的原因是间隔由大到小、序列越来越有序，最后间隔为1时插入几乎不用挪动，不是组内换了快速排序。
- "找位置快"和"挪动少"是两回事：折半插入只是找位置快，挪动元素的次数和直接插入排序一样，所以"快"不能当成"组内是折半插入"的理由。

希尔排序就是"分组的直接插入排序"——按间隔分组，每组内部用直接插入排序排好；组内方法选直接插入排序，选A。

【坑位】

别把"折半插入也是插入"当成"希尔排序组内就是折半插入"。

插入类里有很多种，直接插入、折半插入都是插入，但希尔排序的原始定义和教材说法指定的是直接插入排序。看到"组内用哪种"，第一反应应该是"定义怎么写的"，而不是"哪种插入更快"。正确做法：定义题按定义答，希尔等于分组加组内直接插入。

【类比】

像把一副扑克按花色分成四组，每组内部按点数一张一张插排——分组插排就是希尔排序的组内做法。如果组内改成"挑一张基准分两边"，那就不是插排了。术语贴回去：希尔排序的组内排序方法就是直接插入排序。

【口诀】 希尔组内插排法，定义原样不掺假。

【考频】 「希尔排序」在 2009–2022 年解析中出现 7 次 · 中频

第 12 题 · 计算机硬件能够直接执行的程序类型（正确答案：A）

题干与选项：

计算机硬件能够直接执行的是 。

Ⅰ．机器语言程序 Ⅱ．汇编语言程序 Ⅲ．硬件描述语言程序

- A. 仅 Ⅰ
- B. 仅 Ⅰ、Ⅱ
- C. 仅 Ⅰ、Ⅲ
- D. Ⅰ、Ⅱ、Ⅲ

入手点： 先想硬件认识什么：硬件只认识 0、1 组成的机器指令；其他语言写的程序，都要先翻译成机器指令才能执行；翻译这一步不是硬件直接执行的。

专有名词从零解释：

- 先想“机器语言程序”：用 0、1 写成的指令序列，每条指令直接对应硬件的一个操作；硬件读到就能执行，不用翻译。
- 再想“汇编语言程序”：用助记符写的程序，比如用 ADD 表示加法；人看着方便，硬件看着不懂；要先经过“汇编”这个翻译步骤，变成机器语言程序，硬件才能执行。
- 再想“硬件描述语言程序”：用来描述硬件电路结构的设计语言，比如描述“这个芯片里有几个加法器、怎么连线”；它是给设计工具用的，用来生成电路，不是给硬件执行的。
- 再想“直接执行”：硬件读到指令，不经过翻译，马上就能做；只要中间还要翻译一步，就不叫直接执行。

考查知识点：

- 先想 Ⅰ 为什么行：机器语言程序就是 0、1 指令，硬件直接认识，直接执行；所以 Ⅰ 在答案里。
- 再想 Ⅱ 为什么不行：汇编语言程序要先汇编成机器语言程序，硬件才能执行；硬件直接执行的还是汇编后的机器语言，不是汇编语言本身；所以 Ⅱ 不在答案里。
- 再想Ⅲ为什么不行：硬件描述语言是设计语言，描述的是电路长什么样，不是“让硬件去做某件事”的指令；硬件不会去“执行”一段电路描述；所以Ⅲ不在答案里。
- 再想组合：只有 Ⅰ 能直接执行，所以是“仅 Ⅰ”，就是 A。
- 拿生活例子验算：硬件像一个只听得懂本地方言的人；机器语言就是本地方言，直接能听懂；汇编语言像普通话，要先翻译成本地方言才能听懂；硬件描述语言像建筑图纸，是画给施工队看的，不是念给这个人听的。
- 边界情况：有些硬件里有微程序，机器指令还要翻译成微操作；但这属于硬件内部实现，不改变“机器语言是硬件直接执行的程序类型”这个结论。
- 易混对子：不要把“汇编语言程序最终也能运行”当成“硬件直接执行汇编语言”——最终能运行和直接执行是两回事，中间隔着汇编这一步；也不要把“硬件描述语言里有个‘语言’”当成“它也是程序”——它是设计语言，不是执行对象。

正确选项为什么对： 只有机器语言程序能直接执行，所以是“仅 Ⅰ”，选 A。

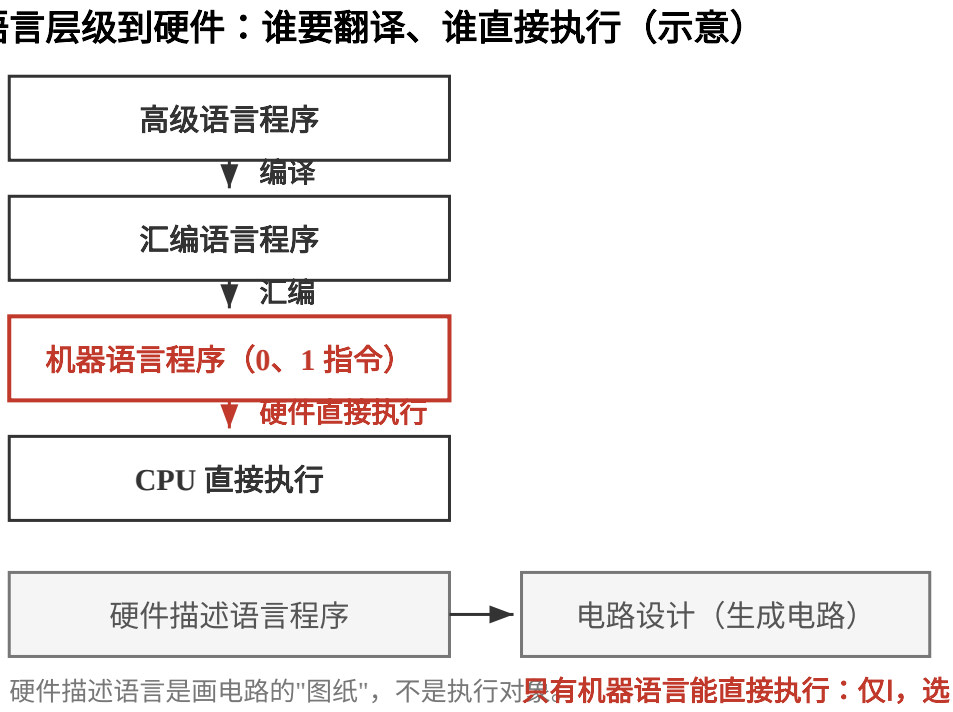
错误选项为什么错＋举例：

- B（仅 Ⅰ、Ⅱ）：把汇编语言也算进去了；汇编语言要先汇编成机器语言，硬件直接执行的还是机器语言，不是汇编语言本身。教训：直接执行不隔翻译这一步。

- C（仅 Ⅰ、Ⅲ）：把硬件描述语言也算进去了；硬件描述语言是描述电路的设计语言，不是给硬件执行的指令。教训：设计语言和执行对象是两回事。
- D（Ⅰ、Ⅱ、Ⅲ）：三种全算进去了；汇编语言要翻译，硬件描述语言根本不是执行对象；只有 Ⅰ 能直接执行。教训：组合题要一个一个验，不能全选。

笔记本一句话： 第 12 题＋硬件直接执行：只认 0、1 的机器语言；汇编要先翻译，硬件描述语言是画电路的图纸。

【图像】



硬件描述语言是画电路的"图纸"，不是执行对象；**只有机器语言能直接执行：仅Ⅰ，选 A。**

语言层级到硬件的翻译链：高级语言经编译到汇编语言，汇编语言经汇编到机器语言，机器语言被硬件直接执行；硬件描述语言单独指向电路设计

【定位】 计算机组成原理 · 第1章 计算机系统概述 · 硬件直接执行的程序类型

【易混对比】

- "最终能运行"和"直接执行"是两回事：汇编语言程序最终能运行，但中间隔了"汇编"翻译这一步，硬件直接执行的还是翻译后的机器语言，不是汇编语言本身。
- "硬件描述语言"和"机器语言"是两回事：一个是描述电路结构的"图纸"，给设计工具生成电路用；一个是0、1组成的指令，给硬件执行用——名字里都有"语言"，干的事完全不同。
- "翻译步骤"和"硬件内部实现"是两回事：机器指令在硬件内部还可能翻译成微操作，但这属于硬件自己干的事，不改变"机器语言是直接执行对象"的结论。

硬件只认0、1组成的机器指令：机器语言程序直接执行；汇编要先翻译成机器语言；硬件描述语言是画电路的图纸，根本不是执行对象——仅 Ⅰ，选A。

【坑位】

组合题要一个一个验，不能全选，也别漏了"直接"两个字。

"直接执行"的意思是硬件读到指令、不经过翻译马上就能做。汇编语言程序"最终能运行"不等于"直接执行"，中间隔着汇编这一步。硬件描述语言更不是程序执行对象。正确做法：把Ⅰ、Ⅱ、Ⅲ逐个问一遍"硬件读到它用不用翻译"，要翻译的都排除。

【类比】

硬件像一个只听得懂本地方言的人：机器语言就是本地方言，直接能听懂；汇编语言像普通话，要先翻译成本地方言才能听懂；硬件描述语言像建筑图纸，是画给施工队看的，不是念给这个人听的。术语贴回去：只有机器语言程序能被硬件直接执行。

【口诀】 硬件只认零和一，汇编翻译才执行，描述语言画图纸。

【考频】 「硬件直接执行」在 2009–2022 年解析中出现 4 次 · 中频

第 13 题 · 限定 1 和 0 个数时 8 位补码能表示的最小整数（正确答案：B）

题干与选项：

由 3 个“1”和 5 个“0”组成的 8 位二进制补码，能表示的最小整数是 。

- A. -126
- B. -125
- C. -32
- D. -3

入手点： 先定符号：最小的数一定是负数，符号位一定是 1；再定剩下的 1 放哪：要让整个数尽可能小，就要让“去掉符号位后的无符号值”尽可能小。

专有名词从零解释：

- 先想“补码”：计算机里表示带符号整数的办法；正数的补码就是它本身，负数的补码是“取反加 1”；8 位补码能表示的范围是从 -128 到 127。
- 再想“符号位”：补码的最高位，0 表示非负数，1 表示负数；符号位是 1 的数一定是负数。
- 再想“无符号值”：把 8 位 0、1 串当成没有符号的普通二进制数读出来的大小；比如 10000011 当无符号数读是 131。
- 再想“补码值和无符号值的关系”：符号位是 1 时，补码表示的整数等于无符号值减去 256；比如无符号值 131，对应的补码值就是 131－256＝-125。

考查知识点：

- 先想为什么最小的数是负数：正数最小是 0，负数都比 0 小；所以最小的数一定是负数，符号位一定是 1；这就用掉了 3 个 1 里的 1 个。
- 再想剩下的 1 怎么放：还剩 2 个 1、5 个 0 要放在低 7 位里；要让无符号值尽可能小，就要让高位尽可能是 0，1 尽量往低位放；低 7 位里最小的放法是把 2 个 1 放在最低两位，也就是 0000011。
- 再想整个 8 位串：符号位 1 加上低 7 位 0000011，合起来是 10000011；无符号值是 128＋2＋1＝131。
- 再想补码值：131－256＝-125，就是 B。
- 拿小例子验算：4 位补码，2 个 1、2 个 0，最小的数是符号位 1、剩下的 1 放最低位，串是 1001，无符号值 9，补码值 9－16＝-7；如果把剩下的 1 放高位，串是 1010，无符号值 10，补码值 -6，比 -7 大，说明放低位才最小。
- 边界情况：如果 1 的个数是 1 个，符号位占了唯一的 1，低位全是 0，串是 10000000，对应 -128，是 8 位补码的最小值；本题有 3 个 1，所以最小值不是 -128。
- 易混对子：不要以为“1 越多、数越小”——补码负数里，无符号值越小，补码值越小（越负）；1 放高位会让无符号值变大，补码值反而变大（没那么负）；也不要把补码值和原码值搞混，原码的负数是符号位加绝对值，算法不一样。

正确选项为什么对： 最小的串是 10000011，无符号值 131，补码值 -125，就是 B，选 B。

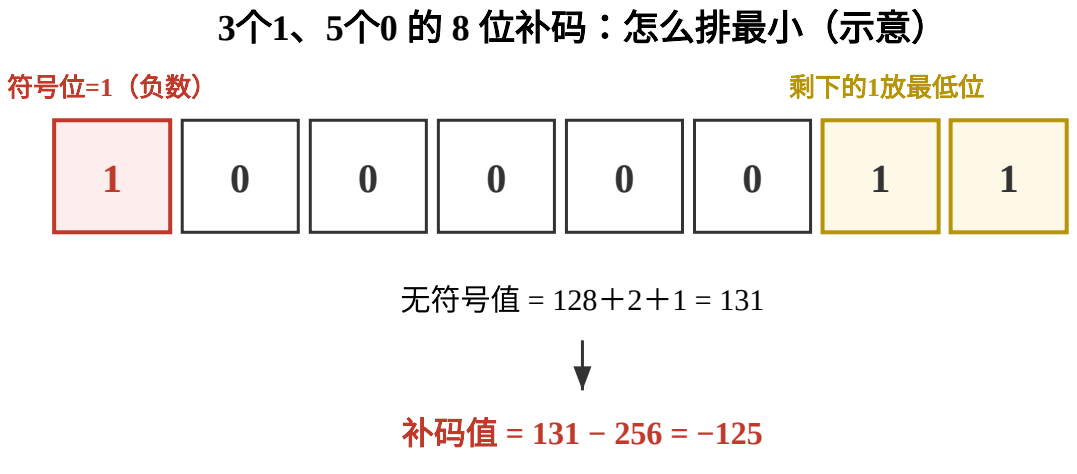
错误选项为什么错＋举例：

- A（-126）：对应的无符号值是 130，串是 10000010；这个串里只有 2 个 1，不符合“3 个 1”的条件。教训：先数 1 的个数，个数不对的串直接排除。

- C（-32）：对应的无符号值是 224，串是 11100000；这个串里有 3 个 1，个数对了，但 1 都在高位，无符号值大，补码值 -32 比 -125 大得多，不是最小。教训：1 放高位，无符号值大，补码值就不够小。
- D（-3）：对应的无符号值是 253，串是 11111101；这个串里有 7 个 1，个数不对，而且 1 都在高位，补码值 -3 接近 0，离最小很远。教训：个数和位置两个条件都要满足。

笔记本一句话： 第 13 题＋定个数求补码最小：符号位占 1 个 1，剩下的 1 往低位放，无符号值最小、补码值最负；本题 10000011＝-125。

【图像】



8位补码最小整数示意：位串10000011，符号位1标红，两个1放在最低位，无符号值131，补码值131减256等于负125

【定位】 计算机组成原理 · 第2章 数据的表示和运算 · 定点数补码极值

【易混对比】

- "补码值"和"无符号值"是两回事：同一个8位串10000011，当无符号数读是131，当补码读是负125——符号位为1的串，补码值等于无符号值减256。
- "1放高位"和"1放低位"是两回事：1放高位，无符号值大，补码值就不够负；要让补码值最小（最负），1要尽量往低位放。
- "补码负数"和"原码负数"是两回事：补码的负数是"取反加1"来的，算法和原码"符号位加绝对值"不一样，别套原码的思路。

符号位占掉1个1，剩下的1全部往最低位放：串是10000011，无符号值131，补码值是负125，选B。

【坑位】

先数1的个数，个数不对的串直接排除；再看1的位置，高位1越多越不够负。

A项负126对应的串是10000010，只有2个1，不符合"3个1"的条件，先排除。C项负32的串是11100000，个数对了但1全在高位，无符号值大，补码值不够负。正确做法：个数和位置两个条件都要满足——个数先验，位置再往低放。

【套路】

- 第一步，定符号位：最小的数一定是负数，符号位一定是1，用掉3个1里的1个。
- 第二步，剩下的1往低位放：还剩2个1、5个0，全部放进低7位，最小的放法是把2个1放在最低两位，得0000011。
- 第三步，串转补码值：合起来是10000011，无符号值是131，补码值是131减256得负125。

符号占一剩低放，串转无符号值再减二五六。

【类比】

像凑一个"最负"的数：符号位是"负号"，必须占1个1；剩下的1是"累赘"——放在高位会让数变大（不够负），只能塞到最低位去"藏"起来。术语贴回去：补码负数里，无符号值越小，补码值越小（越负）。

【口诀】 定数求补码最小，符号占一余低放。

【考频】 「补码」在 2009–2022 年解析中出现 17 次 · 高频

第 14 题 · 浮点数加减运算相关叙述的正误判断（正确答案：D）

题干与选项：

下列有关浮点数加减运算的叙述中，正确的是 。

Ⅰ. 对阶操作不会引起阶码上溢或下溢

Ⅱ. 右规和尾数舍入都可能引起阶码上溢

Ⅲ. 左规时可能引起阶码下溢

Ⅳ. 尾数溢出时结果不一定溢出

- A. 仅 Ⅱ、Ⅲ
- B. 仅 Ⅰ、Ⅱ、Ⅳ
- C. 仅 Ⅰ、Ⅲ、Ⅳ
- D. Ⅰ、Ⅱ、Ⅲ、Ⅳ

入手点： 一条一条过浮点数加减的流程：对阶、尾数加减、规格化、舍入、判溢出；每条说法对应流程里的一步，看它在那一步上说得对不对。

专有名词从零解释：

- 先想“浮点数”：用“阶码＋尾数”的办法表示的数，像科学记数法；比如 1.23×10^5，阶码是 5，尾数是 1.23；计算机里的浮点数也是这个思路。
- 再想“对阶”：加减之前，先把两个数的阶码调成一样大；小阶向大阶看齐，尾数相应右移；对阶只动尾数，不动阶码本身的大小关系。
- 再想“规格化”：把尾数调成标准形式；尾数太大（溢出）要右规，尾数太小（前面有很多 0）要左规；右规时阶码加 1，左规时阶码减 1。
- 再想“阶码上溢、下溢”：阶码超过了能表示的最大值，叫上溢，结果太大，算不出来；阶码小于能表示的最小值，叫下溢，结果太小，近似成 0。
- 再想“尾数溢出”：尾数加减后超出了尾数能表示的范围；这时不一定整个结果就溢出了，因为右规可以把尾数调回来。

考查知识点：

- 先想 Ⅰ 为什么对：对阶是“小阶向大阶看齐”，阶码本身只是被拿来比较、选大的那一个，没有加减运算，不会超出阶码的范围；所以对阶不会引起阶码上溢或下溢，Ⅰ 对。
- 再想 Ⅱ 为什么对：右规时阶码加 1，如果阶码已经是最大值，加 1 就上溢了；尾数舍入时，尾数可能因为进位又溢出一次，又要右规一次，阶码又加 1，也可能上溢；所以右规和尾数舍入都可能引起阶码上溢，Ⅱ 对。
- 再想Ⅲ为什么对：左规时尾数左移、阶码减 1；如果阶码已经是最小值，减 1 就下溢了；所以左规可能引起阶码下溢，Ⅲ对。
- 再想Ⅳ为什么对：尾数溢出时，只要右规一次，尾数就调回来了，阶码加 1 后如果还在范围内，结果就是正常的数，不溢出；所以尾数溢出不等于结果溢出，Ⅳ对。
- 再想结论：四条都对，就是 D。
- 拿生活例子验算：科学记数法 9.9×10^99 加 1.0×10^99，尾数加起来 10.9，超出了 1 位整数的范围，右规成 1.09×10^100，阶码加 1；如果阶码最大就是 99，加 1 就上溢了；如果阶码最大是 100，右规后照样是正常的数，尾数溢出不等于结果溢出。

- 边界情况：尾数舍入引起的溢出和尾数加减引起的溢出都要右规；右规一次不够时（舍入后又溢出），可能要再右规一次，阶码再加 1。
- 易混对子：不要把“尾数溢出”和“结果溢出”划等号——尾数溢出是中间状态，右规后可能恢复正常；也不要以为“对阶移尾数会丢精度”就等于“对阶会引起阶码溢出”——丢精度是尾数右移丢掉低位的事，和阶码溢出是两回事。

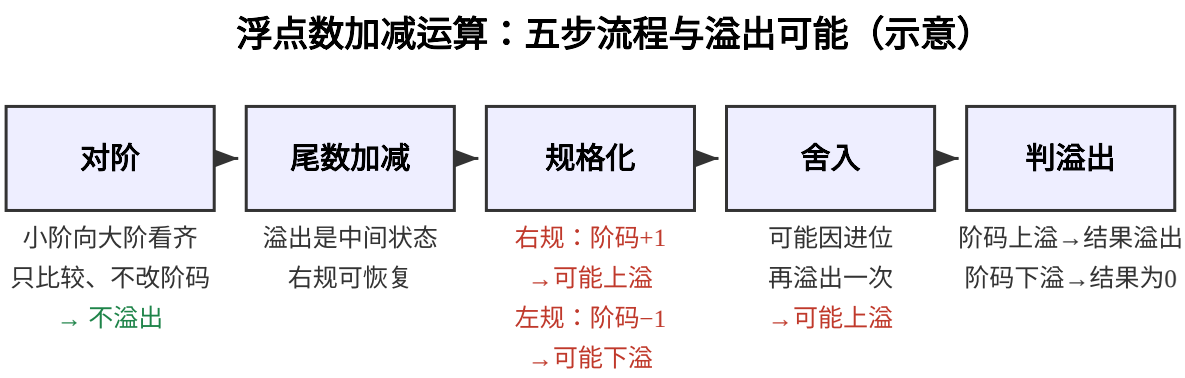
正确选项为什么对： 四条说法逐条验都对，所以是四项全对，就是 D，选 D。

错误选项为什么错＋举例：

- A（只有部分说法对的组合）：漏了对的说法；比如以为对阶会引起阶码溢出，实际上对阶只比较阶码、不加减阶码，不会溢出。教训：对阶这一步不改阶码的值。
- B（只有部分说法对的另一组合）：漏了右规、舍入可能引起上溢，或者漏了左规可能引起下溢；右规阶码加 1、左规阶码减 1，走到边界就可能溢出。教训：规格化改阶码，走到边界就可能溢出。
- C（只有部分说法对的又一组合）：把“尾数溢出一定导致结果溢出”当成真了；实际上尾数溢出右规后可能恢复正常，结果不一定溢出。教训：尾数溢出是中间状态，不是最终结论。

笔记本一句话： 第 14 题＋浮点加减四说法：对阶不改阶码不溢出；右规、舍入可能上溢；左规可能下溢；尾数溢出右规后可能正常，四条全对。

【图像】



Ⅰ 对阶不改阶码 ✓　　Ⅱ 右规、舍入可能上溢 ✓

Ⅲ 左规可能下溢 ✓　　Ⅳ 尾数溢出右规后可正常 ✓

生活例子：科学记数法 $9.9\times10^{99}+1.0\times10^{99}$，尾数 10.9 超了，右规成 1.09×10^{100} 就好。

四条全对，选 D。

浮点数加减运算流程与溢出可能性：对阶只比较不改阶码无溢出，尾数加减后规格化，右规阶码加一可能上溢，左规阶码减一可能下溢，舍入可能再上溢

【定位】 计算机组成原理 · 第2章 数据的表示和运算 · 浮点数加减运算

【易混对比】

- "尾数溢出"和"结果溢出"是两回事：尾数溢出是中间状态，右规一次把尾数调回来、阶码加1后还在范围内，结果就是正常的数。

- "对阶丢精度"和"对阶引起阶码溢出"是两回事：丢精度是小阶尾数右移丢掉低位的事；对阶只比较阶码、选大者，不加减阶码，不会引起阶码溢出。
- "右规"和"左规"是两回事：右规是尾数太大往右调、阶码加1，可能上溢；左规是尾数太小往左调、阶码减1，可能下溢——方向和溢出类型都别反。

对阶不改阶码、不溢出；右规和尾数舍入都可能引起阶码上溢；左规可能引起阶码下溢；尾数溢出右规后可能恢复正常——四条全对，选D。

【坑位】

别把"尾数溢出"直接当成"结果溢出"，也别以为对阶会改阶码。

尾数加减溢出后，右规一次就能把尾数调回来，只要阶码加1后还在范围内，结果就是正常的数。题里Ⅳ说"尾数溢出时结果不一定溢出"，正是这个意思。对阶这一步只是"小阶向大阶看齐"，阶码本身只被比较、不被加减。正确做法：沿着"对阶、尾数加减、规格化、舍入、判溢出"的流程，一步一步看阶码动没动、走到边界没有。

【套路】

- 第一步，看对阶：小阶向大阶看齐，阶码只比较不加减，不会引起阶码上溢或下溢——Ⅰ对。
- 第二步，看右规和舍入：右规阶码加1，走到最大值就上溢；尾数舍入可能因进位再溢出一次、再右规一次，阶码再加1也可能上溢——Ⅱ对。
- 第三步，看左规：左规尾数左移、阶码减1，走到最小值就下溢——Ⅲ对。
- 第四步，看尾数溢出：尾数溢出是中间状态，右规后阶码还在范围内，结果正常，不一定溢出——Ⅳ对。

四步验四条：对阶不动码，右舍可上溢，左规可下溢，尾溢右规救。

【类比】

像科学记数法做加法：先把指数调成一样（对阶，只看不改指数），尾数相加；尾数太大就右移一位、指数加1（右规，指数到头就"爆表"上溢）；尾数太小就左移、指数减1（左规，指数到底就"归零"下溢）；尾数一时超了不等于最后结果爆表，调回来就行。术语贴回去：浮点数加减按对阶、尾数加减、规格化、舍入、判溢出五步走。

【口诀】 对阶不动码，右舍上溢左下溢，尾溢右规可救回。

【考频】 「浮点数加减」在 2009–2022 年解析中出现 3 次 · 中频

第 15 题 · 直接映射 Cache 存放指定容量数据时的总容量下限（正确答案：C）

题干与选项：

假定主存地址为 32 位，按字节编址，主存和 Cache 之间采用直接映射方式，主存块大小为 4 个字，每字 32 位，采用回写（Write Back）方式，则能存放 4K 字数据的 Cache 的总容量的位数至少是 。

- A. 146k
- B. 147K
- C. 148K
- D. 158K

入手点： 先算 Cache 有多少行，再算每行有多少位：每行是“数据＋标记＋有效位＋脏位”四部分加起来；行数乘以每行位数，就是总容量。

专有名词从零解释：

- 先想“Cache”：夹在 CPU 和主存之间的小而快的存储器；CPU 要的数据如果在 Cache 里，就不用去慢的主存拿，这就叫命中。
- 再想“直接映射”：主存的每一块只能放到 Cache 里的一个固定位置；位置由地址的中间一段（行号）决定；找起来快，但容易冲突。
- 再想“Cache 行”：Cache 里存一块数据的基本单位；一行里除了存数据本身，还要存标记位、有效位、脏位这些辅助信息。
- 再想“标记位”：用来记录“这一行存的是主存里哪一块”；地址里除了行号和块内地址，剩下的高位就是标记。
- 再想“有效位”：1 位，1 表示这一行存的数据是有效的，0 表示这一行是空的、数据不可用。
- 再想“脏位”：回写方式下才有的 1 位，1 表示这一行的数据被改过、和主存不一致，换出时要写回主存；0 表示和主存一致，换出时不用写回。
- 再想“回写方式”：也叫 Write Back，写 Cache 时只改 Cache，不马上写主存；等到这一行被换出时，才把改过的数据写回主存；所以必须有脏位记录改没改过。

考查知识点：

- 先想 Cache 有多少行：要存 4K 字，主存块是 4 个字，所以块数是 4K÷4＝1K；直接映射下，Cache 行数至少等于块数，就是 1K 行；行号要 10 位（2^10＝1024）。
- 再想块内地址有几位：主存块 4 个字，每字 32 位，就是 4×4＝16 字节；块内地址要 4 位（2^4＝16）。
- 再想标记位有几位：主存地址 32 位，减去行号 10 位、减去块内地址 4 位，剩下 32－10－4＝18 位。
- 再想每行有多少位：数据 4×32＝128 位，加上标记 18 位，加上有效位 1 位，加上脏位 1 位（回写方式必备），共 128＋18＋1＋1＝148 位。
- 再想总容量：1K 行×148 位＝148K，就是 C。
- 拿生活例子验算：一个储物柜有 1024 个格子，每个格子除了放 128 件东西，还要贴一张 18 个字的标签、一个“有没有东西”的牌子、一个“改没改过”的牌子；总容量要把标签和牌子都算进去，不能只算东西。
- 边界情况：如果改成直写方式，就不用脏位，每行是 147 位，总容量是 147K；本题是回写方式，所以是 148K；直写和回写的区别就在脏位这一位上。

- 易混对子：不要只算数据位 128K 就当成答案——标记位、有效位、脏位都是 Cache 总容量的一部分，漏了任何一位都会算小；也不要把“字”和“字节”搞混，块是 4 个字、每字 32 位，数据位是 128 位，不是 128 字节。

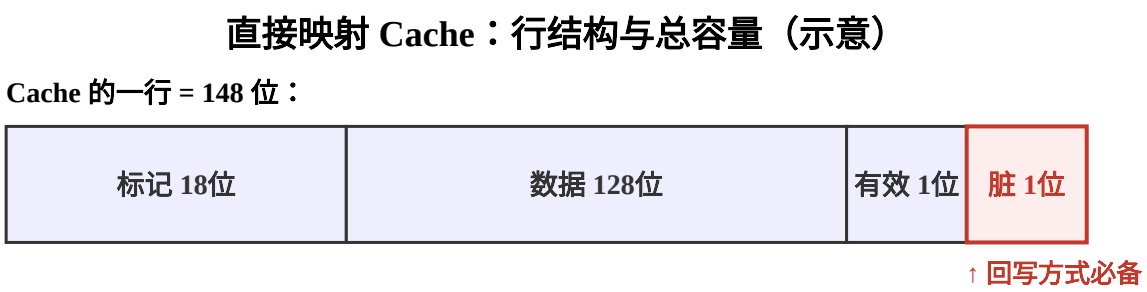
正确选项为什么对： 1K 行，每行 148 位，总容量 148K，就是 C，选 C。

错误选项为什么错＋举例：

- A（146K）：少算了 2 位；比如漏了有效位和脏位，只算了数据 128 位加标记 18 位，就是 146 位一行。教训：有效位和脏位都要算。
- B（147K）：少算了 1 位；比如算了有效位、漏了脏位，或者算了脏位、漏了有效位；回写方式下脏位必备，不能漏。教训：回写方式一定有脏位。
- D（158K）：多算了 10 位；比如把行号 10 位也当成每行要存的位算进去了；行号是地址里用来定位的，不用存在行里。教训：行号不存，标记位才存。

笔记本一句话： 第 15 题＋Cache 总容量：行数×（数据＋标记＋有效＋脏），回写必有脏位；本题 1K×148＝148K。

【图像】



直接映射Cache行结构与容量计算：一行148位等于标记18位加数据128位加有效位1位加脏位1位，1K行总容量148K

【定位】 计算机组成原理 · 第3章 存储器层次结构 · Cache容量计算

【易混对比】

- "回写"和"直写"是两回事：回写写时只改Cache，换出时才写回主存，必须有脏位记录改没改过；直写同时写主存，不用脏位——本题是回写，脏位必备。
- "标记位"和"行号"是两回事：行号是地址中间那段、用来定位行，不存在行里；标记位是地址剩下的高位、记录"这一行存的是主存哪一块"，才存在行里。
- "数据容量"和"总容量"是两回事：总容量等于行数乘以（数据加标记加有效加脏），漏掉任何一位都会算小。

Cache有1K行，每行148位（数据128加标记18加有效1加脏1），总容量148K，选C。

【坑位】

回写方式的脏位是重灾区，有效位也别漏，行号不许存进总容量。

A项146K是漏了有效位和脏位；B项147K是漏了其中一位；D项158K是把行号10位也算进去了。正确做法：先想"回写就有脏位"，再逐项点名——数据、标记、有效、脏，一位都不能少；行号是地址里定位用的，不存。

【套路】

- 第一步，算行数：存4K字，块是4个字，块数等于4K除以4得1K，直接映射行数至少等于块数，得1K行。
- 第二步，拆地址算标记：块内地址4位（16字节），行号10位（1K行），标记等于32减10减4得18位。
- 第三步，凑每行位数：数据4乘32得128位，加标记18位，加有效位1位，加脏位1位（回写必备），得148位。
- 第四步，相乘：1K乘148得148K。

行数乘每行，数据标记加有效，脏位回写不能忘。

【类比】

像一个有1024个格子的储物柜：每个格子除了放128件东西，还要贴一张18个字的标签（标记）、挂一个"有没有东西"的牌子（有效位）、再挂一个"改没改过"的牌子（脏位）；算总容量要把标签和牌子都算进去，不能只算东西。术语贴回去：Cache总容量等于行数乘每行位数，每行等于数据加标记加有效位加脏位。

【口诀】 行数乘每行，脏标有效全算上。

【考频】 「直接映射Cache容量」在 2009–2022 年解析中出现 5 次 · 中频

第 16 题 · 页式虚拟存储并配有 TLB、直写 Cache 时完成一条加法指令的最少访存次数（正确答案：B）

题干与选项：

假定编译器将赋值语句“x=x+3;”转换为指令”add xaddr, 3”，其中 xaddr 是 x 对应的存储单元地址。若执行该指令的计算机采用页式虚拟存储管理方式，并配有相应的 TLB，且 Cache 使用直写（Write Through）方式，则完成该指令功能需要访问主存的次数至少是 。

- A. 0
- B. 1
- C. 2
- D. 3

入手点： 把这条指令要做的事拆成四件：取指令、地址转换、读数据、写数据；一件一件看最少要访问主存几次；能命中就不访问主存，不能省的才算 1 次。

专有名词从零解释：

- 先想“页式虚拟存储”：把程序的地址空间切成一页一页的，页和主存里的页框对应；CPU 给出的是虚拟地址，要先查页表换成物理地址，才能访问主存。
- 再想“TLB”：也叫快表，是页表的一份小而快的副本，放在 CPU 里；地址转换时先查 TLB，查到了就不用去主存查页表，这就叫 TLB 命中。
- 再想“Cache”：夹在 CPU 和主存之间的小而快的存储器；读数据时先查 Cache，查到了就不用去主存，这就叫 Cache 命中。
- 再想“直写方式”：也叫 Write Through，写数据时，Cache 和主存同时写；写操作一定会访问主存一次，省不掉；这和回写方式不一样，回写方式写时只写 Cache，不马上写主存。
- 再想“最少访存次数”：题目问“至少”要访问几次，就是取所有情况里最好的那一种：能命中的都命中，能省的都省掉。

考查知识点：

- 先想取指令：指令本身在主存里，但它可能已经在 Cache 里；最好的情况是 Cache 命中，访问主存 0 次。
- 再想地址转换：虚拟地址换成物理地址要查页表，但 TLB 里可能已经有这一页的对应关系；最好的情况是 TLB 命中，访问主存 0 次。
- 再想读数据 x：读 x 之前要先拿到 x 的值，x 也可能已经在 Cache 里；最好的情况是 Cache 命中，访问主存 0 次。
- 再想写数据 x：算出 x＋3 后要把结果写回 x；直写方式下，写操作必须同时写主存，这 1 次省不掉；所以写数据访问主存 1 次。
- 再想总数：0＋0＋0＋1＝1 次，就是 B。
- 再想为什么不是 0 次：直写方式的写操作一定写主存，所以总数不可能降到 0；如果改成回写方式，写时只写 Cache，总数就可以是 0 次；本题是直写，所以下限是 1。
- 拿生活例子验算：去图书馆借书，书在手边的小书架上（Cache 命中）就不用跑大书库；但还书时规定必须同时登记到大书库（直写），这一次大书库是一定要跑的。

- 边界情况：如果 TLB 没命中，地址转换还要去主存查页表，多 1 次；如果 Cache 没命中，读数据还要去主存拿，多 1 次；这些都是“不是最少”的情况，题目问最少，就不算它们。
- 易混对子：不要把“读数据”也算成 1 次——读数据可以命中 Cache，命中时是 0 次；题目问“最少”，就要按命中算。

> 本题存在公开题库分歧，说明如下：有公开题库对本题给出 2 次，它的算法是把读数据算成 1 次访存；但那与题目里的“至少”矛盾——读数据命中 Cache 时是 0 次，不是 1 次；按“至少”取最优情况，读数据是 0 次，写数据直写是 1 次，总数是 1 次；本书按夸克解析的 B 讲清此推理，分歧点就在于读数据按 0 次还是 1 次算。

正确选项为什么对： 取指、地址转换、读数据都可以命中、0 次，写数据直写必须访存 1 次，总数最少 1 次，就是 B，选 B。

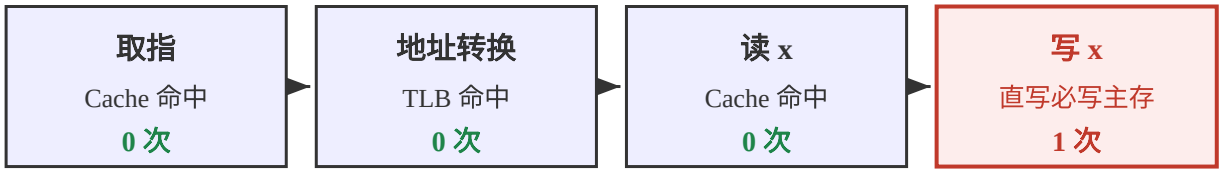
错误选项为什么错＋举例：

- A（0 次）：以为所有操作都能省掉，漏了直写方式的写操作；直写方式写时必须写主存，0 次是不可能的。教训：直写方式的写操作省不掉。
- C（2 次）：把读数据算成了 1 次访存；这是公开题库分歧的那个算法；但题目问“至少”，读数据命中 Cache 时是 0 次，2 次不是最少。教训：问“最少”时，能命中的都按命中算。
- D（3 次）：把取指、读数据、写数据各算 1 次，漏了“命中可以是 0 次”；这是按“每次都访存”算的，不是按“最少”算的。教训：最少和每次都访存是两回事。

笔记本一句话： 第 16 题＋“至少”访存：取指、TLB、读数都能命中 0 次，直写写回必访存 1 次，最少 1 次；读数按 1 次算就是分歧点。

【图像】

add xaddr,3：完成指令的最少访存次数（示意）



问的是"至少"——能命中的都按命中算 0 次。

0 ＋ 0 ＋ 0 ＋ 1 = 最少 **1 次**

直写方式的写操作必须同时写主存，这 1 次省不掉。

若改成回写方式，写时只写 Cache，总数可以是 0 次。

最少 1 次，选 B。

加法指令访存路径：取指Cache命中0次，地址转换TLB命中0次，读x数据Cache命中0次，写x直写必访存1次，最少共1次

【定位】 计算机组成原理 · 第3章 存储器层次结构 · 虚拟存储与Cache访存次数

【易混对比】

- "至少"和"每次都访存"是两回事：题目问最少，就是取最好的情况——能命中的都按命中算0次；按"每次都访存"算出来的是上限，不是答案。
- "直写"和"回写"是两回事：直写写操作同时写主存，这1次省不掉；回写写时只写Cache，总数可以降到0次——本题是直写，下限是1。
- "读数据"和"写数据"是两回事：读数据可以命中Cache、按0次算；写数据在直写方式下必须访存主存，按1次算。

取指、地址转换、读数据都能命中、按0次算，直写方式的写操作必须访存1次——最少1次，选B。

【坑位】

看到"至少"两个字，就把能命中的全按命中算；别把读数据也按1次算。

读数据命中Cache时是0次，不是1次，这是"至少"和"每次都访存"的区别。A项0次漏了直写方式的写操作——直写写时必须同时写主存，0次不可能。正确做法：把指令拆成取指、地址转换、读数据、写数据四件，一件一件问"能不能命中省掉"，省不掉的才计1次。

【套路】

- 第一步，拆四件：取指、地址转换、读数据、写数据。
- 第二步，能命中的按0次：取指Cache命中0次，地址转换TLB命中0次，读数据Cache命中0次。
- 第三步，直写写操作计1次：直写方式写时同时写主存，省不掉，计1次。
- 第四步，求和：0加0加0加1得1，最少1次。

至少取最优，命中全按零，直写写必访。

【类比】

像去图书馆借一本书再还一本：要看的书在手边小书架上（Cache命中）就不用跑大书库，查目录卡片也在手边（TLB命中）不用跑；但还书时规定必须同时登记到大书库（直写），这一趟大书库一定要跑。术语贴回去：完成一条加法指令最少访存1次，就是直写那次写回。

【口诀】 至少问最少，命中按零算，直写写必访。

【考频】 「访存次数」在 2009–2022 年解析中出现 1 次 · 低频

第 17 题 · 工作期间需要周期性刷新的存储器（正确答案：B）

题干与选项：

下列存储器中，在工作期间需要周期性刷新的是 。

- A. SRAM
- B. SDRAM
- C. ROM
- D. FLASH

入手点： 先想每种存储器靠什么存信息：靠电容电荷存的，电荷会漏，就要刷新；靠触发器存的，通电就一直保持，不用刷新；不靠电存的，断电也不丢，更不用刷新。

专有名词从零解释：

- 先想“SRAM”：静态随机存储器，靠触发器电路存信息；触发器只要通着电，就一直保持 0 或者 1，不用刷新；速度快、价格贵，常用来做 Cache。
- 再想“SDRAM”：同步动态随机存储器，属于 DRAM 的一种，靠电容上的电荷存信息；电容会漏电，电荷存 1～2 毫秒就漏得差不多了，所以必须周期性刷新，把电荷补满。
- 再想“ROM”：只读存储器，出厂时信息就写死了，断电也不丢，工作期间不用刷新。
- 再想“FLASH”：闪存，靠浮栅上的电荷存信息，但电荷被绝缘层包着，漏得非常慢，断电也不丢，工作期间不用周期性刷新。

考查知识点：

- 先想 SDRAM 为什么要刷新：电容漏电是物理规律，电荷会自己跑掉；如果不隔一段时间把电荷补满，存的 1 就会变成 0，信息就丢了；所以 SDRAM 必须周期性刷新，B 对。
- 再想 SRAM 为什么不用刷新：触发器是两个反相器交叉连起来的电路，只要通着电，状态就一直保持，不会自己跑掉；所以 SRAM 不用刷新。
- 再想 ROM 为什么不用刷新：ROM 的信息是出厂时固化的，断电也不丢；工作期间读出来就行，不用补写。
- 再想 FLASH 为什么不用刷新：FLASH 的电荷被绝缘层包着，漏得极慢，能保持好几年；工作期间不用周期性刷新；FLASH 的擦写是另外一回事，不是刷新。
- 拿生活例子验算：SDRAM 像一个漏水的水桶，要隔一会儿就加一次水，不然水就漏光了；SRAM 像一个上了锁的柜子，东西放进去就一直在；ROM 和 FLASH 像刻在石头上的字，不用补刻。
- 边界情况：SDRAM 的刷新周期一般是几十毫秒刷新全部行一次，刷新时不能正常读写，系统会留出刷新时间；这个时间开销是 DRAM 的代价之一。
- 易混对子：不要把“DRAM”和“SRAM”搞混——名字里都有 RAM，但一个靠电容、要刷新，一个靠触发器、不用刷新；也不要把“FLASH 要擦写”当成“FLASH 要刷新”——擦写是改信息，刷新是补电荷，两回事。

正确选项为什么对： SDRAM 靠电容存信息，电荷会漏，必须周期性刷新，选 B。

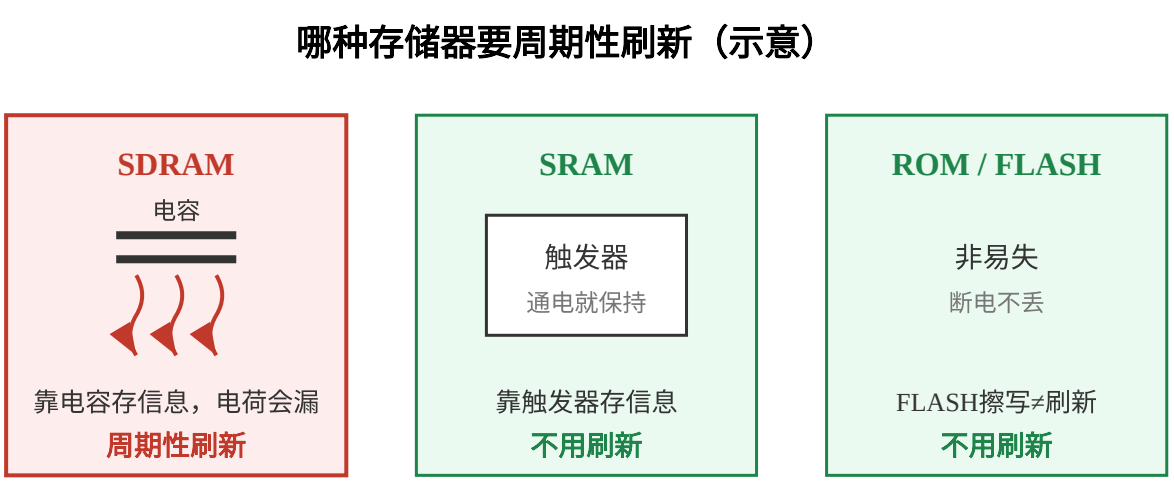
错误选项为什么错＋举例：

- A（SRAM）：SRAM 靠触发器存信息，通电就保持，不用刷新；把它当成要刷新的，是把 SRAM 和 DRAM 搞混了。教训：SRAM 是静态的，不用刷新。
- C（ROM）：ROM 的信息出厂就固化了，断电也不丢，工作期间只读不补。教训：只读存储器不用刷新。

- D（FLASH）：FLASH 的电荷被绝缘层包着，漏得极慢，断电也不丢，工作期间不用周期性刷新。教训：非易失存储器不用刷新。

笔记本一句话： 第 17 题＋要刷新的存储器：只有靠电容的 DRAM（SDRAM）要刷新；SRAM 靠触发器、ROM 和 FLASH 非易失，都不用。

【图像】



漏水的水桶要隔会儿加水（刷新补电荷）；上了锁的柜子、刻在石头上的字都不用。

只有 SDRAM 要周期性刷新，选 B。

DRAM电容漏电刷新示意：SDRAM电容电荷不断漏掉需周期性补满，SRAM触发器通电保持不用刷新，ROM和FLASH非易失不用刷新

【定位】 计算机组成原理 · 第3章 存储器层次结构 · 存储器分类

【易混对比】

- "DRAM"和"SRAM"是两回事：名字里都有RAM，一个靠电容存信息、电荷会漏，要周期性刷新；一个靠触发器存信息、通电就保持，不用刷新。
- "FLASH擦写"和"刷新"是两回事：擦写是改写存的信息，刷新是把漏掉的电荷补满；FLASH电荷被绝缘层包着、漏得极慢，工作期间不用周期性刷新。
- "易失"和"非易失"是两回事：SRAM、DRAM断电丢信息，是易失的；ROM、FLASH断电不丢，是非易失的——但易失的不都要刷新，只有靠电容的DRAM要。

只有靠电容存信息的DRAM（SDRAM）电荷会漏，必须周期性刷新；SRAM靠触发器、ROM和FLASH非易失，都不用——选B。

【坑位】

看到RAM别急着选，先问"靠什么存信息"。

DRAM和SRAM名字像，存信息的原理完全不同：电容漏电要补，触发器通电就稳。FLASH要擦写不等于要刷新，擦写是改信息、刷新是补电荷。正确做法：按"存储原理"分类——靠电容的才要刷新，其他都不用。

【类比】

SDRAM像一个漏水的水桶，要隔一会儿就加一次水，不然水漏光、信息就丢了；SRAM像一个上了锁的柜子，东西放进去就一直在；ROM和FLASH像刻在石头上的字，不用补刻。术语贴回去：DRAM靠电容存信息，电荷会漏，必须周期性刷新。

【口诀】 电容漏电要刷新，触发器稳石头刻。

【考频】 「刷新」在 2009–2022 年解析中出现 8 次 · 中频

第 18 题 · 多体交叉存储器访问序列中可能发生访存冲突的地址对（正确答案：D）

题干与选项：

某计算机使用 4 体交叉编址存储器，假定在存储器总线上出现的主存地址（十进制）序列为

8005，8006，8007，8008，8001，8002，8003，8004，8000，则可能发生访存冲突的地址对是 。

- A. 8004 和 8008
- B. 8002 和 8007
- C. 8001 和 8008
- D. 8000 和 8004

入手点： 先算每个地址落在哪个体：低位交叉编址下，体号由地址对体数取余决定；再看哪两个地址体号相同，体号相同的那一对就可能冲突。

专有名词从零解释：

- 先想“多体交叉存储器”：把主存分成几个体，每个体可以独立工作；多个体并行起来，整体带宽就上去了。
- 再想“低位交叉编址”：地址的低位决定体号，高位决定体内的位置；连续的地址会落在不同的体里，这样连续访问就能并行。
- 再想“访存冲突”：两个地址落在同一个体里，如果前一次访问还没完成，后一次访问又到了，同一个体同一时间只能做一次访问，后一次就得等，这就叫冲突。

考查知识点：

- 先想体号算法：4 体交叉，体号＝地址对 4 取余，再按 1～4 编号：余 1 是 1 号体，余 2 是 2 号体，余 3 是 3 号体，余 0 是 4 号体。
- 再想逐个算：8005 除以 4 余 1，是 1 号体；8006 余 2，是 2 号体；8007 余 3，是 3 号体；8008 余 0，是 4 号体；8001 余 1，是 1 号体；8002 余 2，是 2 号体；8003 余 3，是 3 号体；8004 余 0，是 4 号体；8000 余 0，是 4 号体。
- 再想找同体对：8004 和 8000 都是 4 号体，同一体，前一次存取未完成时后一次又到，可能冲突，就是 D。
- 再想为什么是“可能”不是“一定”：同体只是冲突的必要条件，不是充分条件；如果前一次访问已经完成了，后一次再到，就不冲突；题目问“可能”，同体加上“前一次还没完成、后一次就到了”才算；序列里相邻两次访问落在同一体，最直接满足这个条件。
- 拿小例子验算：4 个窗口排队，地址 5、9、13 都对 4 余 1，都是 1 号窗口；5 和 9 如果前后脚到，1 号窗口忙不过来，就可能排队；5 和 6 分别是 1 号和 2 号窗口，各走各的，不冲突。
- 边界情况：高位交叉编址下，体号由地址高位决定，连续地址会落在同一个体里，冲突更多；本题是低位交叉，连续地址落在不同体里，冲突少；两种编址的体号算法不一样，别混了。
- 易混对子：不要用地址的高位算体号——低位交叉的体号看低位；也不要把“地址相邻”当成“冲突”——相邻地址在低位交叉下恰恰落在不同体里，不冲突。

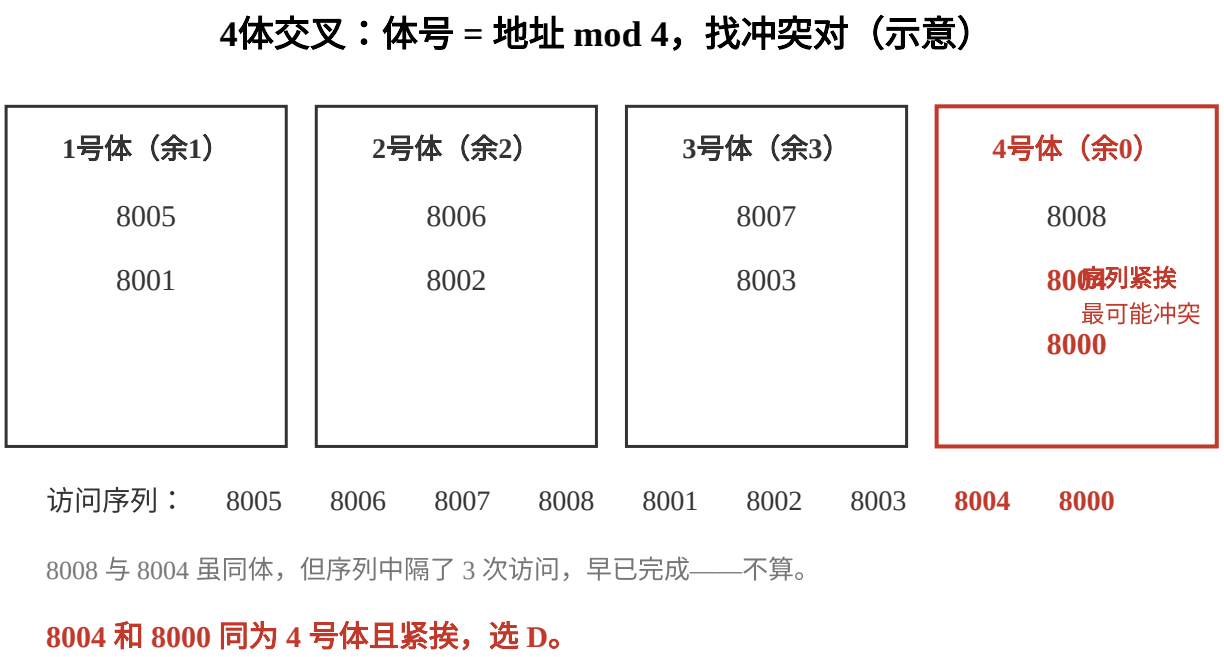
正确选项为什么对： 8004 和 8000 都是 4 号体，同体可能冲突，选 D。

错误选项为什么错＋举例：

- A（8004 和 8008）：8004 和 8008 确实都是 4 号体，但它们在序列里不相邻——8008 是第 4 个，8004 是第 8 个，中间隔了 8001、8002、8003 三次访问；到 8004 到来时，8008 的那次访问早已完成，不构成“前一次未完成、后一次又到”；而 8004 和 8000 在序列里是紧挨着的两次访问，同体且相邻，才最直接构成可能冲突。教训：同体之外，还要看两次访问在序列里是不是紧挨着。
- B（8002 和 8007）：8002 是 2 号体，8007 是 3 号体，不同体，各走各的，不可能冲突。教训：先算体号，体号不同直接排除。
- C（8001 和 8008）：8001 是 1 号体，8008 是 4 号体，不同体，不可能冲突。教训：体号不同，再近的地址也不冲突。

笔记本一句话： 第 18 题＋低位交叉冲突：体号＝地址对体数取余（余 0 是最后一号体），同体才可能冲突；本题 8004 和 8000 都是 4 号体。

【图像】



4体交叉存储器地址分布与冲突对：地址模4取余定体号，8004和8000同为4号体且在序列中紧挨，最可能冲突

【定位】 计算机组成原理 · 第3章 存储器层次结构 · 多体交叉存储器

【易混对比】

- "同体"和"冲突"是两回事：同体只是冲突的必要条件，还要"前一次访问未完成、后一次又到"；序列里紧挨着的同体对最直接满足，隔了好几次访问的同体对早已完成、不冲突。
- "低位交叉"和"高位交叉"是两回事：低位交叉的体号看地址低位（地址对体数取余），连续地址落在不同体；高位交叉的体号看高位，连续地址落在同一体——两种编址的体号算法别混。
- "地址相邻"和"冲突"是两回事：低位交叉下相邻地址恰恰落在不同体里，各走各的，不冲突。

体号等于地址对4取余（余0是4号体）；8004和8000都是4号体，且在序列里紧挨着，前一次未完成、后一次又到，最可能冲突——选D。

【坑位】

同体之外，还要看两次访问在序列里是不是紧挨着。

A项8004和8008确实都是4号体，但它们在序列里不相邻——8008是第4个，8004是第8个，中间隔了8001、8002、8003三次访问，到8004到来时8008那次早已完成。而8004和8000是序列里紧挨着的两次访问，同体且相邻，才最直接构成"可能冲突"。正确做法：先算体号筛同体对，再看序列位置，紧挨的才算。

【套路】

- 第一步，算体号：4体低位交叉，体号等于地址对4取余，余1是1号体、余2是2号体、余3是3号体、余0是4号体。
- 第二步，筛同体对：逐个算出8005到8000的体号，找出体号相同的地址对。
- 第三步，看序列位置：同体对里，在序列中紧挨着的那一对最可能冲突——8004和8000。

体号取余看低位，同体还要看紧挨。

【类比】

像4个窗口排队办事：地址5、9、13都对4余1，都是1号窗口；5和9如果前后脚到，1号窗口忙不过来就可能排队（冲突）；5和6分别是1号和2号窗口，各办各的不冲突。术语贴回去：低位交叉下，体号等于地址对体数取余，同体且相邻的两次访问可能发生访存冲突。

【口诀】 体号取余看低位，同体紧挨才冲突。

【考频】 「访存冲突」在 2009–2022 年解析中出现 1 次 · 低频

第 19 题 · 总线定时相关叙述的错误项（正确答案：C）

题干与选项：

下列有关总线定时的叙述中，错误的是 。

- A. 异步通信方式中，全互锁协议最慢
- B. 异步通信方式中，非互锁协议的可靠性最差
- C. 同步通信方式中，同步时钟信号可由各设备提供
- D. 半同步通信方式中，握手信号的采样由同步时钟控制

入手点： 一条一条对照三种定时方式：同步、异步、半同步；每种方式的时钟和握手信号各归谁管，管错了的那条就是错的。

专有名词从零解释：

- 先想“总线”：连接 CPU、主存、输入输出设备的一组公共线路；数据、地址、控制信号都走总线。
- 再想“同步通信”：大家跟着同一个时钟走；时钟到了，数据就有效；像大家跟着同一个节拍器跳舞，节拍器只有一个，不能各打各的。
- 再想“异步通信”：没有统一时钟，靠握手信号约定；发送方说“我准备好了”，接收方说“我收到了”，一来一回确认后，数据才算传完；握手分三种：非互锁、半互锁、全互锁。
- 再想“半同步通信”：同步为主，异步为辅；平时跟着时钟走，遇到慢设备时，慢设备可以发一个等待信号，让时钟多等几个周期；握手信号的采样还是由同步时钟控制的。
- 再想“互锁”：握手信号要不要等对方回应；非互锁是发了就不管对方回不回，半互锁是等对方回应一次，全互锁是来回都确认。

考查知识点：

- 先想 C 为什么错：同步通信要求所有设备遵循统一时钟；时钟只能由一个设备提供（通常是总线时钟）；如果多设备各提供一个时钟，节拍就乱了，数据什么时候有效就说不清了；所以“同步时钟信号可由多设备提供”是错的，C 就是答案。
- 再想 A 为什么对：异步通信里，全互锁要来回确认两次，握手最完整，速度最慢；非互锁和半互锁确认少，速度快一些；所以“全互锁协议最慢”是对的。
- 再想 B 为什么对：非互锁是发了就不管对方回不回，万一对方没收到，发送方也不知道，可靠性最差；所以“非互锁协议的可靠性最差”是对的。
- 再想 D 为什么对：半同步通信里，平时跟着同步时钟走，握手信号什么时候采样、等几个周期，都由同步时钟控制；所以“握手信号的采样由同步时钟控制”是对的。
- 拿生活例子验算：同步通信像全班跟着一个节拍器跳舞，节拍器只能有一个；如果两个人各打一个节拍器，舞步就乱了；异步通信像两个人传纸条，传过去、回一句“收到了”，才算传完；全互锁是传过去、回一句、再确认一句，最慢但最可靠。
- 边界情况：半同步通信的等待信号可以让慢设备多等几个时钟周期，但时钟本身还是统一的；等待信号不改变“时钟只有一个”这个前提。
- 易混对子：不要把“半同步里有握手信号”当成“半同步是异步”——半同步的主时钟还是同步时钟，握手只是用来插等待周期的；也不要把“非互锁最快”当成“非互锁最可靠”——最快和最可靠往往是反的。

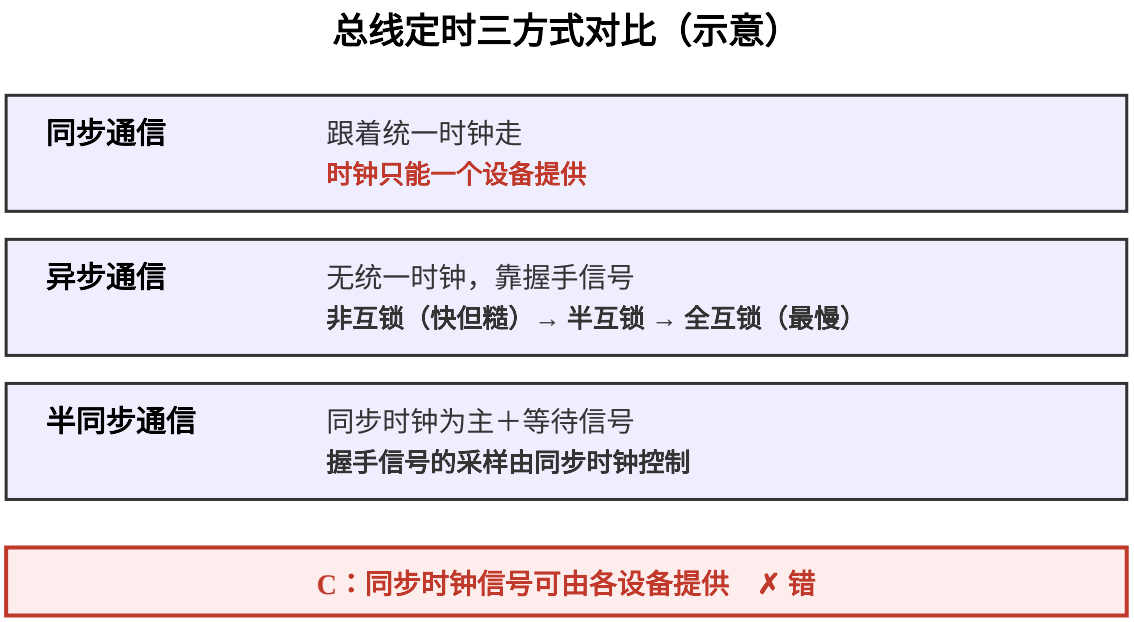
正确选项为什么对： 同步通信只能有一个统一时钟，多设备提供时钟会乱套，所以 C 错，题目问错误项，选 C。

错误选项为什么错＋举例：

- A（全互锁协议最慢）：全互锁来回确认两次，握手最完整，确实最慢；这条说法是对的，题目问错误项，所以A不是答案。教训：问错误项时，对的说法要排除。
- B（非互锁协议可靠性最差）：非互锁发了不管对方回不回，丢了也不知道，确实可靠性最差；这条说法是对的，不是答案。教训：最快和最不可靠往往是同一个。
- D（半同步握手采样由同步时钟控制）：半同步的主时钟是同步时钟，握手信号的采样确实由同步时钟控制；这条说法是对的，不是答案。教训：半同步的“半”在等待周期，不在时钟归属。

笔记本一句话： 第19题＋总线定时：同步时钟只能有一个提供者，多设备提供就错；全互锁最慢、非互锁最不可靠、半同步采样归同步时钟。

【图像】



总线定时三方式对比：同步一个统一时钟只能一个提供者，异步靠握手分非互锁半互锁全互锁，半同步主时钟加等待信号握手采样归同步时钟

【定位】 计算机组成原理 · 第6章 总线 · 总线定时

【易混对比】

- "半同步里有握手信号"和"半同步是异步"是两回事：半同步的主时钟还是同步时钟，握手信号只用来插等待周期，采样还是由同步时钟控制。
- "非互锁最快"和"非互锁最可靠"是两回事：非互锁发了就不管对方回不回，速度快但丢了也不知道——最快和最不可靠往往是同一个。
- "同步"和"异步"是两回事：同步是大家跟着同一个时钟走，时钟只能有一个提供者；异步没有统一时钟，靠握手信号一来一回确认。

同步通信只能有一个统一时钟，多设备各提供一个时钟、节拍就乱了——"同步时钟信号可由各设备提供"是错的，选C。

【坑位】

问"错误项"时，对的说法要排除，别顺手选了对的。

A项全互锁来回确认两次、确实最慢，B项非互锁发了不管、确实可靠性最差，D项半同步握手采样确实由同步时钟控制——三条都对，都不是答案。正确做法：先把三条对的划掉，剩下的就是错的；同步方式里抓住"时钟只能有一个提供者"这一条。

【套路】

- 第一步，看同步：统一时钟，只能一个设备提供——C说"可由各设备提供"，错。
- 第二步，看异步互锁：全互锁确认最多、最慢（A对）；非互锁不确认、可靠性最差（B对）。
- 第三步，看半同步：主时钟是同步时钟，握手信号的采样由同步时钟控制（D对）。

同步一时钟，异步看互锁，半同步采样归时钟。

【类比】

同步通信像全班跟着一个节拍器跳舞——节拍器只能有一个，两个人各打一个，舞步就乱了；异步通信像两个人传纸条，传过去、回一句"收到了"才算完；全互锁是传过去、回一句、再确认一句，最慢但最可靠。术语贴回去：同步通信的同步时钟信号只能由一个设备提供。

【口诀】 同步时钟一提供，全锁最慢非锁糙。

【考频】 「总线定时」在 2009–2022 年解析中出现 1 次 · 低频

第 20 题 · 磁盘访问一个扇区的平均存取时间（正确答案：B）

题干与选项：

若磁盘转速为 7200 转/分，平均寻道时间为 8ms,每个磁道包含 1000 个扇区，则访问一个扇区

的平均存取时间大约是 。

- A. 8.1ms
- B. 12.2ms
- C. 16.3ms
- D. 20.5ms

入手点： 把存取时间拆成三段：寻道时间、旋转延迟时间、传送时间；三段加起来就是答案；其中传送一个扇区的时间非常小，几乎可以忽略。

专有名词从零解释：

- 先想“寻道时间”：磁头从当前位置移动到目标磁道的时间；题目直接给平均值 8 毫秒，不用算。
- 再想“旋转延迟时间”：磁头到了目标磁道后，还要等磁盘转到目标扇区转到磁头下面；平均要等半圈；转速越快，半圈时间越短。
- 再想“传送时间”：目标扇区转到磁头下面后，把扇区数据读出来的时间；一个扇区占磁道的一小部分，传送时间就是转过这一小部分的时间。
- 再想“转速”：7200 转每分钟，意思是 1 分钟转 7200 圈；换算成 1 圈的时间，就是 60 秒除以 7200。

考查知识点：

- 先想每转时间：60 秒除以 7200 等于 0.00833 秒，就是 8.33 毫秒；磁盘转一圈要 8.33 毫秒。
- 再想平均旋转延迟：平均等半圈，就是 8.33 除以 2，等于 4.17 毫秒。
- 再想传送时间：每磁道 1000 个扇区，一个扇区占磁道的千分之一；传送时间是 8.33 除以 1000，约 0.01 毫秒，非常小，几乎可以忽略。
- 再想总数：8（寻道）＋4.17（旋转延迟）＋0.01（传送，忽略不计）≈12.2 毫秒，就是 B。
- 拿生活例子验算：找一本书，先走到书架那一排（寻道），再等转盘转到那本书的位置（旋转延迟，平均等半圈），最后把书拿下来（传送，时间很短）；总时间就是三段加起来。
- 边界情况：如果题目问的是“最长存取时间”，旋转延迟要按一整圈算，不是半圈；本题问“平均”，所以按半圈算。
- 易混对子：不要把“转速 7200 转每分钟”直接当成“每转 7200 毫秒”——单位要换算，60 秒除以 7200 才是每转时间；也不要把“每磁道 1000 个扇区”当成“传送时间很大”——一个扇区只占千分之一圈，传送时间很小。

正确选项为什么对： 8＋4.17≈12.2 毫秒，就是 B，选 B。

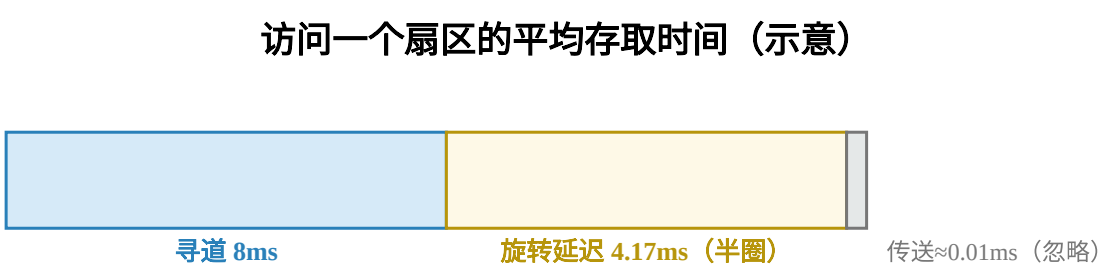
错误选项为什么错＋举例：

- A（8.1 毫秒）：只算了寻道 8 毫秒加传送 0.01 毫秒，漏了平均旋转延迟 4.17 毫秒。教训：旋转延迟是存取时间的大头，不能漏。
- C（16.3 毫秒）：把旋转延迟按一整圈 8.33 毫秒算了，8＋8.33≈16.3；这是最长存取时间，不是平均存取时间。教训：平均按半圈算，最长按整圈算。

- D（20.5 毫秒）：把寻道、整圈旋转、传送都按最大值加起来了，还多算了；平均存取时间不是各项最大值之和。教训：平均和最长要分清。

笔记本一句话： 第 20 题＋磁盘平均存取：寻道＋半圈旋转延迟＋传送（可忽略）；7200 转时半圈 4.17 毫秒，本题 8＋4.17≈12.2 毫秒。

【图像】



每转时间 = 60s ÷ 7200 = 8.33 ms

平均旋转延迟 = 半圈 = 4.17 ms

传送一个扇区 = 8.33 ÷ 1000 ≈ 0.01 ms（可忽略）

总计 = 8 ＋ 4.17 ≈ **12.2 ms**

平均按半圈算；按整圈算的是最长存取时间（16.3ms）。

≈ 12.2 ms，选 B。

磁盘平均存取时间构成：寻道8毫秒加半圈旋转延迟4.17毫秒加传送0.01毫秒可忽略，共约12.2毫秒，平均按半圈最长按整圈

【定位】 计算机组成原理 · 第7章 输入/输出系统 · 磁盘存取时间

【易混对比】

- "平均"和"最长"是两回事：平均旋转延迟按半圈算（4.17毫秒），最长按整圈算（8.33毫秒）——C项16.3毫秒就是按整圈算的最长，不是平均。
- "转速7200转/分"和"每转7200毫秒"是两回事：每转时间等于60秒除以7200得8.33毫秒，单位要换算，不能直接拿转速当时间。
- "传送时间"和"旋转延迟"是两回事：一个扇区只占磁道的千分之一，传送时间约0.01毫秒、可忽略；旋转延迟平均半圈4.17毫秒，是存取时间的大头之一。

寻道8毫秒加半圈旋转延迟4.17毫秒加传送约0.01毫秒可忽略，约12.2毫秒，选B。

【坑位】

旋转延迟是重灾区：平均按半圈算，别按整圈，也别漏掉。

A项8.1毫秒漏了旋转延迟；C项16.3毫秒把旋转延迟按整圈算了，那是最长存取时间；D项是各项最大值乱加。正确做法：先换算每转时间（60秒除以7200），再取半圈得旋转延迟，最后三段相加——传送一个扇区的时间太小，直接忽略。

【套路】

- 第一步，换算每转时间：60秒除以7200转得8.33毫秒每圈。

- 第二步，取半圈：平均旋转延迟等于8.33除以2得4.17毫秒。
- 第三步，三段相加：寻道8加旋转延迟4.17加传送0.01（忽略），约12.2毫秒。

寻道加半圈，传送可忽略，转速先换算。

【类比】

像找一本书：先走到书架那一排（寻道），再等转盘把那本书转到面前（旋转延迟，平均等半圈），最后把书拿下来（传送，时间很短）。术语贴回去：平均存取时间等于平均寻道时间加平均旋转延迟加传送时间。

【口诀】 寻道加半圈，传送可忽略。

【考频】 「平均存取时间」在 2009–2022 年解析中出现 56 次 · 高频

第 21 题 · 中断 I/O 方式控制打印输出时 CPU 与打印控制接口之间不可能交换的信息（正确答案：B）

题干与选项：

在采用中断 I/O 方式控制打印输出的情况下，CPU 和打印控制接口中的 I/O 端口之间交换的信

息不可能是 。

- A. 打印字符
- B. 主存地址
- C. 设备状态
- D. 控制命令

入手点： 先想中断方式下 CPU 和接口各管什么：CPU 管“给什么、问状态、发命令”，接口管“收字符、报状态、执行命令”；主存地址是 CPU 和主存之间的事，不经过接口。

专有名词从零解释：

- 先想“中断 I/O 方式”：也叫程序控制的中断方式；CPU 给设备发命令、设备准备好后发中断信号、CPU 停下手头的事来处理；数据是一次一个字符、由 CPU 亲自搬运的。
- 再想“打印控制接口”：夹在 CPU 和打印机之间的电路；CPU 把要打印的字符、控制命令交给接口，接口再交给打印机；打印机准备好了，接口就向 CPU 发中断。
- 再想“设备状态”：设备现在是忙还是闲、有没有准备好、有没有出错；CPU 要先问状态，才能决定给不给下一个字符。
- 再想“控制命令”：CPU 发给设备的命令，比如“开始打印”“换行”“回车”；设备按命令做事。
- 再想“主存地址”：CPU 访问主存时用的地址，告诉主存“要读第几个单元”；这是 CPU 和主存之间的地址线上的事。

考查知识点：

- 先想打印字符为什么能交换：CPU 要把“打印哪个字符”告诉接口，接口再交给打印机；没有字符，打印机就没东西可打；所以打印字符一定在 CPU 和接口之间交换。
- 再想设备状态为什么能交换：CPU 要问接口“打印机准备好了没有”，接口要把状态报给 CPU；没有状态，CPU 就不知道什么时候给下一个字符；所以设备状态一定交换。
- 再想控制命令为什么能交换：CPU 要给接口发命令，比如“开始打印”；接口按命令驱动打印机；所以控制命令一定交换。
- 再想主存地址为什么不能交换：主存地址是 CPU 访问主存用的，走的是 CPU 和主存之间的地址线，不经过打印接口；打印接口不需要知道数据在主存的哪个地址，它只管收 CPU 给它的字符；所以主存地址不可能在 CPU 和打印接口之间交换，就是 B。
- 拿生活例子验算：CPU 像老板，打印接口像秘书，打印机像打字员；老板给秘书“打这个字”“现在忙不忙”“开始打”这些话，都是老板和秘书之间的事；老板家里的仓库门牌号（主存地址）是老板和仓库之间的事，不用告诉秘书。
- 边界情况：DMA 方式下，接口可以直接访问主存，那时主存地址会出现在接口和主存之间；但本题是中断方式，不是 DMA 方式；中断方式下数据由 CPU 亲自搬，主存地址不经过接口。

- 易混对子：不要把“CPU 从主存取数据”当成“接口也要知道主存地址”——CPU 从主存取数据是 CPU 和主存之间的事，取出来再交给接口，接口只收字符；也不要把“中断方式”和“DMA 方式”搞混，DMA 方式接口才直接和主存打交道。

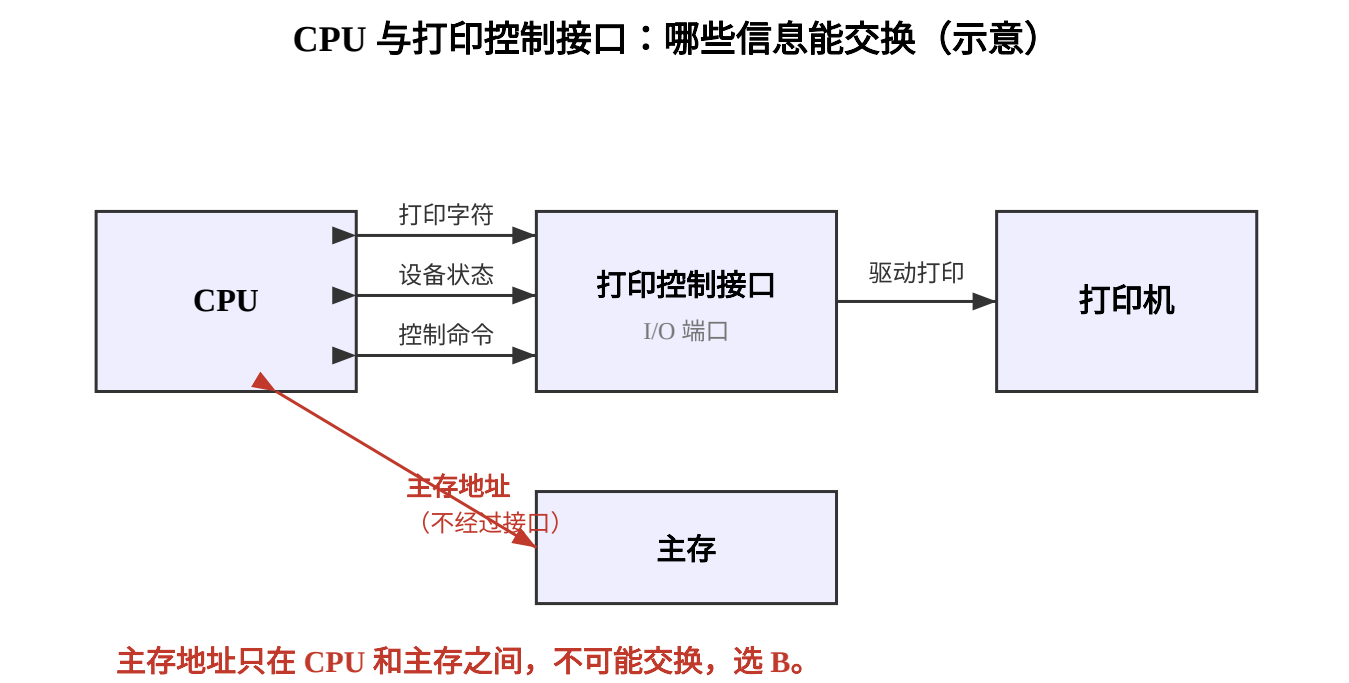
正确选项为什么对： 主存地址是 CPU 和主存之间的事，不经过打印接口，所以不可能交换，选 B。

错误选项为什么错＋举例：

- A（打印字符）：CPU 不给接口字符，接口就没东西交给打印机，打印根本进行不下去；所以打印字符一定交换，不是答案。教训：数据本身是一定要交换的。
- C（设备状态）：CPU 不问状态，就不知道打印机准备好了没有，没法决定给不给下一个字符；所以设备状态一定交换，不是答案。教训：状态是 CPU 控制节奏的依据。
- D（控制命令）：CPU 不发命令，接口就不知道要干什么；所以控制命令一定交换，不是答案。教训：命令是 CPU 驱动设备的手段。

笔记本一句话： 第 21 题＋中断打印交换信息：字符、状态、命令都交换，主存地址是 CPU 和主存之间的事，不经过接口。

【图像】



CPU与打印控制接口间的信息流向：打印字符、设备状态、控制命令在CPU和接口之间双向交换，主存地址只在CPU和主存之间不经过接口

【定位】 计算机组成原理 · 第7章 输入/输出系统 · I/O方式

【易混对比】

- "中断方式"和"DMA方式"是两回事：DMA方式接口直接和主存打交道，才会涉及主存地址；中断方式下数据由CPU亲自一个一个字符搬，主存地址不经过接口。
- "CPU从主存取数据"和"接口知道主存地址"是两回事：取数据是CPU和主存之间的事，CPU取出来再把字符交给接口，接口只管收字符、不用知道字符在主存的哪个地址。

- "数据、状态、命令"和"地址"是两回事：前三者是CPU指挥设备干活必需的，后者是CPU和主存之间的"门牌号"，走不同的线。

主存地址走CPU和主存之间的地址线，不经过打印控制接口——不可能在CPU和接口之间交换，选B。

【坑位】

别把"CPU从主存取数据"当成"接口也要知道主存地址"。

中断方式下，CPU从主存取出要打印的字符，再交给接口；接口只收字符、报状态、执行命令，它不需要、也不可能知道字符在主存的哪个地址。正确做法：按"谁和谁打交道"画线——主存地址只出现在CPU和主存之间，接口那条线上没有它。

【套路】

- 第一步，列三方：CPU、打印控制接口、主存。
- 第二步，画线：CPU和接口之间画打印字符、设备状态、控制命令三条线；CPU和主存之间画主存地址一条线。
- 第三步，找孤线：哪条信息只在CPU和主存之间、不经过接口——主存地址。

三方画线找孤线，地址只在主存间。

【类比】

CPU像老板，打印接口像秘书，打印机像打字员，主存像仓库：老板给秘书"打这个字"（打印字符）、"现在忙不忙"（设备状态）、"开始打"（控制命令），都是老板和秘书之间的事；仓库的门牌号（主存地址）是老板和仓库之间的事，不用告诉秘书。术语贴回去：中断I/O方式下，主存地址不可能在CPU和打印控制接口之间交换。

【口诀】 字符状态命令换，地址只在主存间。

【考频】 「中断I/O」在 2009–2022 年解析中出现 6 次 · 中频

第 22 题 · 内部异常三类划分下相关叙述的错误项（正确答案：D）

题干与选项：

内部异常（内中断）可分为故障（fault）、陷阱（trap）和终止（abort）三类。下列有关内部异常的叙述中，错误的是 。

- A. 内部异常的产生与当前执行指令相关
- B. 内部异常的检测由 CPU 内部逻辑实现
- C. 内部异常的响应发生在指令执行过程中
- D. 内部异常处理后返回到发生异常的指令继续执行

入手点： 先把三类的“返回方式”背下来：故障返回原指令、陷阱返回下一条指令、终止不返回；再看哪条说法把三类混为一谈，那条就是错的。

专有名词从零解释：

- 先想“内部异常”：也叫内中断，是 CPU 执行指令时自己内部出的问题；和外部中断不一样，外部中断是外面设备发来的，内部异常是当前指令执行时产生的。
- 再想“故障”：也叫 fault，比如缺页、除零；特点是“可以修复”——操作系统处理完后，返回原指令重新执行一遍，这次就能成功了。
- 再想“陷阱”：也叫 trap，比如系统调用、断点；特点是“有意为之”——程序主动要求进内核做事，做完后返回下一条指令，继续往下执行，不重复原指令。
- 再想“终止”：也叫 abort，比如硬件严重故障；特点是“没法修复”——操作系统处理完后，不返回原程序，程序就此结束。
- 再想“响应发生在指令执行过程中”：内部异常不是等指令执行完才处理，而是在指令执行到一半、出问题的地方就处理。

考查知识点：

- 先想 A 为什么对：内部异常的产生和当前正在执行的指令有关——缺页是因为这条指令要访问的页不在主存，除零是因为这条指令要除以零；没有这条指令，就没有这个异常；所以 A 对。
- 再想 B 为什么对：内部异常是 CPU 内部逻辑检测到的，比如除零检测、缺页检测，都是 CPU 自己的电路在指令执行时发现的；所以 B 对。
- 再想 C 为什么对：内部异常在指令执行过程中就响应了，不用等指令执行完；比如缺页是在访存那一步就发现的，马上就转去处理；所以 C 对。
- 再想 D 为什么错：D 说“处理后返回发生异常的指令继续执行”；这只对故障成立；陷阱返回的是下一条指令，终止根本不返回；把三类都说成“返回原指令”，就是错的；所以 D 是答案。
- 拿生活例子验算：故障像写作业时发现缺了一页纸，补上纸、重写这一题；陷阱像写作业时主动去问老师问题，问完回来写下一题，不重写刚才那题；终止像作业本被水泡烂了，没法写了，直接不写了；三者的“回来方式”不一样，不能一概而论。
- 边界情况：有些故障处理完后返回原指令，但原指令执行时又出同样的故障，就会反复处理；操作系统要防止这种死循环，但这不改变“故障返回原指令”这个分类。
- 易混对子：不要把“内部异常”和“外部中断”的返回方式搞混——外部中断处理完一般返回被打断的指令继续执行，和故障有点像，但陷阱和终止的返回方式完全不同；也不要把“故障”和“陷阱”搞混，故障重做原指令，陷阱做下一条指令。

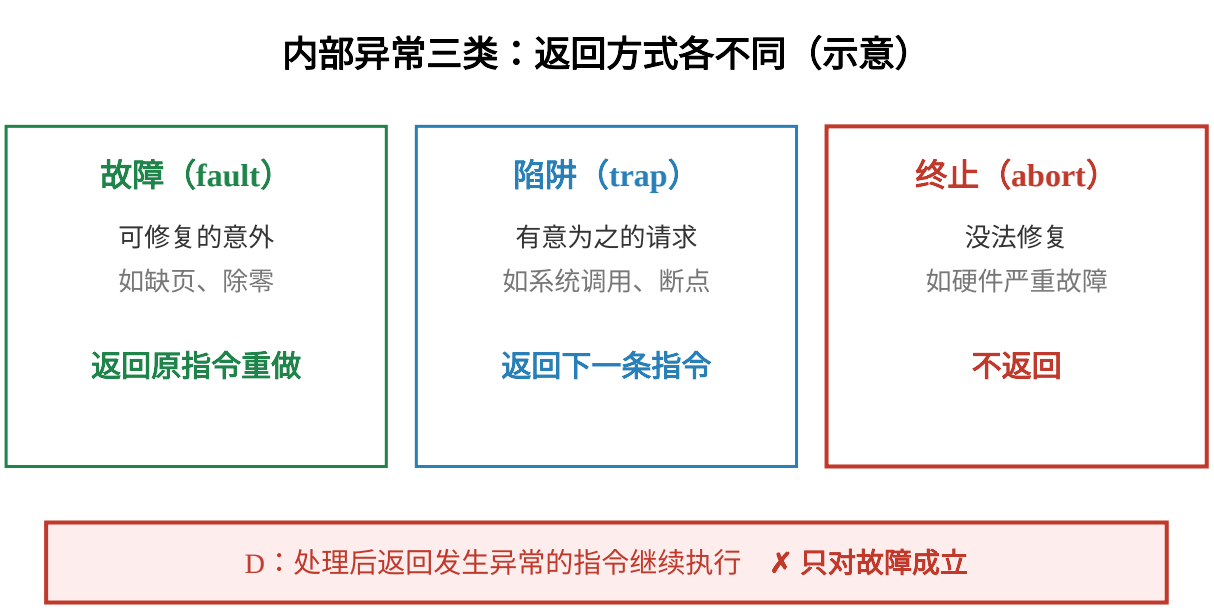
正确选项为什么对： 只有故障返回原指令，陷阱返回下一条、终止不返回；D 把三类说成一样，所以 D 错，题目问错误项，选 D。

错误选项为什么错＋举例：

- A（产生与当前指令相关）：缺页、除零都是当前指令引起的，没有当前指令就没有这个异常；这条说法是对的，不是答案。教训：内部异常的“内部”就是指和当前指令有关。
- B（检测由 CPU 内部逻辑实现）：除零、缺页都是 CPU 自己的电路检测到的，不是外面设备报来的；这条说法是对的，不是答案。教训：内部异常的检测在 CPU 内部。
- C（响应发生在指令执行过程中）：内部异常在指令执行到一半就响应，不等指令执行完；这条说法是对的，不是答案。教训：内部异常不等指令结束。

笔记本一句话： 第 22 题＋内部异常三类返回：故障回原指令、陷阱回下一条、终止不返回；说“都回原指令”就是错的。

【图像】



【定位】 计算机组成原理 · 第7章 输入/输出系统 · 中断与异常

【易混对比】

- "故障"和"陷阱"是两回事：故障是"可以修复"的意外（缺页、除零），处理完返回原指令重做一遍；陷阱是"有意为之"的请求（系统调用、断点），做完返回下一条指令、不重复原指令。
- "内部异常"和"外部中断"是两回事：内部异常和当前指令有关、响应发生在指令执行过程中；外部中断是外面设备发来的，一般等指令执行完再响应。
- "返回原指令"和"返回下一条指令"是两回事：前者把出问题的指令重做一遍，后者跳过它继续往下——D项把三类说成"都返回原指令"，错就错在这里。

只有故障返回原指令：陷阱返回下一条指令，终止根本不返回——D把三类混为一谈，错，选D。

【坑位】

三类返回方式各背各的，别用"故障"的套"陷阱"和"终止"。

D项"处理后返回发生异常的指令继续执行"只对故障成立。陷阱是程序主动进内核办事，办完继续往下走，返回下一条；终止是硬件严重故障、没法修复，处理完不返回原程序。正确做法：先背"故障回原、陷阱回下、终止不回"，再逐条验说法，凡是把三类说成一样的都是错的。

【套路】

- 第一步，背三类返回：故障回原指令，陷阱回下一条指令，终止不返回。
- 第二步，验A：缺页、除零都由当前指令引起——对。
- 第三步，验B、C：检测由CPU内部逻辑实现（B对），响应在指令执行过程中、不等指令结束（C对）。
- 第四步，验D："都返回原指令"只对故障成立，陷阱和终止不是——D错。

故障回原陷阱下，终止不回；混为一谈即错。

【类比】

故障像写作业发现缺了一页纸，补上纸、重写这一题（回原指令）；陷阱像写作业时主动去问老师问题，问完回来写下一题、不重写刚才那题（回下一条）；终止像作业本被水泡烂了，没法写了，直接不写了（不返回）。术语贴回去：故障、陷阱、终止的返回方式各不相同。

【口诀】 故障回原陷阱下，终止不回记清它。

【考频】 「内部异常」在 2009–2022 年解析中出现 1 次 · 低频

第 23 题 · 处理外部中断时操作系统应保存的内容（正确答案：B）

题干与选项：

处理外部中断时，应该由操作系统保存的是 。

- A. 程序计数器(PC)的内容
- B. 通用寄存器的内容
- C. 快表(TLB)中的内容
- D. Cache 中的内容

入手点： 先分清楚“硬件保存什么、操作系统保存什么”：硬件只保存最关键的一个——断点；剩下的现场，包括通用寄存器，都是操作系统保存的；硬件自己管的东西，操作系统不用保存。

专有名词从零解释：

- 先想“外部中断”：也叫外中断，是外面设备发来的信号，比如键盘按下了、打印机准备好了；CPU 收到后，停下手头的事，先去处理设备的事。
- 再想“断点”：被打断的程序执行到哪一条指令；处理完中断后，要回到断点继续执行；断点就是程序计数器（PC）的值。
- 再想“程序计数器”：也叫 PC，存着下一条要执行的指令的地址；CPU 每执行一条指令，PC 就指向下一条。
- 再想“通用寄存器”：CPU 里一堆通用的小存储单元，程序算中间结果、存地址都用它们；被打断时，它们里面的值就是程序的“现场”。
- 再想“块表”：也叫 TLB，是页表的小而快的副本，由硬件管理；地址转换时查它，不用操作系统每次都保存。
- 再想“Cache”：CPU 和主存之间的小而快的存储器，由硬件管理；里面的数据是硬件按规则换进换出的，操作系统不用在中断时保存它。

考查知识点：

- 先想 PC 为什么不是操作系统保存的：外部中断发生时，硬件自动把断点（PC 的值）保存起来（压栈或者存到特定寄存器）；操作系统不用操心 PC，回来时硬件会把 PC 恢复。
- 再想通用寄存器为什么是操作系统保存的：通用寄存器里的值是被打断程序的中间结果；中断处理程序自己也要用通用寄存器，会把原来的值覆盖掉；所以操作系统在处理中断前，要先把通用寄存器里的值保存起来，处理完再恢复；不保存，被打断的程序回来后算出来的结果就错了。
- 再想 TLB 为什么不是操作系统保存的：TLB 由硬件管理，地址转换时硬件自己查、自己换；中断处理不需要把 TLB 的内容存起来，处理完 TLB 还在那，照样能用。
- 再想 Cache 为什么不是操作系统保存的：Cache 由硬件管理，数据换进换出是硬件按规则做的；中断处理不需要把 Cache 的内容存起来，处理完 Cache 还在那，照样能用。
- 拿生活例子验算：写作业时被叫去接电话，电话是外面来的事（外部中断）；接电话前，家人帮你把“写到第几题”记下来（硬件保存 PC），你自己把草稿纸上的中间结果收好（操作系统保存通用寄存器）；书架上的书（TLB、Cache）不用收，回来还在那。
- 边界情况：有些机器上，硬件除了保存 PC，还会保存一两个关键寄存器；但通用寄存器的保存，永远是操作系统的事；题目问“操作系统应保存的”，答案就是通用寄存器。
- 易混对子：不要把“硬件保存 PC”和“操作系统保存 PC”搞混——PC 是硬件保存的，不是操作系统保存的；也不要把“TLB、Cache 也要保存”当成答案——它们由硬件管理，不用操作系统在中断时保存。

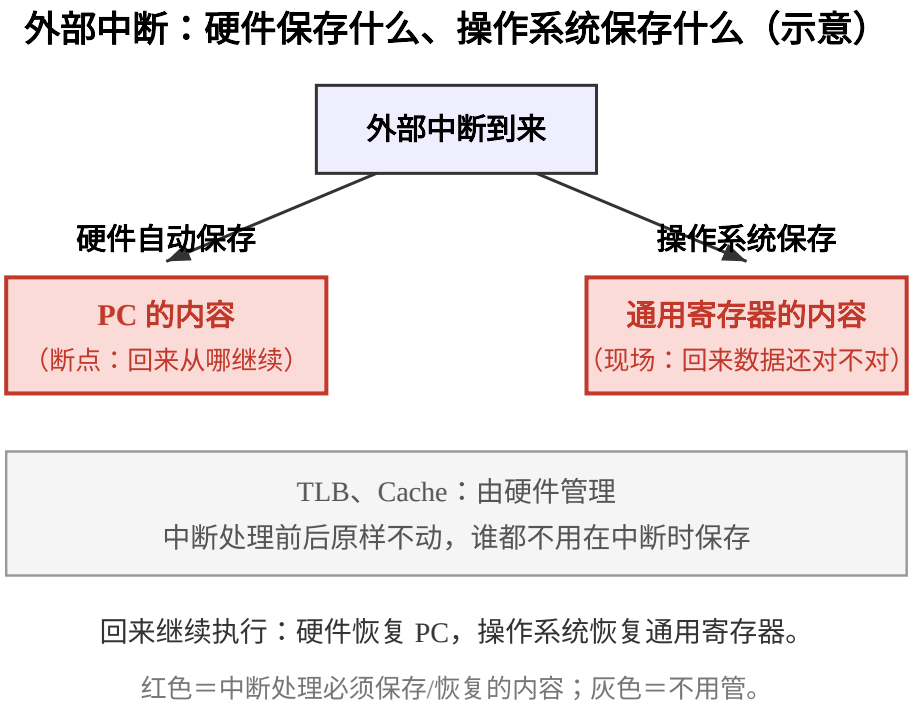
正确选项为什么对： 操作系统的现场保护保存的是通用寄存器，选 B。

错误选项为什么错＋举例：

- A（PC 的内容）：PC 由硬件在中断发生时自动保存，操作系统不用保存；题目问“操作系统应保存的”，PC 不是。教训：硬件和操作系统的分工要分清。
- C（TLB 中的内容）：TLB 由硬件管理，地址转换时硬件自己用，中断处理不用操作系统保存它。教训：硬件管的东西，操作系统不用在中断时存。
- D（Cache 中的内容）：Cache 由硬件管理，数据换进换出是硬件的规则，中断处理不用操作系统保存它。教训：Cache 也是硬件管的。

笔记本一句话： 第 23 题＋外部中断现场保护：硬件保存断点 PC，操作系统保存通用寄存器；TLB、Cache 由硬件管，不用操作系统存。

【图像】



中断现场保存分工图：硬件保存断点 PC，操作系统保存通用寄存器，TLB 与 Cache 由硬件管理、无需保存

【定位】 操作系统 · 第5章 输入/输出管理 · 中断处理与现场保护

【易混对比】

- "硬件保存"和"操作系统保存"是两回事：硬件在中断发生的瞬间自动保存断点（PC），操作系统插不上手；操作系统保存的是中断处理程序自己会覆盖掉的东西——通用寄存器。
- "通用寄存器"和"TLB"是两回事：通用寄存器里是被打断程序的中间结果，不保存就被覆盖；TLB 由硬件管理、随用随查，中断前后原样不动，不用保存。
- "保存断点"和"保存现场"是两回事：断点解决"回来从哪条指令继续"（PC），现场解决"回来数据还对不对"（通用寄存器）。

外部中断到来时，硬件自动保存断点 PC；操作系统在进入中断处理程序之前保存通用寄存器、返回之前恢复；TLB 和 Cache 由硬件管理，操作系统不用管。

【坑位】

题干问的是"操作系统"保存的，别把硬件自动保存的 PC 选进去。

PC 在中断发生的瞬间就被硬件自动保存（压栈或存入特定寄存器），操作系统的中断处理程序根本不用碰它。正确做法：先划清"硬件干什么、操作系统干什么"的界——硬件保断点，操作系统保会被自己踩坏的通用寄存器。

【套路】

- 第一步，定主体：题干问"操作系统保存的"，先把"硬件自动保存"的选项划掉（PC 出局）。
- 第二步，找"会被踩坏的"：中断处理程序自己也要用通用寄存器，用之前必须先保存被打断程序的值（通用寄存器入选）。
- 第三步，排除硬件管的：TLB、Cache 由硬件管理，中断时不用操作系统保存（C、D 出局）。

硬件保断点，系统保通用；TLB、Cache 硬件管，不用存。

【类比】

课堂上老师正在板书，校长进门有事（外部中断）。门卫先记下"讲到第几页"（硬件保存 PC），老师自己把黑板上的半截板书抄到纸上再擦黑板（操作系统保存通用寄存器），腾出黑板给校长写通知；墙上的课程表（TLB、Cache）不用动，回来还在原处。术语贴回去：断点就是 PC 的值，半截板书就是通用寄存器里的中间结果。

【口诀】 外中断，硬件保 PC；通用寄存器，系统来保底。

【考频】 「外部中断」在 2009–2022 年解析中出现 12 次 · 高频

第 24 题 · 不可能导致 CPU 从用户态变为内核态的指令（正确答案：C）

题干与选项：

假定下列指令已装入指令寄存器。则执行时不可能导致 CPU 从用户态变为内核态(系统态)的

是 。

- A. DIV R0,R1 ; (R0)/(R1)→R0
- B. INT n ; 产生软中断
- C. NOT R0 ; 寄存器 R0 的内容取非
- D. MOV R0,addr ; 把地址 addr 处的内存数据放入寄存器 R0 中

入手点： 一条一条看这条指令有没有可能“出事”或者“主动要求进内核”：能出异常、能陷入的，就可能进内核；纯运算、绝不会出事的，就不可能进内核。

专有名词从零解释：

- 先想“用户态、内核态”：CPU 的两种运行状态；用户态下只能执行普通指令，不能碰特权指令和关键资源；内核态下可以执行所有指令，操作系统跑在内核态。
- 再想“异常”：指令执行时出问题，比如除零、缺页；出了异常，CPU 就转去内核态，让操作系统处理。
- 再想“陷入”：也叫 trap，程序主动要求进内核，比如系统调用指令；执行陷入指令，CPU 就从用户态切换到内核态。
- 再想“除零异常”：除法指令的除数是 0，算不出来，CPU 就产生异常，进内核处理。
- 再想“缺页异常”：指令要访问的页不在主存里，CPU 就产生异常，进内核把页调进来。

考查知识点：

- 先想 DIV 为什么可能：DIV 是除法指令，如果除数是 0，就会产生除零异常，CPU 进内核处理；所以 DIV 可能导致切换。
- 再想 INT n 为什么可能：INT n 就是陷入指令，专门用来主动要求进内核；执行它，CPU 一定从用户态切换到内核态；所以 INT n 可能导致切换。
- 再想 MOV 为什么可能：MOV 是传送指令，如果它要访问的地址对应的页不在主存里，就会产生缺页异常，CPU 进内核处理；所以 MOV 可能导致切换。
- 再想 NOT 为什么不可能：NOT 是按位取反的纯运算，只对寄存器里的数做运算，不访问主存、不除法、不陷入；它绝不会产生异常，也不会主动要求进内核；所以 NOT 不可能导致切换，就是 C。
- 拿生活例子验算：DIV 像做除法，万一除数是 0，就得出事找老师（进内核）；INT n 像主动举手找老师；MOV 像去仓库拿东西，万一东西不在仓库，就得出事找老师；NOT 像在草稿纸上把 0 改成 1、1 改成 0，纯纸上运算，永远不会出事找老师。
- 边界情况：MOV 如果访问的是已经在主存里的地址，就不会缺页，不进内核；但题目问“不可能”，MOV 是“可能”进内核，不是“一定”进内核；NOT 是“绝不可能”，两者要分清。
- 易混对子：不要把“可能”和“一定”搞混——DIV、MOV 是可能进内核，不是每次都进；NOT 是绝不可能进，这是唯一的区别；也不要以为“所有指令都可能缺页”——NOT 不访问主存，根本没有缺页这回事。

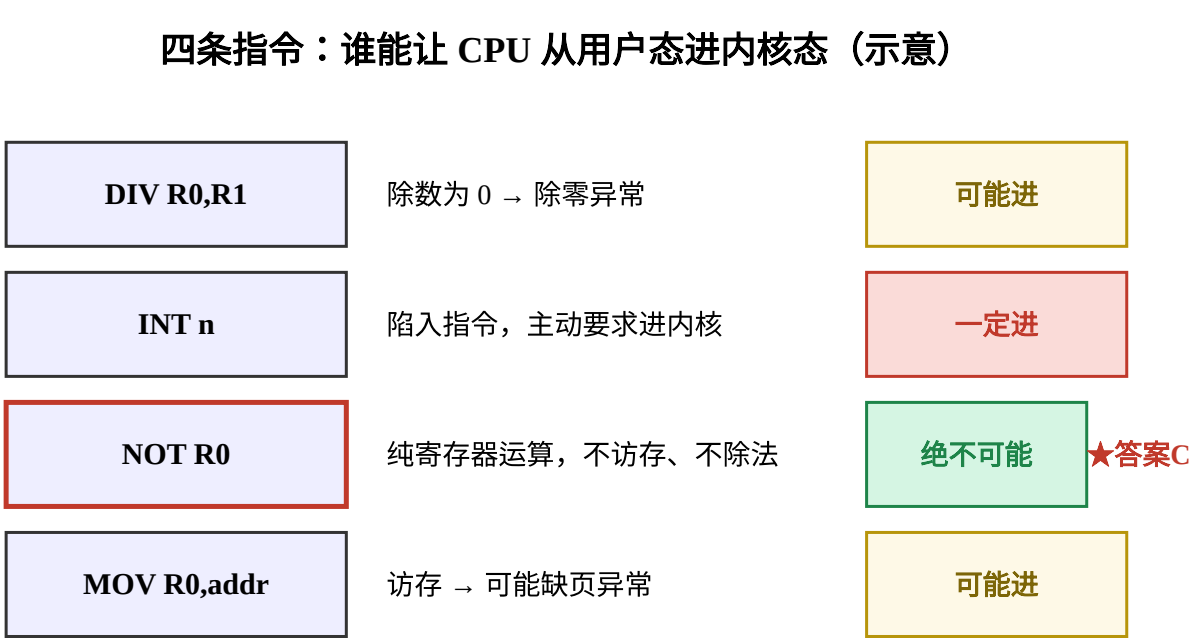
正确选项为什么对： NOT 是纯运算，绝不会引起异常或陷入，不可能导致切换，选 C。

错误选项为什么错＋举例：

- A（DIV）：除数是 0 时产生除零异常，进内核；所以 DIV 可能导致切换，不是答案。教训：除法指令要想到除零。
- B（INT n）：INT n 本身就是陷入指令，执行它就进内核；所以它可能导致切换，不是答案。教训：陷入指令就是主动进内核的。
- D（MOV）：访存时可能缺页，缺页异常进内核；所以 MOV 可能导致切换，不是答案。教训：访存指令要想到缺页。

笔记本一句话： 第 24 题＋不可能进内核的指令：纯运算的 NOT 绝不会异常、不会陷入；DIV 想除零、INT 是陷入、MOV 想缺页，都可能进。

【图像】



四条指令进内核判定图：DIV 可能进（除零异常）、INT n 一定进（陷入）、NOT 绝不可能进、MOV 可能进（缺页异常）

【定位】 操作系统 · 第1章 计算机系统概述 · 用户态与内核态的切换

【易混对比】

- "可能进内核"和"一定进内核"是两回事：DIV、MOV 是"可能"（除零、缺页时才进）；INT n 是"一定"（陷入指令执行即切换）；NOT 是"绝不可能"。
- "访存指令"和"纯运算指令"是两回事：访存的 MOV 可能缺页；只碰寄存器的 NOT 不访存，缺页这回事和它无关。
- "陷入"和"异常"是两回事：INT n 是程序主动要求进内核；DIV 除零、MOV 缺页是执行中出事、被动进内核。

NOT 是纯寄存器运算，不访存、不做除法、不执行陷入指令，既不会触发异常也不会主动陷入，是四条里唯一绝不可能让 CPU 从用户态进内核态的。

【坑位】

把 MOV 当成"绝不可能"——觉得传送数据很普通，不会出事。

MOV 要访存，访的页可能不在主存里，缺页异常一来就进内核。正确做法：看到访存指令先想缺页，看到除法指令先想除零，看到 INT 先想陷入。

【套路】

- 第一步，找"一定进"的：INT n 是陷入指令，执行必进内核，先排除。
- 第二步，找"可能进"的：DIV 想除零，MOV 想缺页，都可能进，排除。
- 第三步，剩下的就是答案：NOT 纯运算，什么都不碰，不可能进。

陷入一定进，除法访存可能进，纯运算绝不可能进。

【类比】

用户态像普通员工区，内核态像机房。进机房要么主动申请（INT n 举手申请），要么出事故被保安带进去（DIV 除零、MOV 缺页）。NOT 像在自己工位上算 1+1，不碰设备不惹事，保安永远不会来找他。术语贴回去：主动申请就是陷入，被动带走就是异常。

【口诀】 陷入必进，除法缺页可能进；NOT 纯运算，内核门都不进。

【考频】 「用户态」在 2009–2022 年解析中出现 15 次 · 高频

第 25 题 · 会使进程从执行态变为就绪态的事件（正确答案：D）

题干与选项：

下列选项中，会导致进程从执行态变为就绪态的事件是

- A. 执行 P(wait)操作
- B. 申请内存失败
- C. 启动 I/O 设备
- D. 被高优先级进程抢占

入手点： 先把三种状态转移的方向记牢：执行→阻塞是“等东西”，执行→就绪是“被抢了 CPU 但不用等东西”，阻塞→就绪是“等的东西到了”；再看每个事件属于哪一种。

专有名词从零解释：

- 先想“执行态”：进程正在占用 CPU、正在跑的状态。
- 再想“就绪态”：进程已经准备好了、万事俱备，只等 CPU 空出来就能跑的状态。
- 再想“阻塞态”：也叫等待态，进程在等某个东西（比如等输入输出完成、等资源），东西没到之前不能跑，哪怕 CPU 空着也不能跑。
- 再想“P 操作”：也叫 wait 操作，是申请资源的动作；如果资源不够，进程就阻塞起来等资源。
- 再想“抢占”：高优先级进程来了，把 CPU 从当前进程手里抢走；被抢的进程不用等任何东西，只是没了 CPU，所以变成就绪态。

考查知识点：

- 先想 P 操作为什么不是：执行 P 操作时，如果资源不够，进程要等资源，这就是执行→阻塞，不是执行→就绪；如果资源够，进程继续执行，状态也不变；所以 P 操作不会导致执行→就绪。
- 再想申请内存失败为什么不是：申请内存失败，进程要等内存，这也是等东西，就是执行→阻塞，不是执行→就绪。
- 再想启动输入输出设备为什么不是：启动输入输出设备本身，只是一个“开始干活”的动作；进程启动完输入输出，如果它要等输入输出的结果，才会阻塞；如果它不等，就继续执行；启动这个动作本身不把进程变成就绪态。
- 再想被高优先级进程抢占为什么是：高优先级进程来了，当前进程被迫让出 CPU；它不用等任何东西，只是没了 CPU；没了 CPU 但准备好了，就是就绪态；所以被抢占才导致执行→就绪，就是 D。
- 拿生活例子验算：执行态像正在讲台上讲课的老师；就绪态像在台下准备好、等着上台的老师；阻塞态像在等粉笔送来的老师；被高优先级进程抢占，像校长来了、让当前老师先下来等着——老师不用等粉笔，只是等着再上台，所以是就绪态；执行 P 操作没拿到资源，像老师发现粉笔不够、要等粉笔送来，所以是阻塞态。
- 边界情况：时间片用完也会导致执行→就绪（轮转调度里，时间片到了，进程被换下来等下一轮）；本题选项里没有时间片用完，但道理和被抢占一样——都是“没了 CPU、不用等东西”。
- 易混对子：不要把“执行→阻塞”和“执行→就绪”搞混——关键看“要不要等东西”：要等东西就是阻塞，不要等东西、只是没了 CPU 就是就绪；也不要把“启动输入输出”当成“等待输入输出”——启动只是开始，等待是启动之后、等结果的那一步。

正确选项为什么对： 被高优先级进程抢占，进程让出 CPU、不用等东西，变成就绪态，选 D。

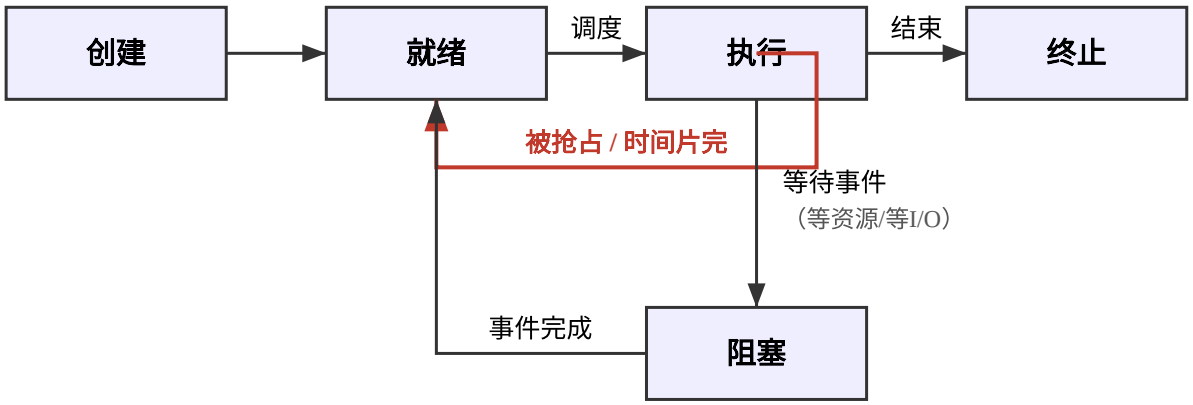
错误选项为什么错＋举例：

- A（执行 P 操作）：P 操作拿不到资源时，进程要等资源，是执行→阻塞；拿得到资源时，进程继续执行，状态不变；两种情况都不是执行→就绪。教训：等资源就是阻塞。
- B（申请内存失败）：申请失败意味着内存不够，进程要等内存，也是执行→阻塞，不是执行→就绪。教训：申请失败要等，就是阻塞。
- C（启动输入输出设备）：启动本身不改变进程状态；进程如果等输入输出结果，那是启动之后等待的那一步导致阻塞，不是启动本身导致就绪。教训：启动和等待是两步，别混了。

笔记本一句话： 第 25 题＋执行→就绪：被抢占才变就绪（没了 CPU、不用等东西）；等资源、等内存、等输入输出都是变阻塞。

【图像】

进程五状态转换：执行→就绪 vs 执行→阻塞（示意）



执行→就绪：没了 CPU、不用等东西（被抢占）——本题答案 D。

执行→阻塞：要等东西（P 操作、申请内存失败、等 I/O 结果）。

要等东西→阻塞；只丢 CPU→就绪。

进程五状态转换图：被抢占导致执行态变为就绪态（红色标出），等事件导致执行态变为阻塞态

【定位】 操作系统 · 第2章 进程管理 · 进程状态转换

【易混对比】

- "执行→就绪"和"执行→阻塞"是两回事：前者是没了 CPU、不用等东西（被抢占、时间片用完）；后者是要等东西（等资源、等 I/O、等内存）。
- "启动 I/O"和"等待 I/O 完成"是两回事：启动只是发出命令，状态不变；等结果的那一步才进阻塞。
- "P 操作拿到资源"和"P 操作没拿到资源"是两回事：拿到就继续执行，拿不到才阻塞；两种情况都不是变就绪。

执行→就绪只有一种画像：失去 CPU 但无事可等。被高优先级进程抢占时，进程只是被迫让出 CPU，万事俱备只差 CPU，所以进就绪态。

【坑位】

把"申请内存失败"选成执行→就绪——觉得"失败了就退回去等着"。

申请失败意味着内存不够、要等内存，"等东西"就是阻塞的定义。正确做法：看到事件先问"要不要等东西"——要等就是阻塞，不要等、只是没了 CPU 就是就绪。

【套路】

- 第一步，问"等不等"：事件发生后，进程要不要等某个东西（资源、I/O、内存）。
- 第二步，对号入座：要等→阻塞；不等、只是没了 CPU→就绪。
- 第三步，验 D：被高优先级进程抢占——不用等任何东西，只是让出 CPU→就绪。

要等东西进阻塞，只丢 CPU 进就绪。

【类比】

执行态像正在台上唱歌的歌手，就绪态像在后台候场、妆发完毕的歌手，阻塞态像在等话筒修好的歌手。高优先级进程抢占像大牌歌手空降、当前歌手被请下台——他不用等任何东西，只是等着再上台，所以是候场（就绪）；P 操作拿不到资源像话筒坏了得等修好，所以是等话筒（阻塞）。术语贴回去：候场就是就绪态，等话筒就是阻塞态。

【口诀】 被抢只丢 CPU，等物才进阻塞；执行变就绪，抢占是正解。

【考频】 「进程状态」在 2009–2022 年解析中出现 8 次 · 中频

第 26 题 · 死锁避免系统与死锁检测系统相关叙述的正误判断（正确答案：B）

题干与选项：

若系统 S1 采用死锁避免方法，S2 采用死锁检测方法。下列叙述中，正确的是 。

Ⅰ．S1 会限制用户申请资源的顺序，而 S2 不会

Ⅱ．S1 需要进程运行所需资源总量信息，而 S2 不需要

Ⅲ．S1 不会给可能导致死锁的进程分配资源，而 S2 会

- A. 仅 Ⅰ、Ⅱ
- B. 仅 Ⅱ、Ⅲ
- C. 仅 Ⅰ、Ⅲ
- D. Ⅰ、Ⅱ、Ⅲ

入手点： 先把“避免、检测、预防”三个词分清：预防是事先定规矩、破坏死锁条件；避免是每次分配前先算一算、会进不安全状态就不给；检测是不管、事后查、查到了再恢复；三条说法要一条一条对号入座。

专有名词从零解释：

- 先想“死锁”：几个进程互相等对方手里的资源，谁也走不下去，僵在那里的状态；比如 A 等 B 的资源，B 等 A 的资源，两个人互相等，永远等下去。
- 再想“死锁预防”：事先定规矩，破坏死锁的四个条件之一；比如规定申请资源必须按固定顺序申请，这就是破坏“循环等待”条件。
- 再想“死锁避免”：每次分配资源前，先算一算“给了之后，系统会不会进不安全状态”；会进不安全状态就不给，不会进才给；用的办法叫银行家算法。
- 再想“死锁检测”：平时不管，任由进程申请；隔一段时间查一次“现在是不是死锁了”，查到了再想办法恢复（比如撤销一个进程、抢占资源）。
- 再想“不安全状态”：系统的一种状态，在这种状态下，有可能走到死锁，但还没死锁；避免方法的原则就是“不进不安全状态”。
- 再想“资源总量信息”：每个进程最多要用多少资源，也就是 Max；银行家算法算“给不给”时，要知道每个进程最多还要多少，才能算出来。

考查知识点：

- 先想 Ⅰ 为什么错：Ⅰ 说“避免系统会限制申请顺序、检测系统不会”；限制申请顺序是死锁预防的做法，不是避免的做法；避免用银行家算法，不限制顺序，只算“给不给”；所以 Ⅰ 把预防的做法安到了避免头上，是错的。
- 再想 Ⅱ 为什么对：避免要用银行家算法算“给不给”，算的时候要知道每个进程最多还要多少资源，也就是资源总量信息；检测是事后查“现在是不是死锁了”，查的时候看的是“现在谁占了什么、谁在等什么”，不需要知道每个进程最多要多少；所以 Ⅱ 对。
- 再想Ⅲ为什么对：避免的原则是“不进不安全状态”，所以可能导致死锁的分配就不给；检测是事后才查，分配时不管会不会死锁，该给就给，死锁了事后再恢复；所以Ⅲ对。
- 再想组合：Ⅰ 错、Ⅱ 对、Ⅲ对，就是“仅 Ⅱ、Ⅲ”，就是 B。

- 拿生活例子验算：避免像银行放贷，每贷一笔先算“贷出去后，银行会不会周转不开”，会周转不开就不贷；检测像银行不管、贷出去再说，月底查账发现坏账了再处理；预防像银行规定“借钱必须按固定顺序借”，从规矩上杜绝互相等。
- 边界情况：避免方法要求进程提前声明最多用多少资源，实际中很难做到，所以避免方法用得少；检测方法不要求提前声明，用得多一些；但考试只考定义和区别，不考用得多不多。
- 易混对子：不要把“预防”和“避免”搞混——预防是事先定规矩，破坏条件；避免是每次分配前算一算，不进不安全状态；“限制申请顺序”是预防，不是避免；也不要把“检测”和“避免”的“要不要总量信息”搞混——避免要总量信息，检测不要。

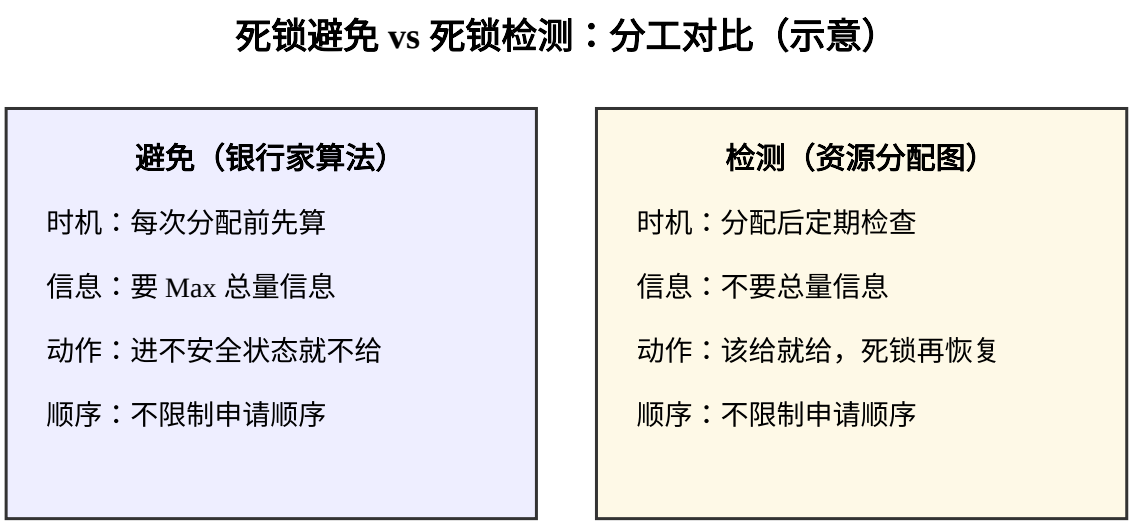
正确选项为什么对： Ⅰ错、Ⅱ对、Ⅲ对，组合是“仅Ⅱ、Ⅲ”，选 B。

错误选项为什么错＋举例：

- A（仅Ⅰ、Ⅱ）：把Ⅰ也算对了；Ⅰ说的“限制申请顺序”是预防的做法，不是避免的做法；避免不限顺序，只算给不给。教训：限制顺序是预防，不是避免。
- C（仅Ⅰ、Ⅲ）：同样把Ⅰ算对了，漏了Ⅱ；Ⅱ是对的，避免要总量信息、检测不要。教训：Ⅰ错、Ⅱ对，不能把Ⅰ和Ⅱ的位置换了。
- D（Ⅰ、Ⅱ、Ⅲ）：把Ⅰ也算对了；Ⅰ是错的，三条不可能全对。教训：全对的组合要先验每一条，错一条就不能选全对。

笔记本一句话： 第 26 题＋避免对检测：限制顺序是预防不是避免；避免要总量信息、拒绝进不安全状态的分配，检测事后才查；Ⅰ错Ⅱ对Ⅲ对。

【图像】



死锁避免与死锁检测对比图：避免用银行家算法事前算、检测用资源分配图事后查

【定位】 操作系统 · 第2章 进程管理 · 死锁处理策略（预防/避免/检测/解除）

【易混对比】

- "避免"和"预防"是两回事：预防是事先定规矩、破坏死锁条件（比如限制申请顺序）；避免是每次分配前用银行家算法算一算，不进不安全状态才给——Ⅰ错就错在把预防的"限制顺序"安到避免头上。
- "避免"和"检测"是两回事：避免是事前算、要总量信息、拒绝危险分配；检测是事后查、只看占有与等待的现状、不要总量信息、分配时照给不误。
- "不安全状态"和"死锁状态"是两回事：不安全是"可能走向死锁"，还没死；避免的红线是不进不安全，而不是等死锁了才动手。

Ⅰ把预防的"限制申请顺序"张冠李戴给避免，所以错；Ⅱ对（避免要 Max 总量信息，检测只看现状）；Ⅲ对（避免拒绝进不安全状态的分配，检测分配时照给、事后恢复）——组合"仅Ⅱ、Ⅲ"。

【坑位】

Ⅰ读着最顺眼——"避免系统会限制申请顺序"，一顺就选了含Ⅰ的组合。

限制申请顺序是破坏"循环等待"条件的死锁预防做法；避免用的是银行家算法，不限制顺序、只算给不给。正确做法：看到"限制顺序"四个字，先对到"预防"名下，再看主语是不是预防。

【套路】

- 第一步，先贴标签：限制顺序→预防；银行家/不安全→避免；事后查/恢复→检测。
- 第二步，一条一条验：Ⅰ含"限制顺序"但主语是避免→错；Ⅱ"避免要总量、检测不要"→对；Ⅲ"避免拒分、检测照分"→对。
- 第三步，拼组合：Ⅰ错Ⅱ对Ⅲ对→仅Ⅱ、Ⅲ。

限制顺序是预防，银行家算不安全，检测事后才查账。

【类比】

银行家算法像银行放贷：每贷一笔先算"贷出去后会不会周转不开"（不安全状态），会就不贷；但银行不规定你"必须先借 A 再借 B"（不限制顺序）——那是另一家"预防银行"的规矩（限制顺序、破坏循环等待）。检测像银行闭眼放贷、月底查坏账，查到再追回。术语贴回去：周转不开就是不安全状态，查坏账就是死锁检测，追回就是死锁恢复。

【口诀】 预防定规矩，避免算一算，检测事后查；总量信息，避免才要它。

【考频】 「死锁避免」在 2009–2022 年解析中出现 3 次 · 中频

第 27 题 · 限定页框数并采用 LRU 算法时下一页访问应淘汰的页号（正确答案：A）

题干与选项：

系统为某进程分配了 4 个页框，该进程已访问的页号序列为 2,0,2,9,3,4,2,8,2,4,8,4,5。若进程要

访问的下一页的页号为 7，依据 LRU 算法，应淘汰页的页号是 。

- A. 2
- B. 3
- C. 4
- D. 8

入手点： 按访问顺序一步一步模拟框里的变化：命中了就把这个页移到“最近用过”的那一端，没命中、框满了就把“最久没用过”的那一端淘汰；模拟到最后，看框里最久没用过的那个页是谁。

专有名词从零解释：

- 先想“页框”：主存里放一页的格子；进程分到几个页框，主存里就能同时放几页。
- 再想“LRU 算法”：也叫最近最久未使用算法；淘汰的原则是“谁最久没被用过，就淘汰谁”；最近刚用过的页，留着最有价值。
- 再想“命中”：要访问的页已经在框里，不用去外存拿，直接用；命中后，这个页就变成“最近用过”的。
- 再想“缺页”：要访问的页不在框里，要去外存拿；框没满就直接放进去，框满了就要先淘汰一个再放进去。

考查知识点：

- 先想模拟过程（框内按久远→最近排）：
- 访问 2：框是空的，放进 2，框内是 [2]。
- 访问 0：框没满，放进 0，框内是 [2,0]。
- 访问 2：命中，把 2 移到最近一端，框内是 [0,2]。
- 访问 9：框没满，放进 9，框内是 [0,2,9]。
- 访问 3：框没满，放进 3，框内是 [0,2,9,3]。
- 访问 4：框满了，淘汰最久没用过的 0，放进 4，框内是 [2,9,3,4]。
- 访问 2：命中，把 2 移到最近一端，框内是 [9,3,4,2]。
- 访问 8：框满了，淘汰最久没用过的 9，放进 8，框内是 [3,4,2,8]。
- 访问 2：命中，把 2 移到最近一端，框内是 [3,4,8,2]。
- 访问 4：命中，把 4 移到最近一端，框内是 [3,8,2,4]。
- 访问 8：命中，把 8 移到最近一端，框内是 [3,2,4,8]。
- 访问 4：命中，把 4 移到最近一端，框内是 [3,2,8,4]。
- 访问 5：框满了，淘汰最久没用过的 3，放进 5，框内是 [2,8,4,5]。
- 再想下一页 7：框满了，淘汰最久没用过的 2，放进 7；所以淘汰的是 2，就是 A。
- 拿生活例子验算：书桌上只能放 4 本书，最久没翻的那本最先被拿下去；刚翻过的书留着，因为很可能还要翻；每次翻一本书，就把它移到“最近翻过”的那一端。
- 边界情况：如果下一页命中了，就不用淘汰；本题下一页 7 不在框里，所以要淘汰；如果框没满，也不用淘汰，直接放进去。

- 易混对子：不要把“最久没用过”和“最先进来”搞混——LRU 看的是“最后一次用的时间”，不是“第一次进来的时间”；命中会刷新“最后一次用的时间”，所以命中过的页不一定是最久没用过的。

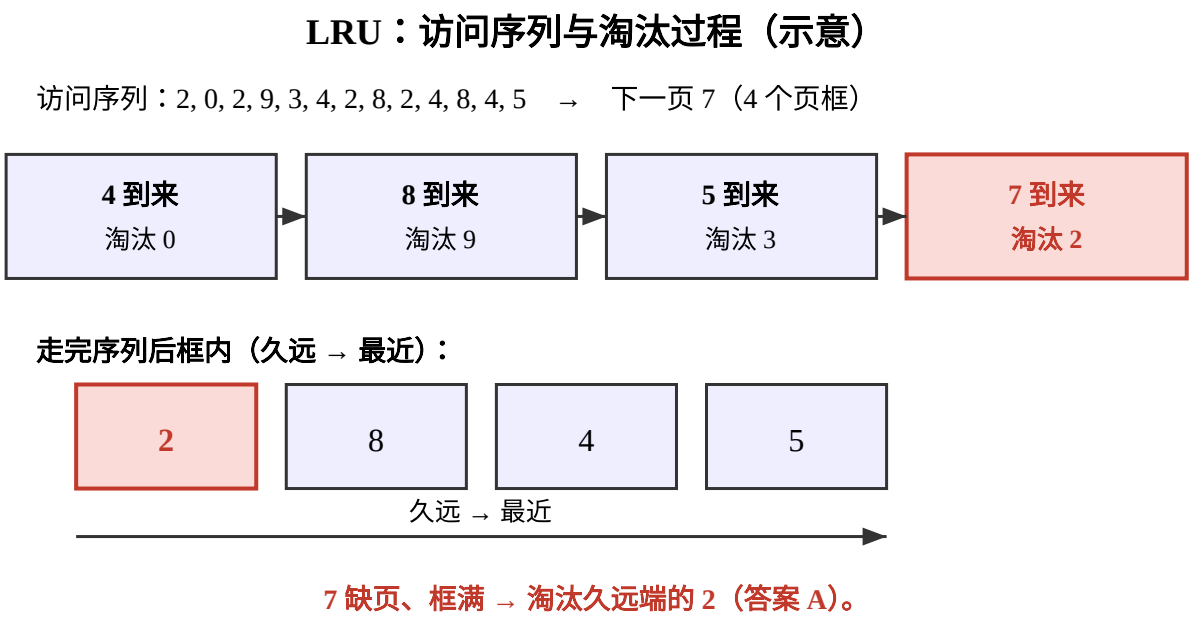
正确选项为什么对： 模拟到最后，框内从久远到最近是 2、8、4、5，最久没用过的是 2，下一页 7 到来时淘汰 2，选 A。

错误选项为什么错＋举例：

- B（3）：3 是访问 5 到来时淘汰的那个，不是下一页 7 到来时淘汰的；模拟到最后，3 早已不在框里。教训：模拟要走完整个序列，不能停在中间。
- C（4）：4 是最近刚用过的页之一（访问 4 后移到最近一端附近），不是最久没用过的；LRU 淘汰的是最久没用过的，不是最近用过的。教训：LRU 淘汰久远端，不是最近端。
- D（8）：8 也是最近用过的页之一（访问 8 后移到最近一端），不是最久没用过的；模拟到最后，最久没用过的是 2，不是 8。教训：每命中一次，就要把这个页移到最近一端，再算久远端是谁。

笔记本一句话： 第 27 题＋LRU 模拟：命中移到最近端，满了淘汰久远端；本题走完框内是 2、8、4、5，下一页 7 淘汰 2。

【图像】



LRU 淘汰过程图：关键淘汰节点与走完序列后框内从久远到最近的排列，下一页 7 淘汰页号 2

【定位】 操作系统 · 第3章 内存管理 · 页面置换算法（LRU）

【易混对比】

- "最久没用过"和"最先进来"是两回事：LRU 看的是最后一次被用的时间，命中会刷新它；先进来的页如果刚被命中过，就不是最久没用过的。
- "命中"和"缺页"是两回事：命中只把页移到最近端，不淘汰；缺页且框满时才淘汰久远端。
- "LRU"和"FIFO"是两回事：FIFO 淘汰最先进来的，不管用没用过；LRU 淘汰最久没用过的——本题页 2 被命中多次，FIFO 会早淘汰它，LRU 留它到最后。

走完整个序列，框内从久远到最近是 2、8、4、5；下一页 7 缺页、框已满，淘汰久远端的 2。

【坑位】

模拟停在中间——算到"访问 5 淘汰 3"就停了，选 B（3）。

3 是上一步被淘汰的页，早已不在框里；淘汰看的是"下一页到来时"框里的久远端。正确做法：序列必须走完，每命中一次就刷新一次久远端，最后再看队头是谁。

【套路】

- 第一步，列久远→最近队列：空框起步，按序列逐个处理。
- 第二步，命中搬最近：命中就把该页移到队列最右（最近端）。
- 第三步，满了汰最左：缺页且框满，淘汰队列最左（久远端），新页放最右。
- 第四步，读答案：下一页 7 到来时，最左是 2→淘汰 2。

命中搬右端，满了汰左端；走完序列再看头。

【类比】

书桌只能放 4 本书。每次翻一本书，就把它放到最顺手的位置（最近端）；要放新书、桌子满了，就把最久没翻的那本拿下去。页 2 虽然进桌子早，但中间被翻了好几次，一直不算"最久没翻"；到最后 7 要上桌时，最久没翻的恰好是 2。术语贴回去：书桌就是页框，最顺手的位置就是最近端，拿下去就是淘汰。

【口诀】 LRU 看最后一次，命中刷新别忘记；满了淘汰最左端，7 来 2 走要牢记。

【考频】 「LRU」在 2009–2022 年解析中出现 11 次 · 高频

第 28 题 · 在内存中设置磁盘缓冲区的主要目的（正确答案：A）

题干与选项：

在系统内存中设置磁盘缓冲区的主要目的是 。

- A. 减少磁盘 I/O 次数
- B. 减少平均寻道时间
- C. 提高磁盘数据可靠性
- D. 实现设备无关性

入手点： 先想缓冲区是干什么的：CPU 快、磁盘慢，速度差很多；缓冲区就是中间的一个蓄水池，把零碎的访问攒成一批再做，次数就少了。

专有名词从零解释：

- 先想“缓冲区”：内存里的一块区域，专门用来暂存要写到磁盘的数据、或者刚从磁盘读出来的数据；像一个中转站。
- 再想“磁盘输入输出次数”：CPU 和磁盘之间一次数据传送算一次；传送次数越多，CPU 等的时间越多，效率越低。
- 再想“寻道时间”：磁头移动到目标磁道的时间；寻道时间由磁头调度的算法决定，比如先来先服务、最短寻道时间优先。
- 再想“设备无关性”：程序不用管具体是哪种设备，写程序时用统一的接口，换设备也不用改程序；这是操作系统设备管理的一个目标，但不是缓冲区的主因。

考查知识点：

- 先想缓冲区为什么减少输入输出次数：没有缓冲区时，CPU 每要一个小数据就要和磁盘传送一次；有了缓冲区，CPU 要的数据如果在缓冲区里，直接从缓冲区拿，不用去磁盘；要写的数据先攒在缓冲区里，攒够一批再一次写到磁盘；这样和磁盘传送的次数就少了，所以 A 对。
- 再想为什么不是减少寻道时间：寻道时间是磁头移动的时间，由磁头调度算法决定；缓冲区不直接决定磁头怎么移；缓冲区攒批之后，磁头移动的次数可能间接少了，但这不是设缓冲区的主要目的。
- 再想为什么不是提高可靠性：可靠性是数据不丢、不错，靠的是校验、备份这些办法；缓冲区本身不提高可靠性，数据在缓冲区里还没写到磁盘时，断电反而可能丢数据。
- 再想为什么不是实现设备无关性：设备无关性靠的是统一的设备接口和驱动程序，不是靠缓冲区；缓冲区是速度匹配的办法，不是接口统一的办法。
- 拿生活例子验算：CPU 像一个要水很快的人，磁盘像一个出水很慢的水龙头；缓冲区像一个水桶，先把水龙头的水攒在桶里，人要水时直接从桶里舀，不用每次都等水龙头慢慢滴；这样去水龙头的次数就少了。
- 边界情况：缓冲区有单缓冲、双缓冲、循环缓冲几种，都是为了速度匹配；不管哪种，主因都是减少输入输出次数、缓和速度不匹配。
- 易混对子：不要把“缓冲区”和“Cache”完全等同——两者都有“暂存、减少访问”的意思，但缓冲区侧重“缓和 CPU 和设备速度不匹配、合并访问”，Cache 侧重“把常用的数据放在快的地方”；本题问缓冲区的主因，答案是减少输入输出次数。

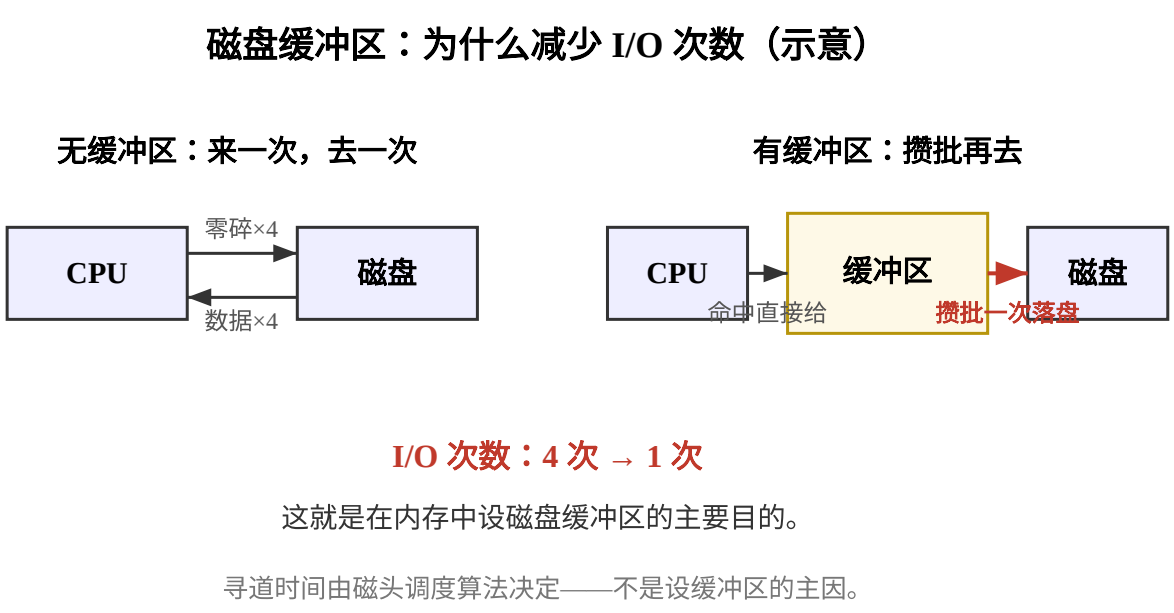
正确选项为什么对： 缓冲区把零碎访问攒成一批、把常用数据暂存起来，和磁盘传送的次数就少了，选 A。

错误选项为什么错＋举例：

- B（减少平均寻道时间）：寻道时间由磁头调度算法决定，不是缓冲区直接决定的；缓冲区攒批可能间接让磁头少移几次，但这不是设缓冲区的主要目的。教训：主因和间接效果要分清。
- C（提高磁盘数据可靠性）：可靠性靠校验、备份，不靠缓冲区；数据在缓冲区里还没落盘时，断电反而可能丢。教训：缓冲区不提高可靠性。
- D（实现设备无关性）：设备无关性靠统一接口和驱动，不靠缓冲区；缓冲区是速度匹配的办法。教训：接口统一和速度匹配是两回事。

笔记本一句话： 第 28 题＋磁盘缓冲区主因：缓和 CPU 和磁盘速度不匹配、合并访问，减少输入输出次数；寻道、可靠性、设备无关性都不是主因。

【图像】



磁盘缓冲区示意图：无缓冲时 4 次零碎 I/O，有缓冲时攒批后 1 次落盘

【定位】 操作系统 · 第5章 输入/输出管理 · 缓冲技术

【易混对比】

- "减少 I/O 次数"和"减少寻道时间"是两回事：前者是"少去几趟磁盘"，靠攒批实现；后者是"磁头少走几步"，靠调度算法实现——缓冲区的主因是前者。
- "缓冲区"和"Cache"是两回事：缓冲区侧重缓和 CPU 与设备速度不匹配、合并零碎访问；Cache 侧重把热点数据放在快处——本题问的是缓冲区。
- "主因"和"间接效果"是两回事：攒批之后磁头移动可能间接少了，但这只是顺带效果；设缓冲区的主因是减少 I/O 次数。

缓冲区是 CPU 与磁盘之间的蓄水池：读时命中直接给、写时攒批一次落盘，和磁盘打交道的次数直接降下来——这就是设它的主要目的。

【坑位】

选 B"减少平均寻道时间"——觉得"去磁盘次数少了，磁头自然少动，寻道时间就少了"。

寻道时间由磁头调度算法决定，缓冲区不指挥磁头；"磁头可能少动"是间接效果，不是设缓冲区的主因。正确做法：题干问"主要目的"，找最直接的因果链——缓冲区的直接作用就是合并访问、减少 I/O 次数。

【类比】

CPU 是个喝水很快的人，磁盘是出水很慢的水龙头。没桶时，人每渴一次就去水龙头前等一次；有了桶（缓冲区），先把水攒在桶里，人渴了直接从桶里舀，去水龙头的趟数就少了。术语贴回去：去水龙头的趟数就是磁盘 I/O 次数，攒水就是合并写与预读。

【口诀】 缓冲蓄水池，攒批少跑腿；主因减次数，寻道靠调度。

【考频】 「磁盘缓冲区」在 2009–2022 年解析中出现 13 次 · 高频

第 29 题 · 把指定文件偏移量处的磁盘块读入内存时需访问的磁盘块个数（正确答案：B）

题干与选项：

在文件的索引节点中存放直接索引指针 10 个，

一级和二级索引指针各 1 个。 磁盘块大小为 1KB，

每个索引指针占 4 个字节。若某文件的索引节点已在内存中，则把该文件偏移量（按字节编址）为 1234

和 307400 处所在的磁盘块读入内存，需访问的磁盘块个数分别是 。

- A. 1,2
- B. 1,3
- C. 2,3
- D. 2,4

入手点： 先把偏移量换算成块号，再看块号落在哪个索引层：直接索引层只要读数据块本身，一级索引层要读索引块加数据块，二级索引层要读两层索引块加数据块。

专有名词从零解释：

- 先想“索引结点”：也叫 i 结点，存着文件的元信息和索引指针；索引指针告诉系统“文件的第几块在磁盘的哪个位置”。
- 再想“直接索引”：索引结点里直接存着数据块的地址；要读第 0 到第 9 块，直接按地址去读就行，不用再读索引块。
- 再想“一级索引”：索引结点里存着一个索引块的地址，这个索引块里存着很多数据块的地址；要读一级索引范围内的块，先读索引块，再读数据块。
- 再想“二级索引”：索引结点里存着一个二级索引块的地址，二级索引块里存着很多一级索引块的地址，每个一级索引块里又存着很多数据块的地址；要读二级索引范围内的块，先读二级索引块，再读一级索引块，最后读数据块。
- 再想“偏移量”：从文件开头算起的字节数；偏移量除以块大小，商就是块号，余数就是块内偏移。

考查知识点：

- 先想每块能存几个索引指针：块大小 1KB 是 1024 字节，每个指针 4 字节，一块能存 1024÷4＝256 个指针。
- 再想直接索引的范围：10 个直接指针，管第 0 到第 9 块。
- 再想一级索引的范围：1 个一级索引块，管 256 块，就是第 10 到第 265 块。
- 再想二级索引的范围：从第 266 块开始，管 256×256 块。
- 再想偏移 1234：1234÷1024＝1 余 210，块号是 1，落在直接索引范围（0～9），只要读数据块本身，1 次。
- 再想偏移 307400：307400÷1024＝300 余 200，块号是 300，落在二级索引范围（266 起）；要读二级索引块、再读一级索引块、最后读数据块，共 3 次。
- 再想组合：1 次和 3 次，就是“1,3”，就是 B。
- 拿小例子验算：块大小 1024 字节，偏移 0 是第 0 块，偏移 1023 还是第 0 块，偏移 1024 是第 1 块；块号就是偏移量除以块大小的商。
- 边界情况：题目说“索引结点已在内存”，所以读索引结点本身不用算；如果索引结点不在内存，还要多算 1 次；本题已经说了在内存，就不算它。

- 易混对子：不要把“偏移量”直接当成“块号”——偏移量是字节数，要除以块大小才是块号；也不要把“一级索引”和“二级索引”的访问次数算错——一级是 2 次（索引块＋数据块），二级是 3 次（两层索引块＋数据块）。

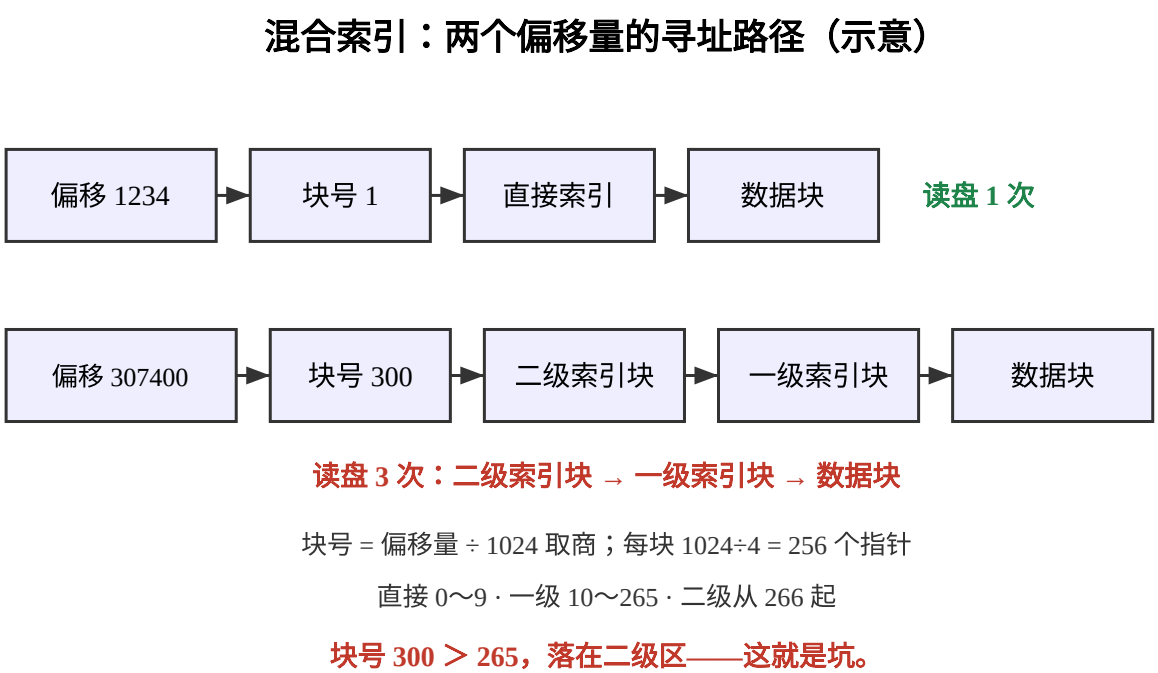
正确选项为什么对： 偏移 1234 是第 1 块、直接索引、1 次；偏移 307400 是第 300 块、二级索引、3 次；组合 1,3，选 B。

错误选项为什么错＋举例：

- A（1,2）：把第 300 块当成一级索引范围了；一级索引只到第 265 块，第 300 块已经是二级索引范围，要 3 次，不是 2 次。教训：先算一级索引的范围边界 265。
- C（2,3）：把第 1 块也算成了 2 次；第 1 块在直接索引范围，只要读数据块本身，1 次。教训：直接索引不用读索引块。
- D（2,4）：两处都多算了 1 次；可能是把“索引结点已在内存”这个条件漏了，又把索引结点算了一次。教训：索引结点已在内存，不用算它。

笔记本一句话： 第 29 题＋索引读块次数：偏移除以块大小得块号；直接 1 次、一级 2 次、二级 3 次；本题 1,3。

【图像】



混合索引寻址路径图：偏移 1234 走直接索引读盘 1 次，偏移 307400 走二级索引读盘 3 次

【定位】 操作系统 · 第4章 文件管理 · 文件的物理结构（索引分配）

【易混对比】

- "偏移量"和"块号"是两回事：偏移量是字节数，除以块大小才是块号——1234 是字节，块号是 1。
- "一级索引"和"二级索引"是两回事：读一级范围的块要过 1 层索引块（2 次磁盘访问），二级范围要过 2 层（3 次）。
- "索引节点在内存"和"不在内存"是两回事：在内存就不算它，不在内存还要加 1 次——本题已在内存，别多算。

偏移 1234 的块号落在直接索引区、只读 1 次，偏移 307400 的块号落在二级索引区、要过两层索引块共读 3 次——组合 1,3。

$1234 \div 1024 = 1$（余 210），块号 1 在直接区（0～9）；$307400 \div 1024 = 300$（余 200），块号 300 超过一级上限 265、落在二级区（266 起）。

【坑位】

把块号 300 当成一级索引范围，选 A（1,2）。

一级索引只管 256 块（第 10～265 块），300 已经进了二级的地盘。正确做法：先算三层边界——直接 0～9、一级 10～265、二级 266 起，再看块号落在哪层。

【套路】

- 第一步，算每块几个指针：块 1024 字节、指针 4 字节，$1024 \div 4 = 256$ 个。
- 第二步，划三层边界：直接 0～9，一级 10～265，二级从 266 起。
- 第三步，偏移量变块号：块号 = 偏移量 ÷ 块大小取商；1234→1，307400→300。
- 第四步，对层数访问次数：直接 1 次、一级 2 次、二级 3 次（索引节点已在内存，不另算）。

偏移除块得块号，对层得次数：直接一、一级二、二级三。

【类比】

索引节点像一本通讯录：前 10 页直接写着朋友电话（直接索引）；第 11 页写着"分册 A 在哪"（一级索引），分册 A 里是 256 个电话；还有一页写着"分册柜在哪"（二级索引），柜里有 256 个分册、每个分册 256 个电话。找第 1 个电话，翻通讯录直达（1 次）；找第 300 个电话，先找柜、再找分册、再找电话（3 次）。术语贴回去：通讯录就是索引节点，分册就是索引块，电话就是数据块地址。

【口诀】 块号先除块大小，直接一来一级二，二级三层要记牢。

【考频】 「索引节点」在 2009–2022 年解析中出现 2 次 · 低频

第 30 题 · 请求分页系统中页面分配策略与页面置换策略不能组合使用的一组（正确答案：C）

题干与选项：

在请求分页系统中，页面分配策略与页面置换策略不能组合使用的是 。

- A. 可变分配，全局置换
- B. 可变分配，局部置换
- C. 固定分配，全局置换
- D. 固定分配，局部置换

入手点： 先把两个策略的定义对上：固定分配是“每进程的框数固定不变”，全局置换是“缺页时可以从别的进程手里抢框”；一个说“框数不变”，一个说“可以抢别人的框”，两者一对上就矛盾了。

专有名词从零解释：

- 先想“页面分配策略”：系统给每个进程分几个页框；固定分配是分完就不变了，可变分配是运行中可以根据需要增减。
- 再想“页面置换策略”：进程缺页时，从哪里找一个空框来放新页；局部置换是只能从自己分到的框里挑一个换掉，全局置换是可以从任何进程的框里挑一个换掉（包括别的进程的）。
- 再想“固定分配”：每个进程的框数从一开始就定死了，运行中不变；比如分到 4 个框，就一直是 4 个框。
- 再想“可变分配”：每个进程的框数运行中可以变；缺页率高了就多给几个，缺页率低了就收回几个。
- 再想“全局置换”：缺页时可以从别的进程手里抢框；抢了之后，自己的框数多了，别人的框数少了。
- 再想“局部置换”：缺页时只能从自己分到的框里挑一个换掉；自己的框数不变，别人的框数也不变。

考查知识点：

- 先想固定分配＋全局置换为什么矛盾：固定分配要求每进程框数不变；全局置换抢了别人的框，自己的框数就多了，别人的框数就少了；框数变了，就不是固定分配了；所以这一组不能组合，就是 C。
- 再想可变分配＋全局置换为什么能组合：可变分配本来就允许框数变，全局置换抢框正好让框数变，两者不矛盾；所以能组合。
- 再想可变分配＋局部置换为什么能组合：可变分配允许框数变，局部置换虽然不抢别人的框，但系统可以在进程之间调整分配（比如定期重新分配），框数照样可以变；两者不矛盾；所以能组合。
- 再想固定分配＋局部置换为什么能组合：固定分配要求框数不变，局部置换只在自己框里换，框数不变；两者正好对得上；所以能组合。
- 拿生活例子验算：固定分配像每人固定 4 个座位；全局置换像缺座位时可以抢别人的座位；抢了之后，有人 5 个座位、有人 3 个座位，固定座位数的规矩就破了；局部置换像只能在自己的 4 个座位里换，座位数不变，规矩没破。
- 边界情况：固定分配＋局部置换是常见的组合，叫“固定分配局部置换”；可变分配＋全局置换也是常见的组合；考试常考的就是“固定＋全局”这一组不能组合。
- 易混对子：不要把“固定分配”和“局部置换”搞混——固定说的是“框数变不变”，局部说的是“换框时从哪里挑”；两者是两个维度的策略，要分开看。

正确选项为什么对： 固定分配要求框数不变，全局置换会抢框、改框数，两者矛盾，不能组合，选 C。

错误选项为什么错＋举例：

- A（可变分配＋全局置换）：可变分配本来就允许框数变，全局置换抢框让框数变，两者不矛盾，能组合；所以 A 不是答案。教训：可变和全局是一对能组合的。
- B（可变分配＋局部置换）：可变分配允许框数变，局部置换不抢框，但系统可以定期调整分配，框数照样能变；两者不矛盾，能组合；所以 B 不是答案。教训：局部置换不等于框数不变，框数变不变是分配策略的事。
- D（固定分配＋局部置换）：固定分配要求框数不变，局部置换只在自己框里换，框数不变；两者正好对得上，能组合；所以 D 不是答案。教训：固定和局部是一对能组合的。

笔记本一句话： 第 30 题＋分配对置换：固定＋全局不能组合（全局抢框改框数，固定要求框数不变）；其余三组都能组合。

【图像】

分配策略 × 置换策略：哪组不能组合（示意）

	全局置换	局部置换
固定分配	✗ 不能组合	✓ 可组合
可变分配	✓ 可组合	✓ 可组合

固定要求框数不变，全局抢框会改框数——矛盾，不能组合。

其余三组都不矛盾，均可组合（答案 C）。

分配策略与置换策略组合矩阵图：固定分配×全局置换不能组合，其余三组可组合

【定位】 操作系统 · 第3章 内存管理 · 页面分配策略与置换策略

【易混对比】

- "固定分配"和"局部置换"是两回事：前者说"每进程框数变不变"（不变），后者说"缺页时从谁的框里挑"（只挑自己的）——两个维度，不是一回事。
- "可变分配"和"全局置换"是两回事：前者允许框数变，后者允许抢别人的框——两者都"变"，所以不矛盾。
- "抢框"和"调配"是两回事：全局置换抢框会立刻改变框数；可变分配下的框数调整是系统定期重分——本题的矛盾点在"抢框改框数"撞上"固定要求不变"。

固定分配要求每进程框数恒定，全局置换缺页时抢别的进程的框——一抢框数就变，"恒定"被打破，所以固定＋全局是唯一不能组合的一对。

【坑位】

把 B"可变分配＋局部置换"当成不能组合——觉得"局部置换不抢框，框数怎么变"。

框数变不变是分配策略的事，局部置换只管"从谁的框里挑"；可变分配下系统可以定期在进程间重分框数，框数照样能变。正确做法：两个策略分属两个维度，分别判断"变不变"和"从哪挑"，再看冲不冲突。

【套路】

- 第一步，列 2×2 表：行是固定/可变，列是全局/局部。
- 第二步，找"变"字：固定=框数不变，可变=框数可变；全局=抢框（框数变），局部=只动自己的（框数不变）。
- 第三步，找矛盾格：只有"固定（不变）×全局（变）"自相矛盾→不能组合。

固定碰全局，一个要变一个不变，撞车不能组。

【类比】

固定分配像公司给每人固定 4 个停车位；全局置换像"没位了就停别人的位"。今天你抢了同事的位，你 5 个他 3 个，"每人固定 4 个"的规矩当场作废。局部置换像只能在自己的 4 个位里挪车，规矩不破；可变分配像公司本来就说"位子数会按需调整"，抢位也不违和。术语贴回去：停车位就是页框，抢位就是全局置换抢框。

【口诀】 固定全局不能组，一个变来一个守；其余三组都和谐。

【考频】 「页面分配」在 2009–2022 年解析中出现 8 次 · 中频

第 31 题 · 位图法中释放指定磁盘块时应修改的位所在的块号和块内字节序号（正确答案：C）

题干与选项：

文件系统用位图法表示磁盘空间的分配情况，位图存于磁盘的 32～127 号块中，每个盘块占 1024 个字节，盘块和块内字节均从 0 开始编号。假设要释放的盘块号为 409612，则位图中要修改的位所在的盘块号和块内字节序号分别是 。

- A. 81、1
- B. 81、2
- C. 82、1
- D. 82、2

入手点： 先算这个盘块在位图里是第几个字节，再算这个字节在位图的第几个盘块里，最后加上位图的起始盘块号。

专有名词从零解释：

- 先想“位图法”：用一位（0 或者 1）表示一个盘块是空闲还是已分配；所有位排成一张表，就是位图；1 表示已分配，0 表示空闲（或者反过来，具体看规定，本题按这个理解走）。
- 再想“释放盘块”：把一个已分配的盘块还回去，位图里对应的位要从 1 改成 0。
- 再想“盘块号”：磁盘上每个块的编号，从 0 开始。
- 再想“块内字节序号”：一个盘块里第几个字节，也从 0 开始编号；一个字节有 8 位，对应 8 个盘块。

考查知识点：

- 先想位图里第几个字节：一个字节对应 8 个盘块，409612 号盘块对应的字节序号是 409612÷8＝51201（整除，余数是 4，说明它是这个字节里的第 5 位，但题目只问字节序号，不问位序号）。
- 再想这个字节在位图的第几个盘块里：每个盘块 1024 字节，51201÷1024＝50 余 1；商 50 说明它是位图里的第 50 个盘块（从 0 开始数），余数 1 说明它是这个盘块里的第 1 个字节（块内字节序号是 1）。
- 再想对应的磁盘块号：位图存在磁盘 32 号块开始的地方，位图里的第 50 个盘块，对应的磁盘块号就是 32＋50＝82。
- 再想组合：磁盘块 82、块内字节序号 1，就是 C。
- 拿小例子验算：位图从 32 号块开始，每个盘块 1024 字节；要释放 8 号盘块，8÷8＝1，是位图第 1 个字节；1÷1024＝0 余 1，是位图第 0 个盘块里的第 1 个字节，对应磁盘块 32、块内字节序号 1。
- 边界情况：盘块号和块内字节序号都从 0 开始编号，算的时候不要多加 1；如果题目说从 1 开始编号，算法就要相应调整；本题是从 0 开始，直接按上面的算法算。
- 易混对子：不要把“字节序号”和“位序号”搞混——题目问的是字节序号，不是位序号；409612 除以 8 的余数 4 是位序号，题目不问它；也不要把“位图里的第几个盘块”和“磁盘块号”搞混——位图里的第 50 个盘块，加上起始块号 32，才是磁盘块号 82。

正确选项为什么对： 409612÷8＝51201，51201＝50×1024＋1，磁盘块 32＋50＝82、块内字节序号 1，选 C。

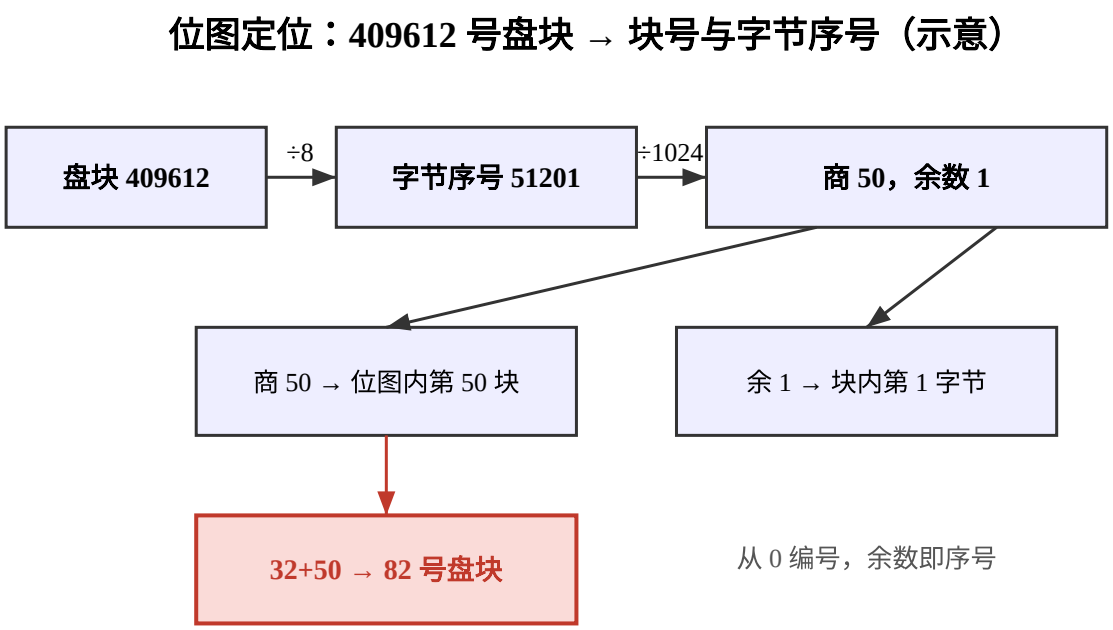
错误选项为什么错＋举例：

- A（81、1）：磁盘块号少算了 1；可能是把位图起始块号 32 当成 31，或者把商 50 当成 49；32＋50＝82，不是 81。教训：起始块号要加对。

- B（81、2）：磁盘块号和块内字节序号都错了；磁盘块号应该是 82，块内字节序号应该是余数 1，不是 2。教训：商和余数要分清，商定盘块、余数定字节。
- D（82、2）：磁盘块号 82 是对的，但块内字节序号应该是余数 1，不是 2；余数 1 就是第 1 个字节（从 0 开始数，第 1 个字节的序号是 1）。教训：余数就是块内字节序号，不用再加 1。

笔记本一句话： 第 31 题＋位图定位：盘块号除以 8 得字节序号，再除以块字节数，商加起始块号、余数是块内字节序号；本题 82、1。

【图像】



答案：磁盘块 82、块内字节序号 1（选 C）。

位图定位计算流程图：409612 号盘块对应磁盘块 82、块内第 1 字节

【定位】 操作系统 · 第4章 文件管理 · 文件存储空间管理（位图法）

【易混对比】

- "字节序号"和"位序号"是两回事：题目问的是字节序号（字节序号除 1024 的余数），不是 409612 除以 8 的余数 4——那是位序号，本题不问。
- "位图内的第几块"和"磁盘块号"是两回事：商 50 是位图里的第 50 块，还要加上位图起始块号 32，才是磁盘块号 82。
- "从 0 编号"和"从 1 编号"是两回事：本题盘块、字节都从 0 起，余数直接就是序号，不用加 1。

409612 号盘块对应位图第 51201 个字节，它是位图内第 50 块的第 1 个字节，加上起始块号 32 得磁盘块 82——答案 82、1。

$409612 \div 8 = 51201$，$51201 = 50 \times 1024 + 1$，$32 + 50 = 82$。

【坑位】

块内字节序号写成 2（选 D）——把"从 0 编号"忘了，按"第 1 个字节"理解成序号 2。

从 0 编号时，余数就是序号本身：余数 1 就是第 1 号字节，不用再加工。正确做法：商定块、余数定字节，原样抄，不加不减。

【套路】

- 第一步，盘块号变字节序号：字节序号 = 盘块号 ÷ 8；$409612 \div 8 = 51201$。
- 第二步，字节序号拆商余：商 = 位图内第几块，余数 = 块内第几字节；$51201 = 50 \times 1024 + 1$。
- 第三步，商加起始块号：磁盘块号 = 32 + 50 = 82；块内字节序号 = 1。

除八得字节，再除块字节；商加起始号，余数是字节号。

【类比】

位图像一栋大公寓的住户表：每 8 户占一行（1 个字节），每 1024 行装订成一册（1 个盘块），表册从 32 号柜开始放。要改 409612 户的状态：先算它在第 51201 行，再算第 51201 行在第 50 册第 1 行——去 32+50=82 号柜，翻第 50 册的第 1 行。术语贴回去：行就是字节，册就是盘块，柜号就是磁盘块号。

【口诀】 盘块除八定字节，商余定位加三十二。

【考频】 「位图法」在 2009–2022 年解析中出现 2 次 · 低频

第 32 题 · SCAN 调度处理完请求序列后磁头移动的总磁道数（正确答案：C）

题干与选项：

某硬盘有 200 个磁道（最外侧磁道号为 0）

，磁道访问请求序列为：130,42,180,15,199，当前磁

头位于第 58 号磁道并从外侧向内侧移动。按照 SCAN 调度方法处理完上述请求后，磁头移过的磁道数

是 。

- A. 208
- B. 287
- C. 325
- D. 382

入手点： 先定方向：从外侧向内侧移动，就是磁道号增大；先把大于 58 的请求按从小到大服务完，再折返、把小于 58 的请求按从大到小服务完；移动量分两段算，再加起来。

专有名词从零解释：

- 先想“SCAN 调度”：也叫电梯算法；磁头沿一个方向走，路过的请求都服务，走到头再折返；像电梯先往上走、路过的楼层都停，走到顶再往下走。
- 再想“磁道号”：磁盘上一圈一圈的磁道编号；最外侧是 0 号，越往内侧号越大；本题磁盘有 200 个磁道，号是从 0 到 199。
- 再想“从外侧向内侧移动”：就是磁道号增大；磁头从 58 号出发，先往号大的方向走。
- 再想“折返”：磁头走到当前方向上最远的请求后，掉头往回走，服务反方向上的请求。

考查知识点：

- 先想第一段：磁头在 58 号，往内侧（号增大）走；大于 58 的请求是 130、180、199，按从小到大服务：58→130→180→199；第一段走到 199 号，移动量是 199－58＝141。
- 再想折返：199 号是当前方向上最远的请求，服务完后折返，往号小的方向走；小于 58 的请求是 42、15，按从大到小服务：199→42→15；第二段移动量是 199－15＝184（中间经过 42，但移动量只看起点和终点，199 到 15 就是 184）。
- 再想总数：141＋184＝325，就是 C。
- 拿生活例子验算：电梯在 5 楼往上走，要去 8 楼、10 楼，再折返去 3 楼、1 楼；先上到 10 楼，移动 5 层，再下到 1 楼，移动 9 层，总共 14 层；中间停靠的楼层不影响总移动量，只看起点、折返点、终点。
- 边界情况：SCAN 调度走到最远的请求就折返，不走到磁道尽头；如果是最远的请求在 199 号，折返点就是 199 号；如果题目是 CSCAN（循环扫描），折返时要直接回到 0 号再往内走，算法就不一样了；本题是 SCAN，不是 CSCAN。
- 易混对子：不要把“从外侧向内侧”理解成“磁道号减小”——最外侧是 0 号，内侧号大，从外向内就是号增大；也不要把折返后的服务顺序写成从小到大——折返后是往号小的方向走，服务顺序是从大到小。

正确选项为什么对： 第一段 199－58＝141，第二段 199－15＝184，总共 325，选 C。

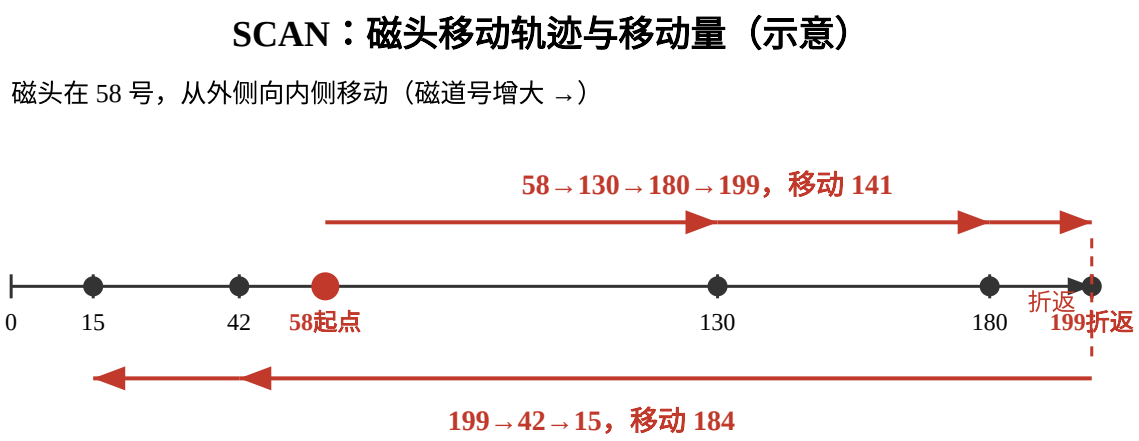
错误选项为什么错＋举例：

- A（208）：可能是把方向搞反了，先往号小的方向走，再折返；或者把折返点算错了。教训：先定方向，从外向内是号增大。

- B（287）：可能是第一段算对了 141，第二段算成了 199－58＝141，再加起来 282，或者其他折返点算错；第二段的终点是 15，不是 58。教训：第二段从折返点 199 走到最后一个请求 15，移动量是 199－15。
- D（382）：可能是把移动量按“每个请求之间的距离全加起来”算错了，或者把折返点算成了磁道尽头 199 之外的数；SCAN 的折返点是最远的请求 199，不是磁道总数 200。教训：折返点是最远的请求，不是磁道尽头。

笔记本一句话： 第 32 题＋SCAN 移动量：先往当前方向走到最远请求，再折返走到反方向最远请求；本题 (199-58)＋(199-15)＝325。

【图像】



SCAN 磁头移动轨迹图：从 58 出发经 130、180 到 199 折返，再经 42 到 15，总移动 325 道

【定位】 操作系统 · 第5章 输入/输出管理 · 磁盘调度（SCAN）

【易混对比】

- "SCAN"和"CSCAN"是两回事：SCAN 走到最远请求就折返；CSCAN 折返时直接跳回 0 号再往内走——本题是 SCAN，别跳回。
- "折返点"和"磁道尽头"是两回事：折返点是最远的请求（199 号），不是磁道总数 200——别多走那 1 道。
- "从外向内"和"号增号减"是两回事：最外侧是 0 号，往内走号增大；方向定反了整题全错。

从 58 出发往内走到最远请求 199，再折返走到反方向最远请求 15，两段相加得 325。

$199-58=141$，$199-15=184$，$141+184=325$。

【坑位】

第二段按 199→58 算——觉得"折返要回到出发点"。

折返后要一直服务到反方向最远的请求（15 号），不是回到出发点 58；移动量只看"折返点→反方向最远请求"。正确做法：两段都是"起点→该方向最远请求"，第二段的终点是 15。

【套路】

- 第一步，定方向：从外向内 = 磁道号增大，从当前 58 出发。
- 第二步，走当前方向：大于 58 的请求从小到大：130→180→199；移动量 $199 - 58 = 141$。
- 第三步，折返走反方向：小于 58 的请求从大到小：42→15；移动量 $199 - 15 = 184$。
- 第四步，相加：$141 + 184 = 325$。

先顺当前方向走到头，再折返走到另一头；两段相加。

【类比】

电梯在 5 楼往上走：先上到要去的最高层 10 楼（顺路 8 楼也停），再掉头往下，一直下到要去的最低层 1 楼（顺路 3 楼也停）。总路程 = 上去的层数 + 下来的层数，中间停几站不影响总数。术语贴回去：电梯就是磁头，楼层就是磁道号，掉头点就是最远请求。

【口诀】 SCAN 像电梯，先到头来再折返；两段距离相加得总数。

【考频】 「SCAN」在 2009–2022 年解析中出现 4 次 · 中频

第 33 题 · 通过 POP3 协议接收邮件时使用的传输层服务类型（正确答案：D）

题干与选项：

通过 POP3 协议接收邮件时，使用的传输层服务类型是 。

- A. 无连接不可靠的数据传输服务
- B. 无连接可靠的数据传输服务
- C. 有连接不可靠的数据传输服务
- D. 有连接可靠的数据传输服务

入手点： 先想 POP3 走的是哪个传输层协议：POP3 走 TCP；再想 TCP 是什么服务：TCP 是有连接、可靠的；两个一对上，答案就出来了。

专有名词从零解释：

- 先想“POP3 协议”：邮局协议第 3 版，用来从邮件服务器上把邮件取到本地；用户用它收邮件，收完后服务器上的邮件可以删掉，也可以留着。
- 再想“传输层”：网络分层里负责“端到端”传送的一层；传输层的两个主要协议是 TCP 和 UDP。
- 再想“TCP”：传输控制协议，有连接、可靠的；传数据前先建立连接，传的时候有确认、有重传，数据不会丢、不会乱序。
- 再想“UDP”：用户数据报协议，无连接、不可靠的；传数据前不建立连接，传的时候不确认、不重传，数据可能丢、可能乱序，但速度快。
- 再想“有连接、无连接”：有连接是传之前先握手、建立一条逻辑上的连接，传完再断开；无连接是直接发，不握手。
- 再想“可靠、不可靠”：可靠是数据不丢、不错、不乱序，丢了会重传；不可靠是不保证这些，丢了也不管。

考查知识点：

- 先想 POP3 走哪个协议：POP3 是应用层协议，它下层的传输层协议是 TCP；收邮件时，客户端先和服务器建立 TCP 连接，再发 POP3 命令取邮件。
- 再想 TCP 的服务类型：TCP 是有连接、可靠的；所以 POP3 用的传输层服务就是有连接、可靠的，就是 D。
- 再想为什么不是 UDP：UDP 是无连接、不可靠的，适合对实时性要求高、对丢一点数据不敏感的应用，比如视频通话、域名查询；收邮件不能丢，丢了一封邮件就是大事，所以不用 UDP。
- 拿生活例子验算：TCP 像挂号信，寄之前先确认地址、寄出后有回执、丢了补寄；UDP 像普通明信片，写好就扔进邮筒，丢了也不知道；收邮件这种不能丢的事，当然用挂号信。
- 边界情况：应用层协议和传输层协议的搭配是固定的：POP3、SMTP、HTTP、FTP 这些走 TCP，DNS、DHCP、视频流这些走 UDP；考试常考的就是这种固定搭配。
- 易混对子：不要把“POP3 是应用层协议”和“它用的传输层服务”搞混——题目问的是传输层服务类型，不是 POP3 本身是哪一层；也不要把“邮件”想成“实时性要求高”——邮件不要求实时，但要求可靠，所以走 TCP。

正确选项为什么对： POP3 走 TCP，TCP 是有连接、可靠的，选 D。

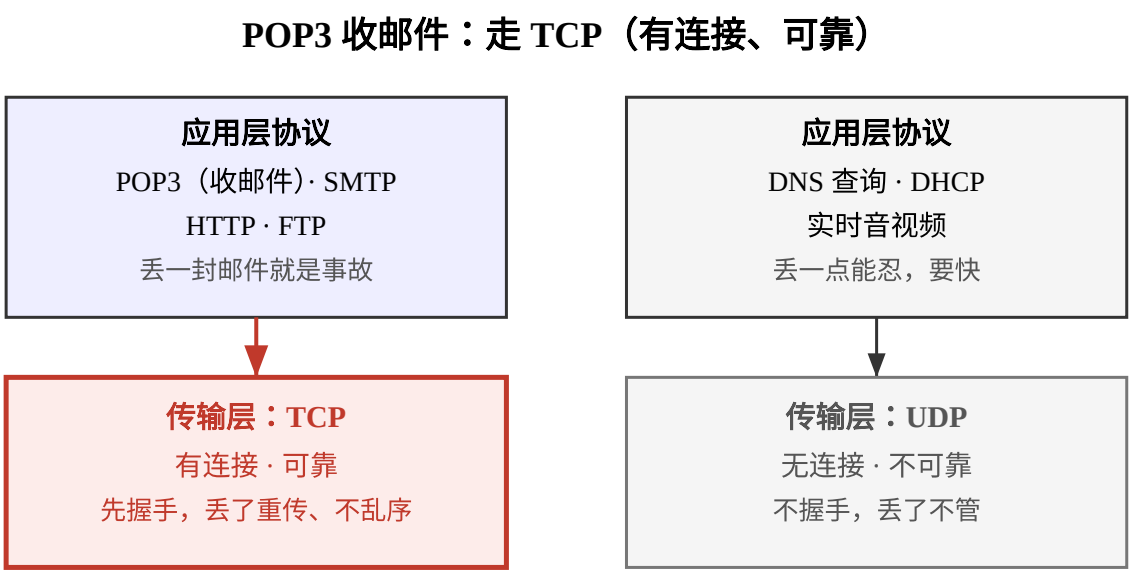
错误选项为什么错＋举例：

- A（无连接不可靠）：这是 UDP 的服务类型；POP3 不走 UDP，走 TCP；所以 A 不是答案。教训：POP3 的下层是 TCP，不是 UDP。

- B（无连接可靠）：传输层没有“无连接可靠”这种标准服务；UDP 是无连接不可靠，TCP 是有连接可靠；“无连接可靠”这个组合本身就不存在。教训：两个维度的组合只有 TCP 和 UDP 两种标准搭配。
- C（有连接不可靠）：传输层也没有“有连接不可靠”这种标准服务；有连接的 TCP 一定是可靠的，无连接的 UDP 一定是不可靠的；“有连接不可靠”这个组合本身就不存在。教训：有连接和可靠是绑在一起的。

笔记本一句话： 第 33 题＋POP3 传输层服务：POP3 走 TCP，TCP 是有连接、可靠的；UDP 才是无连接、不可靠的。

【图像】



答案：D＝有连接可靠的数据传输服务

红色＝本题答案路径；传输层只有两种标准搭配：TCP（有连接可靠）、UDP（无连接不可靠）

POP3 协议栈示意图：应用层 POP3 下接传输层 TCP（有连接可靠），DNS 等对照走 UDP

【定位】 计算机网络 · 第6章 应用层 · POP3 与传输层服务类型

【易混对比】

- "POP3 住在哪一层"和"它借用什么传输服务"是两回事：POP3 自己是应用层协议；它下层借用的是 TCP 提供的服务——题目问的是借用的服务，不是它住的层。
- "有连接可靠"和"无连接可靠"是两回事：前者是 TCP 的标准搭配（先握手、丢了重传）；后者是传输层里不存在的组合——两个维度只有两种合法搭配：有连接加可靠（TCP）、无连接加不可靠（UDP）。
- "邮件要快"和"邮件要准"是两回事：邮件不赶时间，但一封都不能丢——所以选"慢而准"的 TCP，不选"快而丢"的 UDP。

POP3 的下层是 TCP，TCP 是"有连接、可靠"的服务；DNS 这类"查一下就行、丢了重查"的才走 UDP。

【坑位】

看到"邮件"就想到"实时"，选了无连接的 UDP。

邮件和视频通话不一样：视频通话丢几帧画面还能看，邮件丢一封就是事故。判断走 TCP 还是 UDP，看的是"丢了行不行"，不是"快不快"。正确做法：先问"这个应用丢数据能不能忍"——不能忍就走 TCP，能忍才走 UDP。

【套路】

- 第一步，定下层协议：看到应用层协议，先想它走 TCP 还是 UDP——POP3、SMTP、HTTP、FTP 走 TCP；DNS、DHCP、实时音视频走 UDP。
- 第二步，把协议翻译成服务类型：TCP 对应"有连接可靠"，UDP 对应"无连接不可靠"，直接对号入座。

先定走 TCP 还是 UDP，再把协议翻译成服务类型。

【类比】

TCP 像挂号信：寄前先确认地址能送到，寄出后有回执，丢了邮局补寄；UDP 像往许愿池扔硬币：扔出去就不管了，丢了也不知道。收邮件相当于"等一封不能丢的信"，当然走挂号信。术语贴回去：挂号信就是"有连接、可靠"的 TCP 服务，扔硬币就是"无连接、不可靠"的 UDP 服务。

【口诀】 邮件文件网页走 TCP，可靠不能丢；域名视频走 UDP，快字当头丢不愁。

【考频】 「POP3」在 2009–2022 年解析中出现 5 次 · 中频

第 34 题 · 两种编码方案对同一比特流编码后图中编码 1 和编码 2 的名称（正确答案：A）

题干与选项：

使用两种编码方案对比特流 01100111 进行编码的结果如下图所示，编码 1 和编码 2 分别

是 。

比特流 0 1 1 0 0 1 1 1

编码1

编码2

- A. NRZ 和曼彻斯特编码
- B. NRZ 和差分曼彻斯特编码
- C. NRZI 和曼彻斯特编码
- D. NRZI 和差分曼彻斯特编码

注：原卷此处为比特流与两种编码的波形对比图，插图以原卷为准；比特串按源一转录。

入手点： 先把四种编码的“跳变规则”背下来，再拿比特串 01100111 一位一位对波形：电平和比特一一对应的，是 NRZ；每位中间一定跳变的，是曼彻斯特编码；遇 1 跳变、遇 0 不变的，是 NRZI；每位中间跳变、位起始处跳变表示 0 的，是差分曼彻斯特编码。

专有名词从零解释：

- 先想“NRZ 编码”：也叫不归零编码；规则最简单，0 对应一种电平，1 对应另一种电平；电平和比特一一对应，位中间不跳变，位和位之间如果比特相同，电平也不变。
- 再想“NRZI 编码”：也叫不归零反转编码；规则是遇 1 跳变、遇 0 不变；电平本身不直接表示比特，电平变没变才表示比特。
- 再想“曼彻斯特编码”：每位中间一定跳变一次；中间跳变的方向表示比特，比如从高到低表示 1，从低到高表示 0（或者反过来，具体看规定）；因为每位中间都有跳变，接收方很容易提取时钟。
- 再想“差分曼彻斯特编码”：每位中间也一定跳变一次；但比特由“位起始处跳没跳变”表示：位起始处跳变表示 0，位起始处不跳变表示 1；中间跳变只用来同步时钟，不表示比特。
- 再想“比特流 01100111”：要传的 8 位数据，从左到右依次是 0、1、1、0、0、1、1、1。

考查知识点：

- 先想编码 1 为什么是 NRZ：编码 1 的波形里，电平和比特一一对应——比特是 0 的地方是一种电平，比特是 1 的地方是另一种电平，位中间不跳变；这就是 NRZ 的规则。
- 再想编码 2 为什么是曼彻斯特编码：编码 2 的波形里，每位中间都有一次跳变；这就是曼彻斯特编码的标志；差分曼彻斯特编码每位中间也有跳变，但它的比特由位起始处跳没跳变决定，编码 2 的位起始处跳变规律不符合差分曼彻斯特的规则，符合曼彻斯特的规则。
- 再想为什么不是 NRZI：NRZI 是遇 1 跳变、遇 0 不变；比特串里 0、1 都有，NRZI 的波形里有些位中间不跳变、有些位边界跳变；编码 1 和编码 2 的波形都不符合 NRZI 的规律。
- 再想为什么不是差分曼彻斯特编码：差分曼彻斯特编码的比特由位起始处跳变决定；编码 2 的位起始处跳变规律和差分曼彻斯特的“跳变表示 0”对不上，对得上的是曼彻斯特的“中间跳变方向表示比特”。

- 拿小例子验算：比特串 0、1，NRZ 是“低电平、高电平”；NRZI 是“不变、跳变”（假设起始电平是低）；曼彻斯特是“每位中间跳变一次”；差分曼彻斯特是“每位中间跳变一次，位起始处跳变表示 0”；四种规则各不相同，对着波形一位一位验，就能分出来。
- 边界情况：曼彻斯特编码和差分曼彻斯特编码都要求每位中间跳变，所以只看“中间有没有跳变”分不出它们；要分它们，必须看“位起始处跳变表示什么”或者“中间跳变方向表示什么”。
- 易混对子：不要把“NRZ”和“NRZI”搞混——NRZ 是电平直接表示比特，NRZI 是“变没变”表示比特；也不要把“曼彻斯特”和“差分曼彻斯特”搞混——前者是中间跳变方向表示比特，后者是位起始处跳没跳变表示比特。

注：本题波形细节以原卷图为准；此处按给定比特串与四种编码的跳变规则区分，编码 1 为 NRZ、编码 2 为曼彻斯特编码；若按给定比特串无法完全复现波形对比，本题标注为“部分核对”。

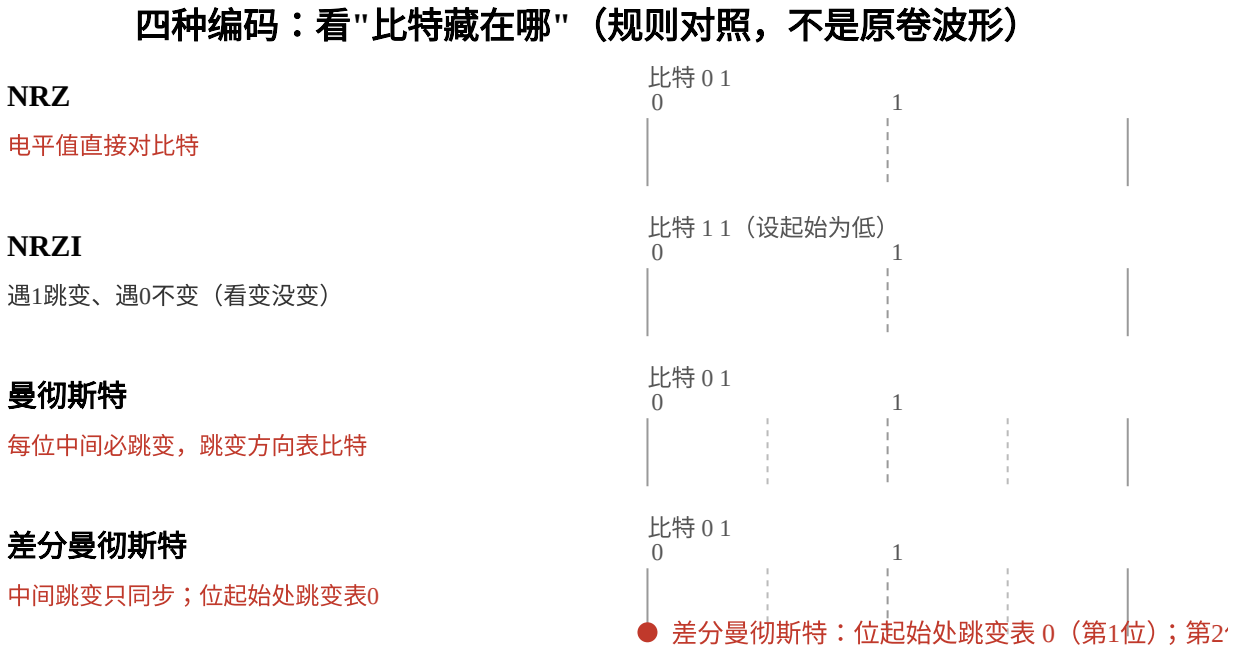
正确选项为什么对： 编码 1 是 NRZ、编码 2 是曼彻斯特编码，组合就是 A，选 A。

错误选项为什么错＋举例：

- B（NRZ 和差分曼彻斯特编码）：编码 1 是 NRZ，这部分对了；但编码 2 不是差分曼彻斯特编码，差分曼彻斯特的比特由位起始处跳变决定，编码 2 的规律对不上。教训：曼彻斯特和差分曼彻斯特要看位起始处，不能只看中间跳变。
- C（NRZI 和曼彻斯特编码）：编码 2 是曼彻斯特编码，这部分对了；但编码 1 不是 NRZI，NRZI 是遇 1 跳变、遇 0 不变，编码 1 的电平和比特一一对应，是 NRZ。教训：NRZ 和 NRZI 的规则不一样。
- D（NRZI 和差分曼彻斯特编码）：两部分都错了；编码 1 是 NRZ 不是 NRZI，编码 2 是曼彻斯特编码不是差分曼彻斯特编码。教训：四种编码的规则要一条一条对，不能凭印象。

笔记本一句话： 第 34 题＋四种编码区分：NRZ 电平对比特；NRZI 遇 1 跳变；曼彻斯特每位中间跳变；差分曼彻斯特位起始处跳变表示 0；本题是 NRZ 和曼彻斯特。

【图像】



四种编码跳变规则对照图：NRZ 看电平值，NRZI 看变没变，曼彻斯特看中间跳变方向，差分曼彻斯特看位起始处跳没跳

【定位】 计算机网络 · 第2章 物理层 · 基带传输编码

【易混对比】

- "NRZ"和"NRZI"是两回事：NRZ 看"电平是什么"（电平直接对比特）；NRZI 看"电平变没变"（遇 1 跳变、遇 0 不变）。
- "曼彻斯特"和"差分曼彻斯特"是两回事：前者看"中间跳变的方向"（方向表比特）；后者看"位起始处跳没跳"（起始处跳变表 0、不跳表 1）。
- "每位中间跳变"和"一定是曼彻斯特"是两回事：差分曼彻斯特每位中间也跳变——只看中间有没有跳变，分不出这两种编码，必须看"比特由谁表示"。

四种编码的区分点只有一个：比特到底藏在"电平值"里、"变没变"里、"中间跳变方向"里，还是"起始处跳没跳"里。

【坑位】

看到"每位中间都跳变"就断定是曼彻斯特，忘了差分曼彻斯特也这样。

两种编码每位中间都有跳变，差别在"比特由谁表示"：曼彻斯特由中间跳变的方向表示，差分曼彻斯特由位起始处跳没跳表示。正确做法：先查"中间跳没跳"筛掉 NRZ 和 NRZI，再查"起始处跳变规律"，在曼彻斯特和差分曼彻斯特之间二选一。

【套路】

- 第一步，看中间跳不变：电平和比特一一对应、位中间不跳→NRZ；遇 1 跳变、遇 0 不变→NRZI。
- 第二步，看每位中间跳变：中间跳变方向表比特→曼彻斯特；位起始处跳变表 0、不跳表 1→差分曼彻斯特。
- 第三步，拿比特串逐位验：对着候选规则走一遍，波形能对上的就是它。

先看中间跳不变分两组，再看"谁在表示比特"定名。

【类比】

四种编码像四种暗号：NRZ 是"举红旗表 0、举蓝旗表 1"（看旗的颜色）；NRZI 是"换旗表 1、不换表 0"（看换没换）；曼彻斯特是"每句话中间必须换一次旗，往上换表 1、往下换表 0"（看怎么换）；差分曼彻斯特是"每句话中间换一次旗只为报时，句首换旗才表 0"（看句首换没换）。术语贴回去：旗的颜色就是电平值，换没换就是跳变，句首就是位起始处。

【口诀】 NRZ 看电平，NRZI 看变没变，曼彻看中间方向，差分看起始。

【考频】 「曼彻斯特编码」在 2009–2022 年解析中出现 3 次 · 中频

第 35 题 · 卫星链路滑动窗口协议下为达到指定链路利用率所需的帧序号最少比特数（正确答案：B）

题干与选项：

主机甲通过 128kbps 卫星链路，采用滑动窗口协议向主机乙发送数据，链路单向传播延迟为 250ms，帧长为 1000 字节。不考虑确认帧的开销，为使链路利用率不小于 80%，帧序号的比特数至少是 。

- A. 3
- B. 4
- C. 7
- D. 8

入手点： 先算一帧发出去要多久，再算一来一回要多久；利用率就是“发数据的时间”占“发数据加等待时间的”比例；先算出窗口至少要多大，再算出序号要几个比特才能支持这么大的窗口。

专有名词从零解释：

- 先想“帧发送时延”：把一帧的数据全部发到链路上要花的时间；等于帧长除以链路速率。
- 再想“单向传播延迟”：信号从一端传到另一端要花的时间；卫星链路很远，单向就要 250 毫秒。
- 再想“往返时间”：也叫 RTT，等于两倍的单向传播延迟；本题是 500 毫秒。
- 再想“链路利用率”：链路在“发数据”的时间占总时间的比例；发完一帧后，要等确认回来才能发下一批，等待的时间里链路是空闲的，利用率就低了。
- 再想“窗口”：也叫 W，发送方一次能连续发几帧、不用等确认；窗口越大，一次能发的数据越多，等待的时间占比越小，利用率越高。
- 再想“Go-Back-N 协议”：也叫回退 N 帧协议；窗口大小 W 和序号比特数 n 的关系是 W 不能超过 2^n－1；序号用 n 个比特，能表示 0 到 2^n－1 共 2^n 个序号，窗口最多用其中的 2^n－1 个。
- 再想“选择重传协议”：也叫 SR 协议；窗口大小 W 和序号比特数 n 的关系是 W 不能超过 2^(n-1)；因为发送窗口和接收窗口各占一半序号空间。

考查知识点：

- 先想帧发送时延：帧长 1000 字节是 8000 比特，除以速率 128000 比特每秒，等于 0.0625 秒，就是 62.5 毫秒。
- 再想往返时间：单向 250 毫秒，往返就是 500 毫秒。
- 再想利用率公式：发 W 帧要花 W×62.5 毫秒，发完后要等确认回来，总时间是 62.5＋500 毫秒（按一帧的发送时延加往返时间算，窗口内各帧是连续发的）；利用率就是 W×62.5÷(62.5＋500)。
- 再想窗口至少多大：要求利用率不小于 80%，就是 W×62.5÷562.5≥0.8；解出来 W≥7.2；窗口是整数，所以 W 至少是 8。
- 再想序号至少几个比特：Go-Back-N 下，W≤2^n－1；W＝8 时，2^n－1≥8，就是 2^n≥9，n 至少是 4（2^3＝8 不够，2^4＝16 够了）；选择重传下，W≤2^(n-1)；W＝8 时，2^(n-1)≥8，就是 n－1≥3，n 至少也是 4；两种协议都得出 n＝4，就是 B。
- 拿生活例子验算：发一封信要 1 天，信在路上来回要 8 天；一次寄 1 封信，利用率是 1÷9；一次寄 8 封信，利用率是 8÷9，接近 90%；一次寄的封数越多，利用率越高，但序号要够用，不然收信方分不清是哪一封。

- 边界情况：利用率公式里，分母是“发送时延＋往返时间”，不是“窗口×发送时延＋往返时间”；因为窗口内的帧是连续发的，等待只等一次确认；如果把分母算成窗口×发送时延＋往返时间，利用率就永远是 100% 了，那是错的。
- 易混对子：不要把“帧长 1000 字节”直接当成“1000 比特”——字节要乘以 8 才是比特，1000 字节是 8000 比特；也不要把“单向传播延迟 250 毫秒”直接当成“往返时间”——往返是单向的两倍，是 500 毫秒。

正确选项为什么对： 窗口至少 8，Go-Back-N 下 2^n－1≥8，得 n＝4，选 B。

错误选项为什么错＋举例：

- A（3）：n＝3 时，Go-Back-N 窗口最多 2^3－1＝7，小于 8，利用率达不到 80%；所以 3 个比特不够。教训：窗口要先算出来，再反推比特数。
- C（7）：7 是窗口的近似值（7.2），不是比特数；题目问的是“帧序号最少比特数”，不是“窗口大小”；把窗口值当成比特数，就答错了。教训：窗口和比特数是两步，先算窗口，再算比特。
- D（8）：8 是窗口的最小值，也不是比特数；比特数是 4，不是 8；把窗口值当成比特数，就答错了。教训：窗口是 8，比特是 4，两者不要混。

笔记本一句话： 第 35 题＋卫星链路窗口比特：先算发送时延 62.5 毫秒、往返 500 毫秒，利用率≥80% 得窗口≥8，Go-Back-N 下 2^n－1≥8 得 n＝4。

【图像】

滑动窗口：利用率≥80% → 窗口 W=8 → 序号比特 n=4

时间轴（一个周期）

连续发 W 帧：W × 62.5ms	等确认：500ms

发送时延 ＝ 1000字节×8 ÷ 128kbps ＝ 62.5ms；　RTT ＝ 2×250ms ＝ 500ms

利用率 ＝ W×62.5 ÷ (62.5＋500) ≥ 0.8　→　W ≥ 7.2　→　**W＝8**

Go-Back-N：$W \le 2^n - 1$　→　$2^n \ge 9$　→　**n＝4**（2^3＝8 不够，要减 1）

选择重传：$W \le 2^{n-1}$，W＝8 同样得 n＝4；两种协议结论一致。

坑：W＝8 是窗口，不是比特数！
先由利用率定窗口，再由窗口反推比特数 —— 答案 B（n＝4）

注意单位：1000 字节＝8000 比特；单向 250ms，RTT 取两倍。

滑动窗口利用率示意图：连发 W 帧耗时 W×发送时延，再等一个 RTT；利用率公式解出 W=8，反推序号比特 n=4

【定位】 计算机网络 · 第3章 数据链路层 · 滑动窗口协议与链路利用率

【易混对比】

- "窗口 W"和"序号比特 n"是两回事：W 是一次能连发几帧（由利用率算出来，本题是 8）；n 是序号用几个比特表示（由 W 反推出来，本题是 4）——题目问的是 n，不是 W。

- "发送时延"和"传播延迟"是两回事：发送时延是"把帧推出去要多久"，等于帧长除以速率（本题 62.5ms）；传播延迟是"信号在路上跑要多久"（本题单向 250ms，往返取两倍）。
- "回退 N 帧的 $W \le 2^n - 1$"和"选择重传的 $W \le 2^{n-1}$"是两回事：前者窗口占满除一个以外的全部序号，后者发送和接收各占一半——本题 W=8 时两种都算出 n=4，换了题要分开算。

利用率先定窗口大小，再由窗口反推序号比特数；窗口至少 8 时，序号至少 4 个比特。

【坑位】

把算出来的窗口值 8 直接当成比特数，选了 D。

利用率公式解出来的是"一次连发几帧"，即窗口 W；题目问的是"序号用几个比特"，即 n，两者差一步换算：$W \le 2^n - 1$。正确做法：算出 W=8 后再解 $2^n - 1 \ge 8$，$2^3 = 8$ 不够（还要减 1），$2^4 = 16$ 才够，所以 n=4。另外两处单位坑：1000 字节要乘 8 才是比特；单向 250ms，往返时间要乘 2。

【套路】

- 第一步，算两个时间：发送时延＝帧长÷速率（1000 字节＝8000 比特，8000÷128000＝62.5ms）；RTT＝2×单向传播延迟＝500ms。
- 第二步，列利用率不等式：$\frac{W \times 62.5}{62.5+500} \ge 0.8$，解得 $W \ge 7.2$，取整得 W=8。
- 第三步，由窗口反推比特：$2^n - 1 \ge 8$，即 $2^n \ge 9$，n 取 4。

先算时延，再列利用率定窗口，最后由窗口反推比特数。

【类比】

像用传送带运箱子：放一个箱子到带上要 1 分钟（发送时延），箱子走到对面、回执送回来要 8 分钟（往返时间）。一次只放 1 个箱子，带子大部分时间都在空转；一次连放 8 个，空转占比就小了。箱子上要贴编号（序号），8 个箱子同时在路上，编号至少 4 位二进制才够用。术语贴回去：箱子就是帧，连放个数就是窗口 W，编号位数就是序号比特 n。

【口诀】 利用率定窗口，窗口反推比特数；回退 N 帧是二的 n 次减一。

【考频】 「滑动窗口」在 2009–2022 年解析中出现 4 次 · 中频

第 36 题 · CSMA/CD 协议相关叙述的错误项（正确答案：B）

题干与选项：

下列关于 CSMA/CD 协议的叙述中，错误的是 。

- A. 边发送数据帧，边检测是否发生冲突
- B. 适用于无线网络，以实现无线链路共享
- C. 需要根据网络跨距和数据传输速率限定最小帧长
- D. 当信号传播延迟趋近 0 时，信道利用率趋近 100%

入手点： 先把 CSMA/CD 的适用范围记牢：它只用于有线以太网，不用于无线网络；无线网络用的是 CSMA/CA；再看哪条说法把适用范围说错了，那条就是错的。

专有名词从零解释：

- 先想“CSMA/CD”：载波监听多路访问／冲突检测；先听信道忙不忙，忙就不发，闲就发；发的时候还边发边听，听到冲突就停下来、等一会儿再发。
- 再想“CSMA/CA”：载波监听多路访问／冲突避免；先听信道忙不忙，忙就不发，闲也不马上发，先等一会儿、发个请求、收到允许再发；因为无线网络里冲突检测不出来，只能尽量避免冲突。
- 再想“冲突”：两个站点同时发数据，信号在信道上碰到一起，数据就坏了；有线网络里，站点发的时候能听到自己的信号被干扰了，所以能检测到冲突；无线网络里，站点发的时候听不到远处站点的信号，所以检测不到冲突。
- 再想“最小帧长”：CSMA/CD 要求帧不能太短；因为帧太短时，发送方还没发完，冲突信号还没传回来，发送方就以为没冲突、发完了；所以最小帧长要按网络跨距和传输速率来限定，保证发送方在发完之前能听到冲突。
- 再想“信道利用率”：信道在“传有效数据”的时间占总时间的比例；传播延迟越小，冲突检测越快、浪费的时间越少，利用率越高。

考查知识点：

- 先想 B 为什么错：B 说 CSMA/CD 适用于无线网络；无线网络里冲突检测不出来（站点发的时候听不到远处站点的信号），所以无线网络不用 CSMA/CD，用 CSMA/CA；CSMA/CD 只用于有线以太网；所以 B 错，就是答案。
- 再想 A 为什么对：CSMA/CD 的“CD”就是冲突检测，发送方边发边听，听到冲突就停；所以“边发送数据帧边检测是否冲突”是对的。
- 再想 C 为什么对：最小帧长确实要按网络跨距和传输速率来限定；跨距越大，冲突信号传回来越慢，帧就要越长，保证发送方发完之前能听到冲突；所以 C 对。
- 再想 D 为什么对：传播延迟趋近 0 时，冲突信号几乎马上就能传回来，浪费的时间趋近 0，信道几乎一直在传有效数据，利用率趋近 100%；所以 D 对。
- 拿生活例子验算：有线网络像在一个房间里说话，大家都能听到彼此，说话时听到有人同时说话，就停下来；无线网络像隔着几堵墙说话，听不到远处的人在说话，只能先问一句“我能说吗”，等对方说“能说”再说；前者是检测冲突，后者是避免冲突。
- 边界情况：CSMA/CD 的最小帧长公式是“最小帧长（比特）＝2×传播延迟×传输速率”；传播延迟越大、速率越高，最小帧长越大；本题只考“需按跨距和速率限定最小帧长”这个说法，不用算具体数值。

- 易混对子：不要把“CSMA/CD”和“CSMA/CA”搞混——前者是检测冲突、用于有线，后者是避免冲突、用于无线；名字里差一个字母，适用范围完全不同。

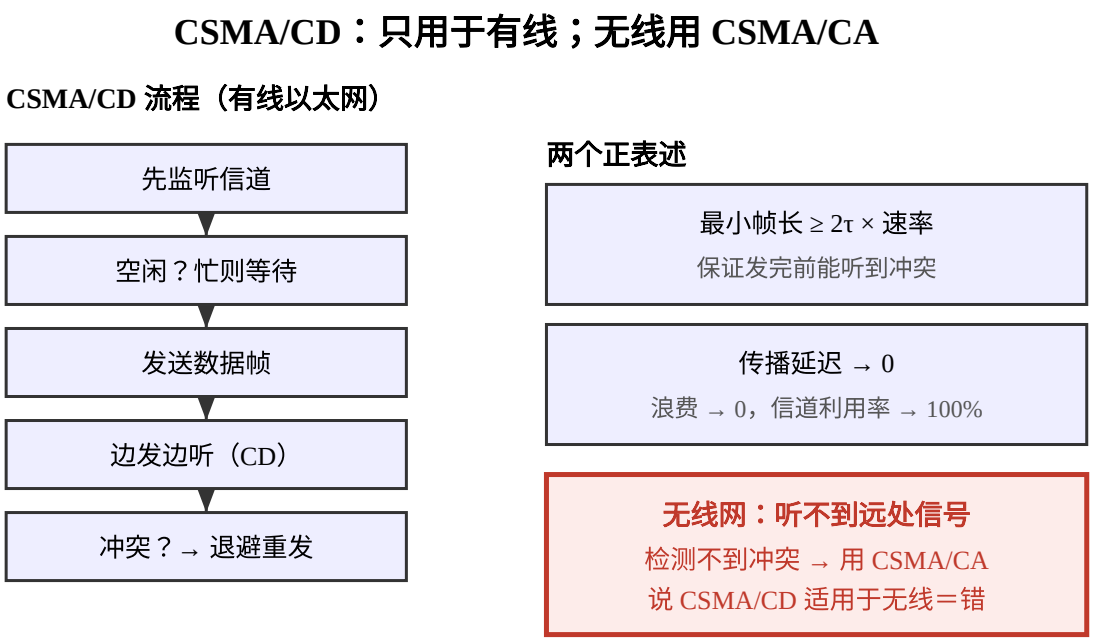
正确选项为什么对： 无线用 CSMA/CA，不用 CSMA/CD；B 说 CSMA/CD 适用于无线，所以 B 错，题目问错误项，选 B。

错误选项为什么错＋举例：

- A（边发送边检测冲突）：这就是 CSMA/CD 里“CD”的意思，发送方边发边听，听到冲突就停；这条说法是对的，不是答案。教训：问错误项时，对的说法要排除。
- C（需按跨距和速率限定最小帧长）：最小帧长确实和跨距、速率有关，跨距大、速率高，最小帧长就大；这条说法是对的，不是答案。教训：最小帧长不是随便定的。
- D（传播延迟趋近 0 时利用率趋近 100%）：传播延迟越小，冲突检测越快、浪费越少，利用率越高；延迟趋近 0，利用率趋近 100%；这条说法是对的，不是答案。教训：延迟和利用率是反向关系。

笔记本一句话： 第 36 题＋CSMA/CD 对 CSMA/CA：有线用检测（CD）、无线用避免（CA）；说 CSMA/CD 适用于无线就是错的。

【图像】



CSMA/CD 流程与争用期示意图：先听后发、边发边听、冲突退避；无线网用 CSMA/CA 对照，标出 B 为错误项

【定位】 计算机网络 · 第3章 数据链路层 · 介质访问控制（CSMA/CD）

【易混对比】

- "CSMA/CD"和"CSMA/CA"是两回事：CD 是冲突检测（边发边听，听到撞了就停），用于有线；CA 是冲突避免（先预约、尽量不撞），用于无线——名字差一个字母，适用范围完全相反。
- "能检测冲突"和"适用于无线"是两回事：无线站点发信号时听不到远处站点的信号，所以冲突检测不出来——不是"无线不想检测"，是"检测不到"。

- "最小帧长"和"最大帧长"是两回事：CSMA/CD 限定的是最小帧长（帧太短，发完时冲突信号还没传回来）；和"帧不能太长"没关系。

无线网检测不到冲突，所以不用 CSMA/CD 而用 CSMA/CA；凡说 CSMA/CD 适用于无线的都是错的。

【坑位】

被"无线链路共享"几个字带偏，觉得 CSMA/CD 既然管共享，无线也能用。

介质访问控制协议分"有线款"和"无线款"：有线能听到冲突，用检测（CD）；无线听不到冲突，用避免（CA）。正确做法：看到 CSMA/CD 先问"有线还是无线"——无线直接排除。

【套路】

- 第一步，先看适用范围：CSMA/CD 只用于有线以太网，无线用 CSMA/CA——适用范围说错的选项直接判错。
- 第二步，再核三条正表述：边发边听（CD 本意）、最小帧长由跨距和速率限定、传播延迟越小利用率越高。

先看有线无线定生死，再核三条正表述。

【类比】

有线网像一间敞亮的会议室：大家说话互相听得见，有人同时开口立刻能听出来，停下重说（检测）；无线网像隔着好几堵墙的房间：听不见隔壁有没有人在说，只能先喊一嗓子"我要说了啊"，等对方回"说吧"再说（避免）。术语贴回去：同时开口就是冲突，听出来就是冲突检测，喊一嗓子就是冲突避免。

【口诀】 有线检测 CD，无线避免 CA；无线说 CD，直接判它错。

【考频】 「CSMA/CD」在 2009–2022 年解析中出现 5 次 · 中频

第 37 题 · 交换机相关叙述的正确项（正确答案：A）

题干与选项：

下列关于交换机的叙述中，正确的是 。

- A. 以太网交换机本质上是一种多端口网桥
- B. 通过交换机互连的一组工作站构成一个冲突域
- C. 交换机每个端口所连网络构成一个独立的广播域
- D. 以太网交换机可实现采用不同网络层协议的网络互联

入手点： 先把交换机的两个隔离作用记牢：交换机隔离冲突域、不隔离广播域；再把“网桥、路由器”各自的活记牢：网桥连两个局域网段，路由器连不同网络；一条一条对，就能找到对的那条。

专有名词从零解释：

- 先想“以太网交换机”：一种多端口的设备，每个端口连一台设备或者一个网段；它按 MAC 地址转发帧，把帧只送到目标端口，不送到所有端口。
- 再想“网桥”：连接两个局域网段的设备，按 MAC 地址转发帧；交换机本质上就是多端口的网桥，端口多了，能连的设备就多了。
- 再想“冲突域”：一次只能有一个站点发数据的范围；在这个范围里，两个站点同时发就会冲突；交换机的每个端口是一个独立的冲突域，端口之间互不冲突。
- 再想“广播域”：广播帧能传到的范围；广播帧发出来，同一广播域里的所有设备都能收到；交换机不隔离广播域，所有端口都在同一个广播域里。
- 再想“路由器”：连接不同网络的设备，按 IP 地址转发分组；不同网络层协议的网络互联，是路由器的活，不是交换机的活。
- 再想“MAC 地址”：也叫物理地址，写在网卡里的地址，数据链路层用的地址；交换机转发帧时看的就是 MAC 地址。

考查知识点：

- 先想 A 为什么对：交换机本质上就是多端口网桥；网桥按 MAC 地址转发帧，交换机也是按 MAC 地址转发帧，只是端口多了；所以 A 对。
- 再想 B 为什么错：B 说“经交换机互连的一组工作站构成一个冲突域”；实际上交换机的每个端口是一个独立的冲突域，一组工作站如果分别连在不同端口上，就在不同的冲突域里，互不冲突；所以 B 错。
- 再想 C 为什么错：C 说“交换机每个端口所连网络构成一个独立广播域”；实际上交换机不隔离广播域，所有端口都在同一个广播域里，广播帧发出来，所有端口都能收到；所以 C 错。
- 再想 D 为什么错：D 说“交换机可实现不同网络层协议网络的互联”；不同网络的互联是路由器的活，路由器按 IP 地址转发；交换机只在数据链路层按 MAC 地址转发，不处理网络层协议的互联；所以 D 错。
- 拿生活例子验算：交换机像一栋楼里的电话总机，每个房间一部电话，总机把电话只接到目标房间，不接到所有房间；冲突域像每个房间里的谈话，房间之间互不干扰；广播域像楼里的广播，广播一响，所有房间都能听到；不同网络的互联像不同城市之间的电话，要通过长途局（路由器）转接，不是楼里总机的活。
- 边界情况：VLAN 技术可以把交换机的端口划分成不同的广播域，但那是 VLAN 的功能，不是交换机本身的默认功能；本题不考 VLAN，只考交换机本身的默认行为。
- 易混对子：不要把“隔离冲突域”和“隔离广播域”搞混——交换机隔离前者、不隔离后者；也不要把“交换机”和“路由器”的活搞混——交换机按 MAC 地址转发、连同一网络，路由器按 IP 地址转发、连不同网络。

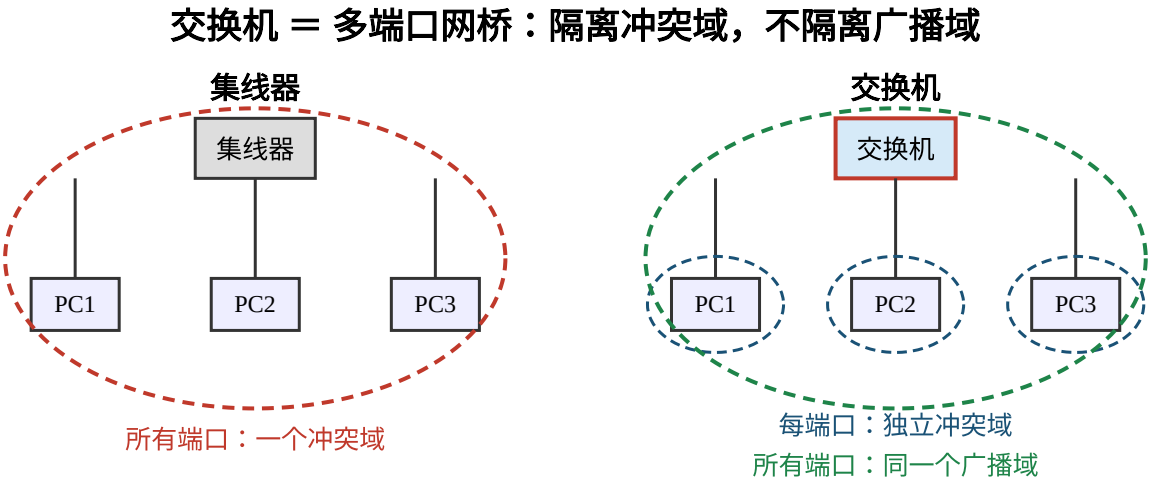
正确选项为什么对： 交换机本质上是多端口网桥，按 MAC 地址转发，选 A。

错误选项为什么错＋举例：

- B（互连的一组工作站构成一个冲突域）：交换机每个端口一个冲突域，不同端口的工作站不在同一个冲突域里；如果所有工作站都连在同一个集线器上，那才是一个冲突域。教训：交换机隔离冲突域。
- C（每个端口所连网络构成独立广播域）：交换机不隔离广播域，所有端口同一广播域；要隔离广播域，要用路由器或者 VLAN。教训：交换机不隔离广播域。
- D（可实现不同网络层协议网络的互联）：不同网络的互联是路由器的活，交换机只在数据链路层转发；交换机不处理网络层协议的互联。教训：交换机和路由器的活要分清。

笔记本一句话： 第 37 题＋交换机：本质是多端口网桥；隔离冲突域、不隔离广播域；不同网络互联是路由器的活。

【图像】



交换机与集线器对照图：交换机每端口独立冲突域、整体一个广播域；集线器整体一个冲突域

【定位】 计算机网络 · 第3章 数据链路层 · 网桥与交换机

【易混对比】

- "隔离冲突域"和"隔离广播域"是两回事：交换机只干前者（每端口一个独立冲突域），不干后者（所有端口还在同一个广播域里）。
- "交换机"和"路由器"是两回事：交换机按 MAC 地址转发、连的是同一个网络；路由器按 IP 地址转发、连的是不同网络——"不同网络互联"是路由器的活。
- "交换机"和"集线器"是两回事：集线器所有端口挤在一个冲突域里（一人说话全员禁言）；交换机每端口独立冲突域（各说各的，互不干扰）。

交换机就是多端口网桥：隔离冲突域、不隔离广播域；跨网络互联找路由器。

【坑位】

把"每端口独立"同时套到冲突域和广播域上，选了 C。

"独立"只对了一半：冲突域确实每端口独立，但广播域不独立——广播帧从一个端口进来，会被送到所有端口。正确做法：记"交换机隔离冲突、不隔离广播"，两个词分开记。

【类比】

交换机像一栋办公楼的电话总机：每个房间一部电话，总机把电话只接到目标房间（按房间号转接）；每个房间里说话互不干扰（每端口独立冲突域）；但楼里的广播一响，所有房间都听得见（同一个广播域）。不同城市之间打电话要走长途局，不是楼里总机的活。术语贴回去：房间号就是 MAC 地址，长途局就是路由器。

【口诀】 交换机是多口桥，冲突隔离广播到；跨网互联找路由，MAC 转发它独到。

【考频】 「多端口网桥」在 2009–2022 年解析中出现 14 次 · 高频

第 38 题 · 路由器按路由表为指定目的地址 IP 分组选择转发接口（正确答案：C）

题干与选项：

某路由器的路由表如下表所示：

| 目的网络 | 下一跳 | 接口 |

|---|---|---|

| 169.96.40.0/23 | 176.1.1.1 | S1 |

| 169.96.40.0/25 | 176.2.2.2 | S2 |

| 169.96.40.0/27 | 176.3.3.3 | S3 |

| 0.0.0.0/0 | 176.4.4.4 | S4 |

若路由器收到一个目的地址为 169.96.40.5 的 IP 分组，则转发该 IP 分组的接口是（ ）。

- A. S1
- B. S2
- C. S3
- D. S4

> 注：原卷路由表后两行接口印作 E3、E4（选项写作 S3、S4），属原卷印制不一致；表格按源一转录，结论以最长前缀匹配为准。

入手点： 先把目的地址和每条路由的目的网络一位一位比，看哪些路由匹配；匹配的路由里，选前缀最长的那一条；前缀最长，就是网络号最具体的那一条。

专有名词从零解释：

- 先想“路由表”：路由器里的一张表，每行写着“去哪个网络、从哪个接口走、下一跳是谁”；路由器收到分组，就查这张表决定从哪个接口转发。
- 再想“目的网络”：比如 169.96.40.0/23，斜杠后面的数字是前缀长度，表示前 23 位是网络号；前缀越长，网络范围越小、越具体。
- 再想“最长前缀匹配”：目的地址可能和好几条路由都匹配，路由器选前缀最长的那一条；因为前缀最长的路由最具体，最可能就是正确的去向。
- 再想“默认路由”：0.0.0.0/0，前缀长度是 0，和所有地址都匹配；如果没有其他路由匹配，就走默认路由；默认路由是最后的兜底。

考查知识点：

- 先想路由表里的四行：169.96.40.0/23、下一跳 176.1.1.1、接口 S1；169.96.40.0/25、下一跳 176.2.2.2、接口 S2；169.96.40.0/27、下一跳 176.3.3.3、表中接口印作 E3；0.0.0.0/0、下一跳 176.4.4.4、表中接口印作 E4。
- 再想目的地址 169.96.40.5 和每条路由比：/23 的网络范围是 169.96.40.0 到 169.96.41.255，40.5 在里面，匹配；/25 的网络范围是 169.96.40.0 到 169.96.40.127，40.5 在里面，匹配；/27 的网络范围是 169.96.40.0 到 169.96.40.31，40.5 在里面，匹配；默认路由 0.0.0.0/0 和所有地址都匹配。
- 再想最长前缀：匹配的四条里，前缀最长的是 /27（27 比 25、23、0 都长）；所以选第三行，接口是选项里的 S3，就是 C。

- 再想为什么不是 S1、S2：S1 对应 /23，S2 对应 /25，前缀都比 /27 短；最长前缀匹配选最长的，不选短的；所以 S1、S2 都不对。
- 再想为什么不是默认路由：默认路由是兜底的，只有没有其他路由匹配时才走；本题有三条具体路由匹配，默认路由排在最后，不选它。
- 拿生活例子验算：寄信时，地址越具体越好；“某市某区某街道某号”比“某市某区”更具体，邮递员按最具体的地址投递；路由表里的前缀长度就是“具体程度”，最长的最具体。
- 边界情况：如果目的地址和任何具体路由都不匹配，才走默认路由；如果路由表里没有默认路由、又没有匹配的路由，分组就被丢弃，并返回差错报文。

> 注意：原卷路由表后两行接口印作 E3、E4，选项写作 S3、S4，属原卷印制不一致；按最长前缀匹配的结论，第三行对应的选项就是 S3，答案以 C 为准，不影响结论。

正确选项为什么对： 169.96.40.5 与 /23、/25、/27 都匹配，最长是 /27，对应选项 S3，选 C。

错误选项为什么错＋举例：

- A（S1）：S1 对应 /23，前缀比 /27 短；最长前缀匹配选最长的，不选短的。教训：匹配的路由里选前缀最长的。
- B（S2）：S2 对应 /25，前缀比 /27 短；同样不是最长的。教训：/25 比 /27 短，排在后面。
- D（S4）：S4 对应默认路由 0.0.0.0/0，前缀长度是 0，是最短的；默认路由是兜底的，有具体路由匹配时不走默认路由。教训：默认路由排在最后。

笔记本一句话： 第 38 题＋最长前缀匹配：目的地址和几条路由都匹配时，选前缀最长的那条；本题 /27 最长，选 S3。

【图像】



最长前缀匹配查表示意图：169.96.40.5 同时匹配 /23、/25、/27 三行，选前缀最长的 /27 走 S3

【定位】 计算机网络 · 第4章 网络层 · 路由选择（最长前缀匹配）

【易混对比】

- "匹配"和"选中"是两回事：目的地址能和好几条路由都匹配，但路由器只选前缀最长的那一条——匹配是入围，选中是夺冠。
- "前缀长"和"网络大"是两回事：前缀越长，网络范围越小、越具体（/27 只管 32 个地址，/23 管 512 个）——"长"的是前缀，"小"的是网络。
- "默认路由"和"具体路由"是两回事：0.0.0.0/0 和谁都匹配，是兜底的；只要有具体路由匹配，默认路由就轮不上。

都匹配时选前缀最长的：169.96.40.5 命中 /23、/25、/27，/27 最具体，走 S3。

【坑位】

数出"169.96.40.5 在 /23 的范围里"就停了，选了 S1。

最长前缀匹配要求把所有匹配的行都列出来再比，不能"先中先得"。正确做法：/23（169.96.40.0–169.96.41.255）、/25（.40.0–.40.127）、/27（.40.0–.40.31）三行都匹配，比前缀长度 23＜25＜27，取最长的 /27。

【套路】

- 第一步，列出所有匹配行：把目的地址和每行的目的网络比，/23、/25、/27 三行都含 169.96.40.5，默认路由天然匹配。
- 第二步，比前缀长度：27＞25＞23＞0，前缀最长的是 /27。
- 第三步，取对应接口：/27 对应选项 S3。

先列全匹配，再比前缀长，取最长行对应的接口。

【类比】

像寄快递填地址：面单上同时写着"某市""某市某区""某市某区某街道"，分拣员按最具体的"某区某街道"分拣，不会只看到"某市"就扔进市级中转。术语贴回去：具体程度就是前缀长度，最具体的地址就是最长前缀匹配。

【口诀】 路由匹配一大把，前缀最长才说话。

【考频】 「最长前缀匹配」在 2009–2022 年解析中出现 2 次 · 低频

第 39 题 · TCP 连接建立成功后经过 4 个 RTT 时的发送窗口大小（正确答案：A）

题干与选项：

主机甲和主机乙新建一个 TCP 连接，甲的拥塞控制初始阈值为 32KB，甲向乙始终以 MSS=1KB 大小的段发送数据，并一直有数据发送；乙为该连接分配 16KB 接收缓存，并对每个数据段进行确认，忽略段传输延迟。若乙收到的数据全部存入缓存，不被取走，则甲从连接建立成功时刻起，未发生超时的情况下，经过 4 个 RTT 后，甲的发送窗口是 。

- A. 1KB
- B. 8KB
- C. 16KB
- D. 32KB

入手点： 先把两个窗口分开算：拥塞窗口按慢启动翻倍，接收窗口按“缓存还剩多少”算；发送窗口是两者的最小值；4 个 RTT 里，拥塞窗口还没到阈值，全程都是慢启动。

专有名词从零解释：

- 先想“慢启动”：TCP 刚建立连接时，拥塞窗口从 1 个 MSS 开始，每经过一个 RTT，拥塞窗口翻一倍；像汽车起步时慢慢加速，速度越来越快。
- 再想“慢启动阈值”：也叫 ssthresh，是慢启动和拥塞避免的分界线；拥塞窗口小于阈值时，每 RTT 翻倍；大于阈值时，每 RTT 只加 1 个 MSS，增速变慢。
- 再想“拥塞窗口”：也叫 cwnd，发送方根据网络拥塞程度自己定的窗口；窗口越大，一次能发的数据越多。
- 再想“接收窗口”：也叫 rwnd，接收方告诉发送方的窗口；等于接收缓存还剩多少空间；接收方缓存满了，发送方就不能再发了。
- 再想“发送窗口”：也叫 swnd，发送方实际能发的窗口；等于拥塞窗口和接收窗口里较小的那一个；两个窗口哪个小，就按哪个发。
- 再想“MSS”：最大报文段长度，一次能发的数据段的最大字节数；本题 MSS 是 1KB。
- 再想“RTT”：往返时间，发一帧、等确认回来的时间；每经过一个 RTT，拥塞窗口翻一倍。

考查知识点：

- 先想初始值：拥塞窗口初始值是 1 个 MSS，就是 1KB；接收缓存是 16KB，一开始接收窗口就是 16KB。
- 再想第 1 个 RTT：发 1KB，收到确认后，拥塞窗口翻倍成 2KB；接收缓存里存了 1KB，还剩 15KB，接收窗口是 15KB。
- 再想第 2 个 RTT：发 2KB，收到确认后，拥塞窗口翻倍成 4KB；接收缓存里存了 1＋2＝3KB，还剩 13KB，接收窗口是 13KB。
- 再想第 3 个 RTT：发 4KB，收到确认后，拥塞窗口翻倍成 8KB；接收缓存里存了 3＋4＝7KB，还剩 9KB，接收窗口是 9KB。
- 再想第 4 个 RTT：发 8KB，收到确认后，拥塞窗口翻倍成 16KB；接收缓存里存了 7＋8＝15KB，还剩 1KB，接收窗口是 1KB。
- 再想 4 个 RTT 后的发送窗口：拥塞窗口 16KB，接收窗口 1KB，取较小的，就是 1KB，就是 A。

- 再想为什么不是 32KB：32KB 是慢启动阈值，不是初始窗口，也不是 4 个 RTT 后的窗口；4 个 RTT 内拥塞窗口从 1KB 翻倍到 16KB，还没到 32KB，全程都是慢启动；把阈值当成初始窗口或者当成发送窗口，都是错的。
- 拿生活例子验算：水桶（接收缓存）一共 16 升，往里倒水，倒 1 升、2 升、4 升、8 升，桶里一共 15 升，还剩 1 升；下次最多只能再倒 1 升，不然桶就满了；发送窗口就是“还能倒多少”，也就是 1 升。
- 边界情况：如果接收方取走了数据，接收窗口就会变大，发送窗口也可能变大；本题说“收到的数据全部存入缓存且不被取走”，所以接收窗口只会越来越小，最后剩 1KB。
- 易混对子：不要把“拥塞窗口”和“发送窗口”搞混——发送窗口是拥塞窗口和接收窗口里较小的那一个；本题拥塞窗口 16KB、接收窗口 1KB，发送窗口是 1KB，不是 16KB；也不要把“慢启动阈值 32KB”当成“初始窗口”——初始窗口是 1KB，阈值是分界线。

正确选项为什么对： 4 个 RTT 后拥塞窗口 16KB、接收窗口 1KB，取较小值，发送窗口是 1KB，选 A。

错误选项为什么错＋举例：

- B（8KB）：8KB 是第 4 个 RTT 里发的数据量，不是 4 个 RTT 后的发送窗口；4 个 RTT 后，拥塞窗口已经翻倍到 16KB，接收窗口剩 1KB，发送窗口是 1KB。教训：第 4 个 RTT 发多少，不等于 4 个 RTT 后的窗口。
- C（16KB）：16KB 是 4 个 RTT 后的拥塞窗口，不是发送窗口；发送窗口要取拥塞窗口和接收窗口里较小的那一个，接收窗口只剩 1KB，所以发送窗口是 1KB。教训：发送窗口取两者最小值。
- D（32KB）：32KB 是慢启动阈值，不是初始窗口，也不是 4 个 RTT 后的窗口；4 个 RTT 内拥塞窗口还没到阈值，全程慢启动；把阈值当成窗口，就答错了。教训：阈值是分界线，不是窗口值。

笔记本一句话： 第 39 题＋TCP 发送窗口：发送窗口＝拥塞窗口和接收窗口里较小的那一个；本题 4 个 RTT 后是 min(16KB,1KB)＝1KB；32KB 只是阈值。

【图像】

慢启动 4 个 RTT：拥塞窗口翻倍，接收窗口见底，发送窗口取小

初始：拥塞窗口＝1KB（1 个 MSS）；接收缓存＝16KB；阈值＝32KB（4 轮内到不了，全程慢启动）

RTT 轮次	本轮发送	拥塞窗口	缓存已存	接收窗口
1	1KB	2KB	1KB	15KB
2	2KB	4KB	3KB	13KB
3	4KB	8KB	7KB	9KB
4	8KB	16KB	15KB	1KB

发送窗口 ＝ min(拥塞 16KB，接收 1KB) ＝ 1KB

32KB 只是慢启动阈值，不是窗口；答案 A

坑：算出拥塞窗口 16KB 就停 → 会误选 C；只看第 4 轮发了 8KB → 会误选 B。

数据"只进不出"：接收窗口 ＝ 16KB − 已存；已存 ＝ 1＋2＋4＋8 ＝ 15KB。

TCP 慢启动 4 个 RTT 示意图：拥塞窗口 1→2→4→8→16KB，接收窗口 16→15→13→9→1KB，发送窗口取小得 1KB

【定位】 计算机网络 · 第5章 传输层 · TCP 拥塞控制（慢启动）

【易混对比】

- "拥塞窗口"和"发送窗口"是两回事：拥塞窗口是发送方自己估的（本题 4 个 RTT 后是 16KB）；发送窗口是"拥塞窗口和接收窗口里较小的那个"（本题是 1KB）——题目问的是发送窗口。
- "慢启动阈值"和"初始窗口"是两回事：阈值 32KB 是"翻倍"和"线性增长"的分界线；初始窗口是 1 个 MSS（本题 1KB）——阈值不是起点。
- "第 4 个 RTT 发了多少"和"4 个 RTT 后的窗口"是两回事：前者是第 4 轮发出去的量（8KB），后者是第 4 轮结束时的状态（拥塞 16KB、接收 1KB、发送 1KB）。

发送窗口取两者之小：4 个 RTT 后拥塞窗口 16KB、接收窗口只剩 1KB，发送窗口是 1KB。

【坑位】

算出拥塞窗口 16KB 就直接选了 C，忘了接收窗口。

接收缓存 16KB 且数据"只进不出"，4 轮下来存了 1+2+4+8＝15KB，只剩 1KB。发送窗口＝min(拥塞窗口，接收窗口)＝min(16KB，1KB)＝1KB。正确做法：两个窗口分开算，最后取小——只算拥塞窗口是半道题。

【套路】

- 第一步，算拥塞窗口：慢启动每 RTT 翻倍，1→2→4→8→16KB（4 个 RTT 后 16KB，未到阈值 32KB，全程慢启动）。
- 第二步，算接收窗口：缓存 16KB 减已存数据，已存 1+2+4+8＝15KB，还剩 1KB。
- 第三步，取小：发送窗口＝min(16KB，1KB)＝1KB。

拥塞接收分开算，发送窗口取较小。

【类比】

像往一个 16 升的水桶倒水：倒水的人越倒越快（第 1 次 1 升、第 2 次 2 升、第 3 次 4 升、第 4 次 8 升），但桶只有 16 升，4 次后桶里 15 升、只剩 1 升空位——下次最多只能再倒 1 升，不然就溢出来了。术语贴回去：倒水速度就是拥塞窗口，桶的空位就是接收窗口，实际能倒的就是发送窗口。

【口诀】 慢启动翻倍涨，接收窗口看余量；发送窗口取小者，阈值只是分界线。

【考频】 「拥塞窗口和接收窗口」在 2009–2022 年解析中出现 6 次 · 中频

第 40 题 · 浏览器发出的 HTTP 请求报文相关叙述的错误项（正确答案：C）

题干与选项：

某浏览器发出的 HTTP 请求报文如下：

GET /index.html HTTP/1.1

Host: www.test.edu.cn

Connection: Close

Cookie: 123456

下列叙述中，错误的是 。

- A. 该浏览器请求浏览 index.html
- B. Index.html 存放在 www.test.edu.cn 上
- C. 该浏览器请求使用持续连接
- D. 该浏览器曾经浏览过 www.test.edu.cn

入手点： 一行一行读报文：请求行说“要什么”，Host 头说“去哪个主机”，Connection 头说“连不连续”，Cookie 头说“以前来没来过”；哪条说法和报文对不上，那条就是错的。

专有名词从零解释：

- 先想“HTTP 请求报文”：浏览器向服务器发请求时写的一段文字；第一行是请求行，后面是头部行，最后是空行和实体。
- 再想“请求行”：比如 GET /index.html HTTP/1.1；GET 是方法，表示“要取东西”；/index.html 是要取的东西的路径；HTTP/1.1 是用的 HTTP 版本。
- 再想“Host 头”：比如 Host: www.test.edu.cn；告诉服务器“要找的是哪个主机上的东西”；一台服务器上可能有多个网站，Host 头用来区分。
- 再想“Connection 头”：比如 Connection: Close；告诉服务器“这次请求完就断开连接，不保持”；如果写的是 Keep-Alive，就是“保持连接，下次请求还用这条连接”。
- 再想“持续连接”：也叫持久连接，一条 TCP 连接上可以发多个请求、收多个响应；不用每个请求都重新建立连接，省时间。
- 再想“非持续连接”：一条 TCP 连接上只发一个请求、收一个响应，完事就断开；下次请求要重新建立连接。
- 再想“Cookie 头”：比如 Cookie: 123456；浏览器以前来过这个网站时，服务器给浏览器种了一个 Cookie，浏览器下次再来时把 Cookie 带上，服务器就知道“这个浏览器以前来过”。

考查知识点：

- 先想 A 为什么对：请求行是 GET /index.html，GET 是“要取东西”，/index.html 是要取的东西；所以“请求浏览 index.html”是对的。
- 再想 B 为什么对：Host 头是 www.test.edu.cn，说明要找的东西在这个主机上；所以“Index.html 存放在 www.test.edu.cn 上”是对的。
- 再想 C 为什么错：Connection 头是 Close，Close 的意思就是“非持续连接”，请求完就断开；C 说“请求使用持续连接”，和 Close 正好相反；所以 C 错，就是答案。
- 再想 D 为什么对：Cookie 头是 123456，Cookie 是服务器以前种下的；浏览器能带上 Cookie，说明它以前来过这个网站、服务器给它种过 Cookie；所以“曾经浏览过 www.test.edu.cn”是对的。

- 拿生活例子验算：去饭店点菜，点菜单上写“要一份宫保鸡丁”（请求行），写“去二楼包间”（Host 头），写“吃完就走、不续桌”（Connection: Close），服务员一看你手里的会员卡（Cookie），就知道你以前来过。
- 边界情况：HTTP/1.1 默认是持续连接，但 Connection: Close 会明确告诉服务器“这次不用持续连接”；报文里写了 Close，就按 Close 走，不按默认走。
- 易混对子：不要把“Connection: Close”和“持续连接”搞混——Close 就是不持续，Keep-Alive 才是持续；也不要把“Cookie”当成“第一次来”——第一次来时浏览器手里没有 Cookie，带上 Cookie 就说明以前来过。

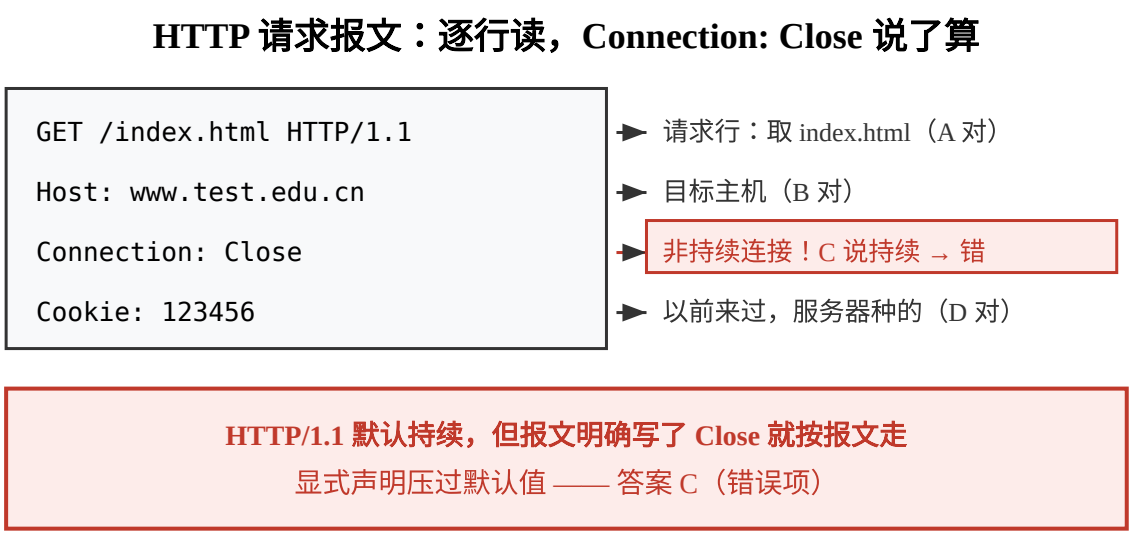
正确选项为什么对： Connection: Close 是非持续连接，C 说持续连接，和报文相反，所以 C 错，题目问错误项，选 C。

错误选项为什么错＋举例：

- A（请求浏览 index.html）：请求行 GET /index.html 就是“要取 index.html”；这条说法是对的，不是答案。教训：请求行里的路径就是要取的东西。
- B（Index.html 存放在 www.test.edu.cn 上）：Host 头就是这个主机；这条说法是对的，不是答案。教训：Host 头告诉服务器要找哪个主机。
- D（曾经浏览过 www.test.edu.cn）：Cookie 是以前种下的，能带上 Cookie 就说明以前来过；这条说法是对的，不是答案。教训：Cookie 是“以前来过”的证据。

笔记本一句话： 第 40 题＋HTTP 请求报文：GET 行是要取的东西，Host 是主机，Connection: Close 是非持续连接，Cookie 说明以前来过；说 Close 是持续连接就是错的。

【图像】



Cookie 是"以前来过"的证据：第一次来的浏览器手里没有 Cookie 可带。

请求行给"取什么"，Host 给"去哪台主机"，各管一摊，别串。

HTTP 请求报文结构图：请求行、Host、Connection: Close、Cookie 四行逐行标注含义，Close 处标红

【定位】 计算机网络 · 第6章 应用层 · HTTP 请求报文

【易混对比】

- "Connection: Close"和"持续连接"是两回事：Close 就是"这次完事就断开"（非持续），Keep-Alive 才是持续——报文里写了 Close，就不按 HTTP/1.1 的默认持续走。
- "Host 头"和"请求行里的路径"是两回事：Host 说"去哪台主机"，路径说"取哪个文件"——主机和文件是两个空位。
- "带着 Cookie"和"第一次来"是两回事：Cookie 是服务器上次种下的；浏览器能带上它，说明以前来过——第一次来的浏览器手里没有 Cookie。

报文写 Close 就是非持续连接；说它是持续连接的选项，和报文反着来，直接判错。

【坑位】

凭"HTTP/1.1 默认持续连接"的印象选了 C，没看报文里的 Connection 行。

默认归默认，报文里明确写了 Connection: Close 就按报文走——显式声明压过默认值。正确做法：逐行读报文，Connection 行写什么，连接就是什么。

【类比】

像去饭店点菜：菜单第一行写"要一份宫保鸡丁"（请求行），备注"送到二楼包间"（Host 头）、"吃完就走不续桌"（Connection: Close），服务员看到你掏出会员卡（Cookie），就知道你是老顾客。说"这桌要续"（持续连接），和纸条上"吃完就走"正好反了。术语贴回去：菜单就是请求报文，会员卡就是 Cookie。

【口诀】 请求行看取啥，Host 看主机，Close 不持续，Cookie 证来过。

【考频】 「Cookie」在 2009–2022 年解析中出现 1 次 · 低频
